TEKsystems Global Services | WORKFORCE DEVELOPMENT

GitHub Copilot for Developers

PARTICIPANT GUIDE

LOGISTICS

Class Hours:

- Instructor will set class start and end times.
- There will be regular breaks in class.

Telecommunication:

- Turn off or set electronic devices to silent (not vibrate)
- Reading or attending to devices can be distracting to other students
- Try to delay until breaks or after class

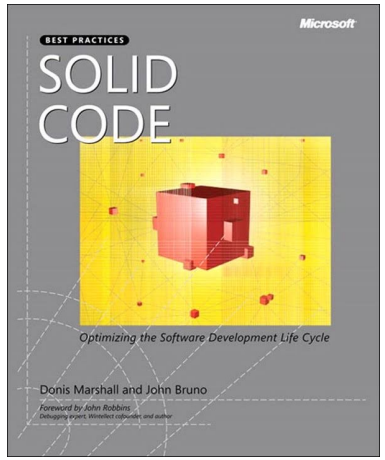
Miscellaneous:

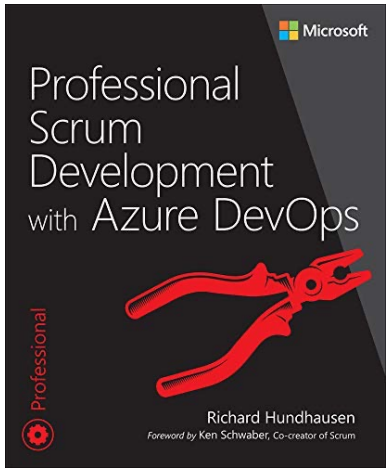
- Courseware
- Bathroom
- Fire drills

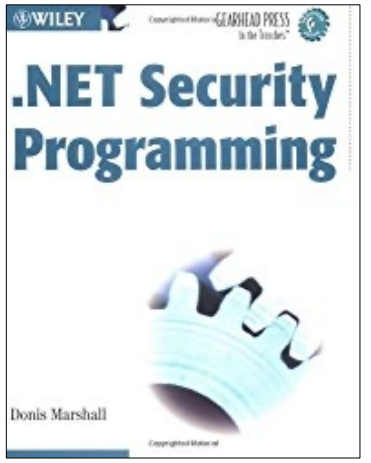
© 2026 by Innovation In Software Corporation

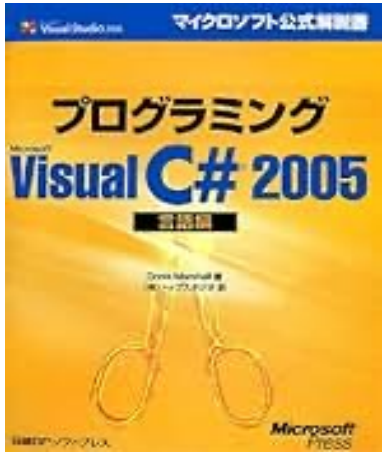
DONIS MARSHALL

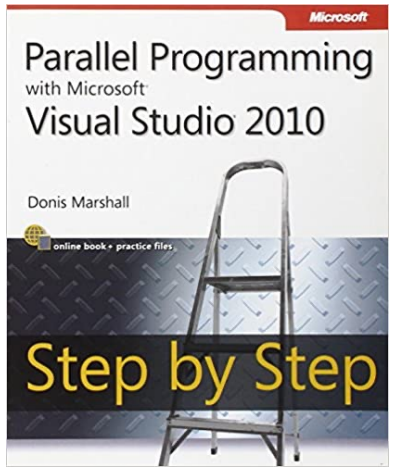
Security Professional
Microsoft MVP
Microsoft Certified
Author

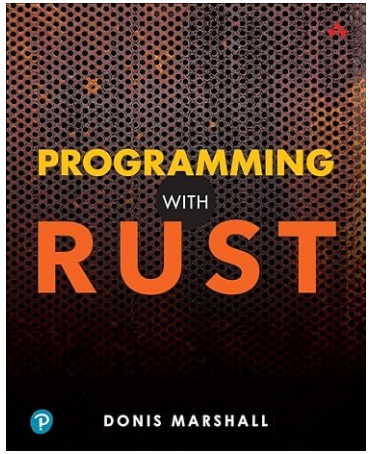












© 2026 by Innovation In Software Corporation

INTRODUCE YOURSELF

Time to introduce yourself:

- Name
- What is your role in the organization
- Indicate GitHub Copilot and VS Code experience

© 2026 by Innovation In Software Corporation

COPILOT

In technology, the term copilot refers to a digital assistant that augments a user's capabilities by providing real-time guidance, automation, and code.

It doesn't replace human judgment but instead supplements it, offering context-aware insights and actionable recommendations.

Just as a copilot in aviation supports the pilot with navigation and decision-making, a technology copilot helps professionals carry out complex tasks more efficiently.

© 2026 Innovation In Software Corporation

GITHUB COPILOT – 2

GitHub Copilot is an AI-powered coding assistant that integrates into development environments such as Visual Studio Code (VS Code). It provides real-time code suggestions, entire functions, or hints as a developer types.

Copilot's strength lies in its ability to understand the context of the current codebase and produce recommendations that feel relevant, saving developers time and effort.

© 2026 Innovation In Software Corporation

POP QUIZ: LINES OF CODE

How many lines of code can the average developer enter before creating a logical or syntactical error?

10 MINUTES

© 2026 by Innovation In Software Corporation

AI ASSISTANT

AI Assistants in tools like VS Code function much like pair programming, where two developers work together at one computer. In traditional pair programming, one person writes the code while the other reviews, suggests improvements, and helps catch mistakes.

With an AI assistant, the “second developer” is virtual—it suggests code, completes functions, helps with syntax, and offers ideas based on patterns it has learned.

© 2026 Innovation In Software Corporation

AI ASSISTANT - 2

However, just like working with a junior teammate, the senior developer (you) remains in charge. The AI may not understand the project's goals, constraints, or business logic—it predicts likely code based on various context. This means you must review, validate, and decide whether suggestions are correct.

© 2026 Innovation In Software Corporation

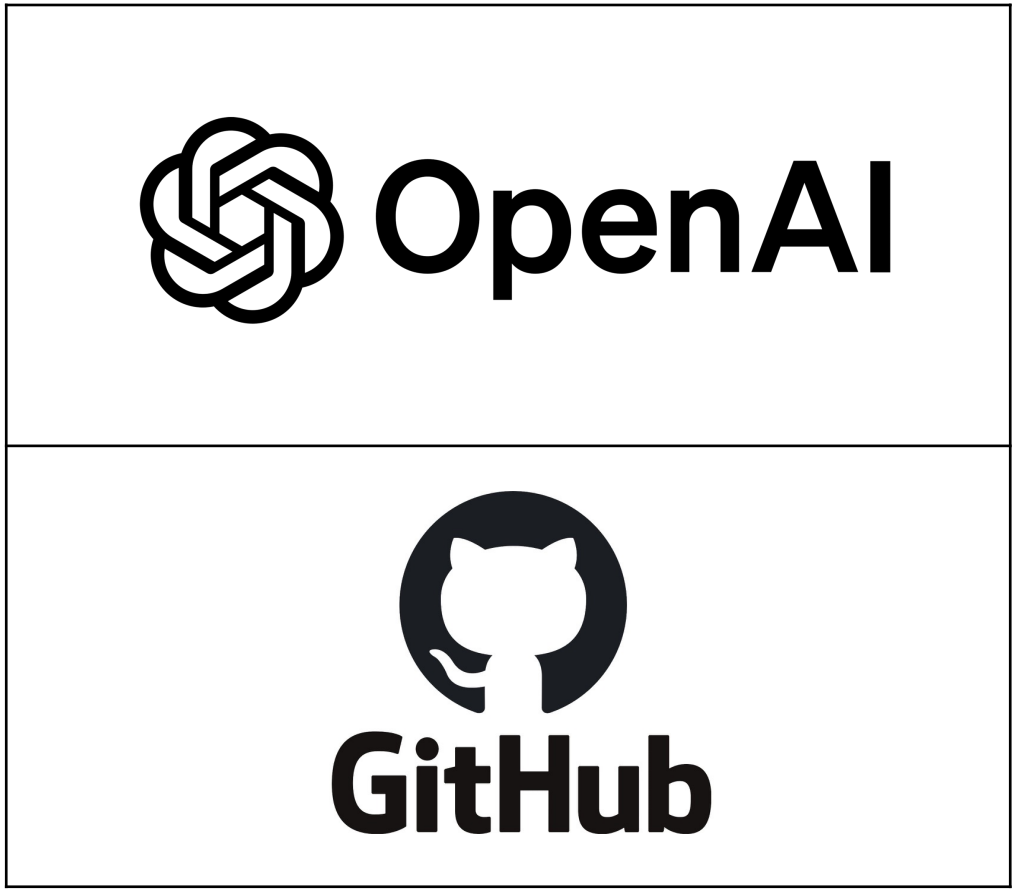
WHO'S DRIVING?

Degradation in Spatial Knowledge Acquisition When Using Automatic Navigation Systems

https://link.springer.com/chapter/10.1007/978-3-540-74788-8_15

© 2026 by Innovation In Software Corporation

GITHUB AND OPENAI



GitHub Copilot emerged in 2021 from a collaboration between GitHub and OpenAI, combining GitHub's massive code base with OpenAI's language model expertise. Built on the Codex model, a GPT-3 descendant fine-tuned for coding - it was designed for programming tasks.

The partnership was strategic: GitHub provided billions of code examples, and OpenAI supplied generative AI research, becoming one of the first large-scale commercial uses of generative AI in software development.

© 2026 Innovation In Software Corporation

GENERATIVE AI

Generative AI is a type of artificial intelligence that doesn't just analyze data, but creates new things—like text, images, music, or code—based on patterns it has learned. Instead of giving fixed answers, it generates original output that looks like it was made by a person.

Analogy: Generative AI is like an incredibly fast assistant who has absorbed knowledge from a vast library—and can go beyond simply recalling what it has read by combining and extending ideas in new ways. You describe what you need, and it generates suggestions—saving you time and effort.

© 2026 by Innovation In Software Corporation

GENERATIVE AI - 2

Generative AI in GitHub Copilot works by learning from patterns found in millions of code repositories and other sources, then creating new code best fits your prompt. It doesn't just simply replace existing code but predicts what might come next, even if new and original, based on context.

Simple example: Suppose you start typing in Python:

```
def add_numbers(a, b):
```

Then pause and wait momentarily.

© 2026 by Innovation In Software Corporation

GENERATIVE AI - 3

Copilot might automatically suggest:

```
    return a + b
```

It “predicted” that since you named the function add_numbers with two inputs, you probably want to return their sum. That’s generative AI at work—it takes what you’ve written, combines it with what it has learned previously, and generates new code for you.

© 2026 by Innovation In Software Corporation

POP QUIZ: GITHUB COPILOT

During code completion, why might you pause momentarily?

10 MINUTES

© 2026 by Innovation In Software Corporation

LLM

An LLM (Large Language Model) is like a scholar who has read a giant library. It was trained on billions of white papers, books, websites, and code samples, and it absorbed the patterns in them rather than storing the documents themselves. Because it has so many parameters (its internal “memory capacity”), it can generate flexible, detailed answers across many topics.

An SLM (Small Language Model) is more like a pocket guide. It has far fewer parameters, so it holds less general knowledge and is often tuned for smaller, specific tasks. It runs faster and uses less computing power, but it will not know as much or handle complex reasoning as well.

© 2026 Innovation In Software Corporation

LLM - SOURCES

Many assume GitHub Copilot is trained solely on GitHub repositories, but the reality is broader. Sources include:

- Public GitHub repositories
- Open-source projects hosted outside GitHub
- Programming Q&A sites
- Technical documentation and manuals
- Educational resources such as tutorials and coding books

This variety ensures Copilot can generalize across multiple programming domains and coding conventions, rather than being biased toward a single platform or community.

© 2026 by Innovation In Software Corporation

LLM – STALENESS

The LLM's training data is not continuously updated. For that reason, it may not reflect real-time changes. This means it may recommend outdated functions, deprecated APIs, or older coding patterns.

To address this, developers should cross-check suggestions with up-to-date references such as official documentation, vendor resources, or active community discussions. Validating against current sources helps ensure projects remain aligned with modern standards and best practice

© 2026 Innovation In Software Corporation

CODE REVIEW

Large Language Models (LLMs) are powerful assistants, but their outputs require human review. They can introduce errors or risks that developers must identify and correct.

Here are some reasons:

- Hallucinations – The model may generate code that looks correct but is functionally wrong or references non-existent libraries or APIs.
- Bias – Training data may contain biased assumptions that influence logic, naming, or implementation choices.

© 2026 by Innovation In Software Corporation

CODE REVIEW - 2

- Staleness – Training data is not continuously updated, which can result in outdated functions, deprecated APIs, or older coding patterns.
- Security Risks – The model may suggest insecure patterns, weak validation, improper error handling, or weak security controls.

Careful human review ensures correctness, security, maintainability, and alignment with current standards.

© 2026 by Innovation In Software Corporation

CODE PILOT CHAT

Copilot Chat extends the traditional completion model by enabling natural language conversations directly within the coding environment. Instead of just typing code and receiving suggestions, developers can ask Copilot questions like, “How do I implement OAuth2 authentication?” or “Explain what this function does.” This conversational interface makes problem-solving more intuitive and context-driven.

The importance of Copilot Chat lies in its ability to bridge the gap between coding knowledge and documentation.

© 2026 Innovation In Software Corporation

USE CASES

Boilerplate and Scaffolding

Copilot accelerates development by generating repetitive setup code such as class definitions, configuration files, or API route handlers. For instance, typing # create a Flask route for /hello can instantly produce a functioning endpoint without you writing the boilerplate manually.

Code Completion and Suggestions

It provides context-aware completions for functions, loops, and conditionals. Start typing for i in range(in Python, and Copilot may suggest the entire loop with a sensible body. This allows developers to move faster and maintain momentum.

Error Handling and Validation

Copilot generates robust error-handling blocks, helping developers avoid common pitfalls. For example, in JavaScript, adding a comment // handle exceptions can prompt Copilot to create a try { ... } catch (error) { ... } structure.

© 2026 by Innovation In Software Corporation

USE CASES - 2

Learning and Language Assistance

Developers working with unfamiliar frameworks or languages can rely on Copilot to propose idiomatic patterns. Typing *# connect to MongoDB* may yield a correct connection snippet in Node.js, Java, or Python.

Test-Driven Development (TDD)

Copilot is excellent when writing tests first. Developers can describe intended behavior in comments or partial test code, and Copilot generates full unit tests or assertions.

© 2026 by Innovation In Software Corporation

USE CASES - 3

Finding Logic Errors and Reviewing Code

Copilot is not just a code generator; it can help detect subtle logic mistakes by suggesting corrections or alternative implementations. For example, if a loop doesn't terminate properly or a condition is inverted, Copilot often proposes the correct fix. It can also suggest more efficient or readable versions of code, ng like a lightweight peer review.

Source Control Assistance

Copilot integrates well with GitHub workflows, helping draft commit messages, suggesting summaries for pull requests, or scaffolding git commands in scripts.

© 2026 by Innovation In Software Corporation

POP QUIZ: USE CASES

What other use cases, if any, should be mentioned?

10 MINUTES

© 2026 by Innovation In Software Corporation

GITHUB COPILOT IS EXTENSIBLE

© 2026 by Innovation In Software Corporation

LLM MODELS

GitHub has expanded Copilot beyond just OpenAI's GPT family. It now also supports models from Google (Gemini) and Anthropic (Claude).

LLM Family (Examples in Copilot)	Provider	Typical Use in GitHub Copilot
GPT-5 mini; GPT-5.4 / 5.4 mini; GPT-5.5; GPT-5.6 Luna / Sol / Terra; GPT-6 Astra / Luna / Sol; GPT-6.1 Sol; GPT-5.3-Codex	OpenAI	General-purpose coding and debugging; mini models for fast, low-cost help; GPT-6 and 6.1 models for deeper reasoning and larger code changes; **GPT-5.3-Codex** is optimized for code-centric agentic work and serves as Copilot's long-term support (fallback) model
Claude Haiku 4.5 / 5.5; Claude Sonnet 5 / 5.5; Claude Opus 4.8 / 5 / 5.5; Claude Fable 5 / 5.1	Anthropic	Haiku = fast lightweight help; Sonnet = balanced daily coding assistance; Opus = deeper reasoning, architecture analysis, and complex debugging; Fable = the most capable tier for the hardest tasks, disabled by default and enabled by an administrator
Gemini 3.7 Flash; Gemini 3.8 Flash; Gemini 4 Argon	Google	Flash = fast, lightweight tasks with large-context analysis; Gemini 4 Argon = Google's newest-generation model for more demanding work
Grok 4.5; Grok 4.6; Grok 4.7	xAI	Agentic coding, long-running tasks, and knowledge work at a lower price than most frontier models
MAI-Code-1.1-Flash	Microsoft	Lightweight model for faster responses and lower-cost tasks (replaces Raptor mini)

© 2026 by Innovation In Software Corporation

LANGUAGES SUPPORTED

GitHub Copilot supports a wide range of programming languages. This is the short list:

Language	Language
Python	JavaScript / TypeScript
Java	C#
Go	Ruby
PHP	C++
Rust	SQL
Shell scripting (Bash, PowerShell)	—

© 2026 by Innovation In Software Corporation

IDE SUPPORTED

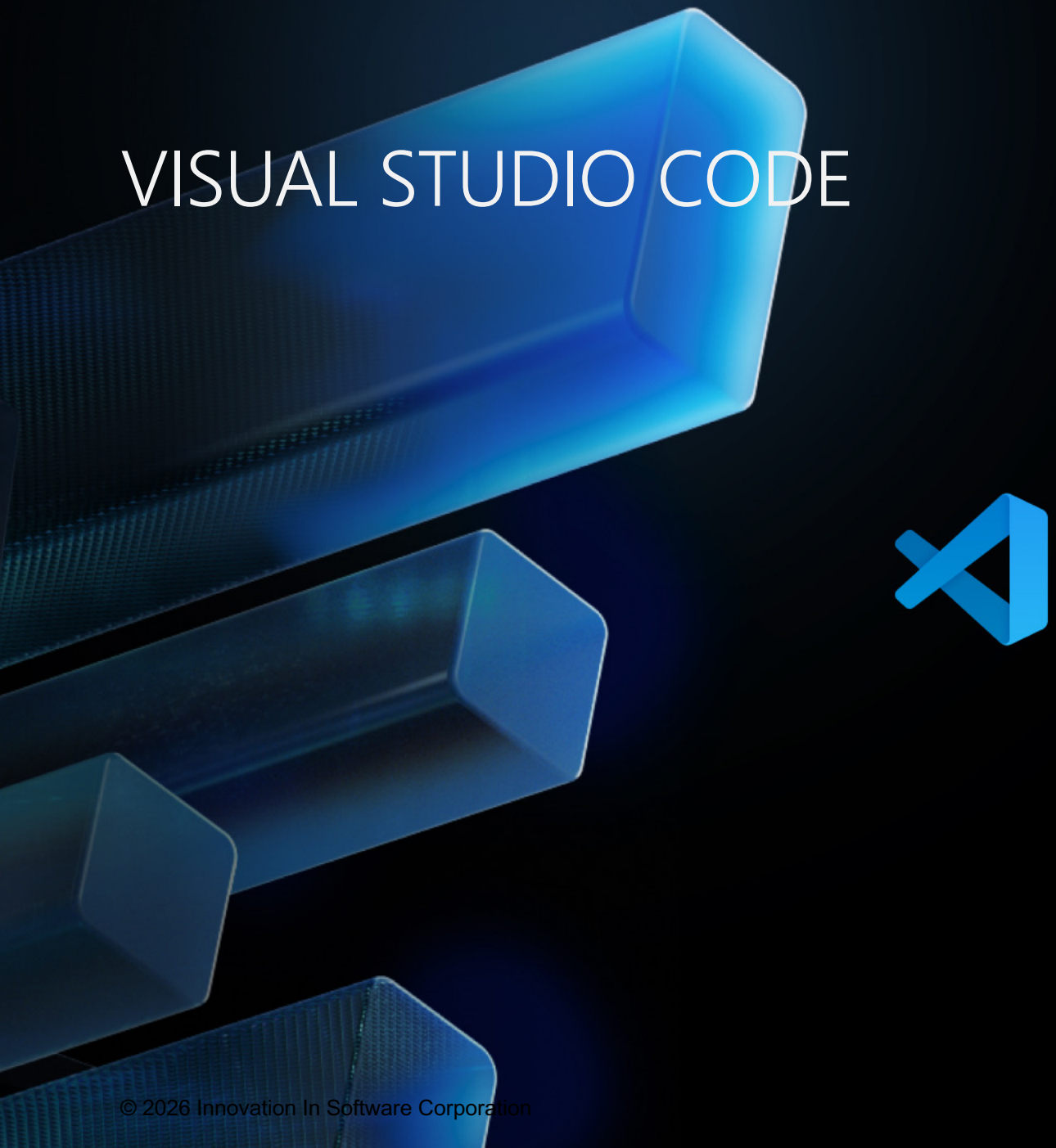
Copilot currently integrates with:

- Visual Studio Code
- Visual Studio (for .NET developers)
- Neovim
- JetBrains IDEs (IntelliJ IDEA, PyCharm, WebStorm, etc.)
- Eclipse

Note: Level of integration and support may vary. Be sure to reference the product documentation.

© 2026 by Innovation In Software Corporation

VISUAL STUDIO CODE



Visual Studio Code (VS Code) is a lightweight yet powerful editor developed by Microsoft. It supports many programming languages and can be extended with thousands of extensions. Core features include IntelliSense code completion, built-in debugging, and Git integration.

A major strength of VS Code is its close integration with GitHub and GitHub Copilot. Developers can commit, push, and review code directly in the editor while Copilot provides AI-powered suggestions inline.

© 2026 Innovation In Software Corporation

GITHUB ACCOUNT

A GitHub account is required to use GitHub Copilot.

- Subscription management: Copilot is a paid service (with free trials for students and certain accounts). GitHub needs your account to handle billing or free access eligibility.
- Authentication: You sign in with your GitHub account in your IDE (VS Code, JetBrains, Visual Studio) to connect to Copilot's cloud models.
- Settings & preferences: Your GitHub account stores Copilot settings like public-code matching, cloud agent repository access, web search, and model-training data use. Editor options, such as per-language controls, are set in your IDE.
- Enterprise access: For business/enterprise users, Copilot is tied to an organization's GitHub account and licensing.

© 2026 by Innovation In Software Corporation

GITHUB LICENSE

GitHub Copilot is a subscription service linked to your GitHub account.

Plan	Intended For	Key Features
Copilot Free	Developers exploring Copilot	2,000 code completions/month and a limited AI credit allowance; models through Auto selection only; limited agent access
Copilot Pro	Individual developers	Unlimited completions, Copilot Chat, model selection, and 1,500 AI credits/month
Copilot Pro+	Power users / advanced developers	7,000 AI credits/month, access to premium models and AI agents
Copilot Max	High-volume individual users	20,000 AI credits/month and priority access to premium models
Copilot Business	Teams & organizations	Pro features plus organization policy controls, license management, and a shared AI credit pool
Copilot Enterprise	Large organizations using GitHub Enterprise Cloud	Business features plus enterprise-grade capabilities, a larger credit pool, and priority access to premium models

© 2026 by Innovation In Software Corporation

CHATGPT

ChatGPT can be used similar to GitHub Copilot — both rely on large language models to generate context-aware code and explanations. *ChatGPT is limited to OpenAI LLMs.*

Copilot is tightly integrated into the coding workflow within IDEs, such as VS Code, and can use LLMs from various vendors.

ChatGPT excels at broader tasks such as debugging discussions, architectural guidance, and natural language exploration of problems.

© 2026 Innovation In Software Corporation

CHATGPT - 2

Aspect	Copilot	ChatGPT
Integration	Built into IDEs (VS Code, JetBrains).	Web/app chat; extensions exist.
Focus	Inline code completion.	Broader Q&A and reasoning.
Context	Uses nearby code (FIM).	Uses conversation history.
Ease	Fast, automatic as you type.	Manual prompts, copy-paste code.
Strengths	Scaffolding, TDD, fixes.	Explaining, design, reviews.
Pros	Seamless coding flow.	Detailed answers, flexible.
Cons	Limited to coding tasks.	Less integrated with IDEs.

© 2026 by Innovation In Software Corporation

COURSE AGENDA

1. Introduction to GitHub Copilot
2. Copilot setup and configuration
3. GitHub Copilot modes
4. Basic code completion
5. Context-aware code suggestions
6. Debugging with Copilot
7. Writing functions and modular code
8. Code documentation
9. Collaboration with Copilot
10. Agentic AI
11. Introduction to testing
12. Advanced Concepts (customization)

© 2026 by Innovation In Software Corporation

LABS

Learning is better when hands-on. Some of the modules have a companion lab reinforcing a kinesthetic learning experience.

- Labs review and reinforce important concepts
- Don’t be surprised - many of the labs extend the topics introduced in the module
- The labs offer an opportunity for real-world experience
- Pair programming can be effective when working on the labs

© 2026 by Innovation In Software Corporation

YOUR CLASS!

Yes, this is your class. What does this mean? You determine the value.

- What is the most important ingredient of class – your participation!
- Your feedback and questions are always welcomed.
- There is no protocol in class. Speak up anytime!
- We welcome your comments during and after class. Just email dmarshall@innovationinsoftware.com.

© 2026 by Innovation In Software Corporation

CONFIRMATION

A study by IT Revolution, surveying over 5,000 developers across multiple Fortune 100 companies, reported measurable productivity improvements with AI coding assistants.

Key Results:

- +26% Task Completion Rate

  More tasks completed, faster delivery.

- +13.5% Weekly Code Commits

  More output, quicker prototyping and testing.

- +38.4% More Code Compilations

  Faster iterations and feedback cycles.

© 2026 by Innovation In Software Corporation

LAB 1 – SETUP

ASSIGNING VIRTUAL MACHINES (VM)

Your trainer will assign virtual machines with credentials to log in. Please note the credentials. You will be assigned the same virtual machine throughout the class.

Your virtual machine has public access to the internet.

Important: Your virtual machine will be destroyed immediately after class.

© 2026 by Innovation In Software Corporation

ENVIRONMENT SETUP

In VS Code, verify that the GitHub Copilot and GitHub Copilot Chat extensions are installed (Ctrl + Shift + X).

Match your current status to the instructions below then follow the action.

Status	Action Required
Scenario 1: GitHub account + GitHub Copilot license.	Open VS Code. Click the Accounts icon and sign-in with your account.
Scenario 2: GitHub account but no GitHub Copilot license.	Open VS Code. Click the Copilot menu, on status bar at the right side, and signup for a free Copilot account.
Scenario 3: No account, no license.	1. Create a personal account at GitHub.com. 2. Open VS Code. Click the Copilot menu, on status bar at the right side, and signup for a free Copilot account.

© 2026 by Innovation In Software Corporation

GITHUB COPILOT

Goal: Confirm GitHub Copilot is operational in VS Code.

In this portion of the lab, you will:

- Open or create a working folder
- Create a simple Python file
- Trigger a Copilot suggestion
- Accept the suggestion

© 2026 by Innovation In Software Corporation

OPEN OR CREATE THE LAB FOLDER

In VS Code

- Select File → Open Folder
- Navigate to your sample files
- Find Lab1 folder, select it, and click Open
- Alternatively, create a new folder named Lab1, then open it

© 2026 by Innovation In Software Corporation

CREATE THE PYTHON FILE

In the Explorer panel, right-click inside the Lab1 folder

- Select New File
- Name the file:

  first.py

- Press Enter

© 2026 by Innovation In Software Corporation

TRIGGER A COPILOT SUGGESTION

Inside first.py, type:

- #create an add function

  Press Enter

- A Copilot suggestion should appear in faded text.
- Press Tab to accept the suggestion.

© 2026 by Innovation In Software Corporation

VERIFICATION

Copilot is operational if:

- A function was generated automatically
- The suggestion appeared without manual prompting
- You accepted it using Tab

© 2026 by Innovation In Software Corporation

Lab completed

FUNDAMENTALS

DETAILS AND WORKFLOW

INTRODUCTION

This module outlines GitHub Copilot as a cloud-based AI coding assistant. It explains how prompts are automatically enriched with contextual information from files, functions, and the surrounding codebase, enabling intelligent code generation, including fill-in-the-middle when supported.

The workflow covers prompt initiation, context collection, secure cloud-based inference, and the delivery of results as ghost text suggestions or chat-based responses. The module also addresses key considerations related to privacy, licensing, and security.

© 2026 by Innovation In Software Corporation

CLOUD-BASED GENERATIVE AI TOOL

GitHub Copilot is not just a local IDE extension—it is a cloud-based service that relies on generative AI models hosted and maintained by GitHub and Microsoft. Every time a developer enters a prompt or writes code, that context is securely transmitted to cloud servers where the model processes it and returns suggestions.

Being cloud-based ensures scalability and continuous improvement. The models can be updated, retrained, and fine-tuned without requiring developers to manually install updates.

© 2026 by Innovation In Software Corporation

PROMPT

Prompts include more than what you type.

When you type a comment or a partial line of code, that input is only part of the prompt. GitHub Copilot automatically enriches it with additional context from the surrounding code, relevant files in the repository, and the project's current structure. As a result, the effective prompt is much larger than the text you typed, leading to more accurate and context-aware suggestions.

This is why Copilot can sometimes anticipate what you want several lines ahead—it is interpreting not just your last keystrokes but the broader context.

© 2026 by Innovation In Software Corporation

CONTEXT

Context is critical to how GitHub Copilot works. The tool doesn't just generate random code—it looks at the file you're writing, the function names, other files you have open, etc. This context tells GitHub Copilot what you're trying to accomplish, so its suggestions are relevant. Without context, its output would be generic and often wrong.

For example, if you write a comment saying *# create calculate function to calculate sales tax*, Copilot uses that clue along with your code structure to suggest a tax calculation function instead of something unrelated.

© 2026 Innovation In Software Corporation

CONTEXT - 2

Copilot gathers context from multiple sources, some more influential than others.

- Function and variable names
- Source code
- Comments
- Open files
- Project-level context
- File name and file type
- Coding conventions

© 2026 Innovation In Software Corporation

3-IN-1 MODEL

Copilot originally used fill-in-the-middle (FIM), predicting code between what comes before and after your cursor, so it could only insert text at that spot. Its newer “3-in-1” model combines code completions, next edit suggestions, and long-distance edits into one model that returns changes as diffs.

As a result, Copilot can finish your line, rewrite code you have already typed, and suggest related changes elsewhere in the file.

© 2026 Innovation In Software Corporation

3-IN-1 MODEL - 2

- Code completions finish what you are typing. As you type, Copilot shows a gray guess of the rest of the line, or even a whole block of code. Press Tab to accept it,or just keep typing to ignore it.
- Next edit suggestions guess your next change. If you rename something, Copilot offers to update the nearby lines that use the old name. Unlike completions, these can change code you already wrote, not just add new code.

© 2026 Innovation In Software Corporation

3-IN-1 MODEL - 3

Long-distance edits do the same thing farther away. After you make a change, Copilot can jump you to another spot in the file that needs the same fix, so you do not miss anything.

© 2026 Innovation In Software Corporation

COPILOT WORKFLOW – STEP 1 - CREATE A PROMPT

A prompt can be:

- Code you begin typing
- A function name or variable name
- Even a natural language question inside Copilot Chat

Copilot uses this as the main clue for what to generate next.

© 2026 by Innovation In Software Corporation

STEP 2 – COPILOT COLLECTS CONTEXT

Before sending anything to the cloud, Copilot gathers context from your environment:

- The current file
- Other open project files in your editor
- Comments
- The project's structure and imported libraries
- File names and file type (Python, JavaScript, HTML, etc.)

This helps Copilot understand not just what you typed, but but the context.

© 2026 by Innovation In Software Corporation

STEP 3 AND STEP 4

Step 3: Prompt + context sent securely

The prompt and gathered context are encrypted and sent to a GitHub Copilot server. This ensures that your data is transmitted securely, never in plain text.

Step 4: AI model generates results

On GitHub's side, one of the supported LLMs (such as GPT-6, GPT-4.1, Gemini, or Claude, depending on your settings) processes the input.

- The model generates one or more code suggestions.
- Optional post-processing may occur, such as basic vulnerability checks or formatting adjustments, before sending results back.

© 2026 by Innovation In Software Corporation

STEP 5 – RESULTS RETURNED TO IDE

The generated suggestion flows back into your IDE (e.g., Visual Studio Code, JetBrains). You'll see it as:

- An inline completion (gray "ghost text")
- A list of alternative suggestions
- A full explanation or snippet in Copilot Chat

You can accept, modify, or reject the suggestions as desired.

© 2026 by Innovation In Software Corporation

LICENSED CONTENT

Another concern is that Copilot's training data may include code under restrictive licenses. While GitHub asserts that Copilot generates new content rather than copying directly, there is debate over whether certain outputs might inadvertently resemble copyrighted or unlicensed code.

Developers should be aware of these risks and watch for potential license issues. GitHub Copilot provides an option to block suggestions that match verbatim public code.

© 2026 Innovation In Software Corporation

PRIVACY

Privacy is a major consideration when using AI-assisted coding tools. Copilot may collect snippets of your code to provide context, raising concerns about sensitive or proprietary data. GitHub provides an option in settings to disable code retention for training purposes, giving organizations more control over their intellectual property.

You should configure these options carefully to balance functionality with privacy requirements.

© 2026 Innovation In Software Corporation

PRIVACY - 2

User engagement—such as whether you accept, modify, or reject suggestions—feeds back into the learning process for Copilot. This helps the system refine its outputs over time. It is important to note that feedback is aggregated and anonymized.

This engagement loop ensures that Copilot becomes more useful the more it is used.

© 2026 Innovation In Software Corporation

SECURITY

The prompt and code should never include sensitive data such as passwords, API keys, or confidential business logic. Even though Copilot transmits information securely, providing such details still increases the chance of exposure or accidental misuse. Developers should treat all prompts and context as if they may be seen outside the immediate coding session.

AI-generated code can also introduce security vulnerabilities, including insecure SQL queries without parameterization, weak authentication flows, or outdated cryptographic functions. To help, GitHub has added post-processing checks like vulnerability detection and filtering for unsafe code patterns.

These safeguards are limited. The primary responsibility for security remains with the development team, who must carefully review every suggestion, apply secure coding practices, and ensure that GitHub Copilot is used as a supportive tool rather than a replacement for sound engineering judgment.

© 2026 by Innovation In Software Corporation

SCOPE OF SUGGESTIONS

This is about where and what Copilot considers as input:

File scope

Copilot looks only at the current file and nearby lines. Suggestions are local and good for finishing code, but unaware of wider project patterns.

Project scope

Copilot uses context from across the workspace, including other files and shared patterns, producing code that better fits the overall architecture.

© 2026 by Innovation In Software Corporation

SCOPE OF SUGGESTIONS - 2

Prompt scope:

Comments or function names you type strongly shape suggestions.

Example: typing # function to calculate factorial will cause a factorial implementation suggestion.

Language scope:

GitHub Copilot supports many languages, but scope can be limited by file type. You can also enable or disable ghost text by language in the GitHub Copilot settings.

© 2026 by Innovation In Software Corporation

SETTINGS / CONFIGURATION

© 2026 by Innovation In Software Corporation

PRIVACY

Privacy panel in GitHub Copilot settings controls how your code and interactions with Copilot are handled.

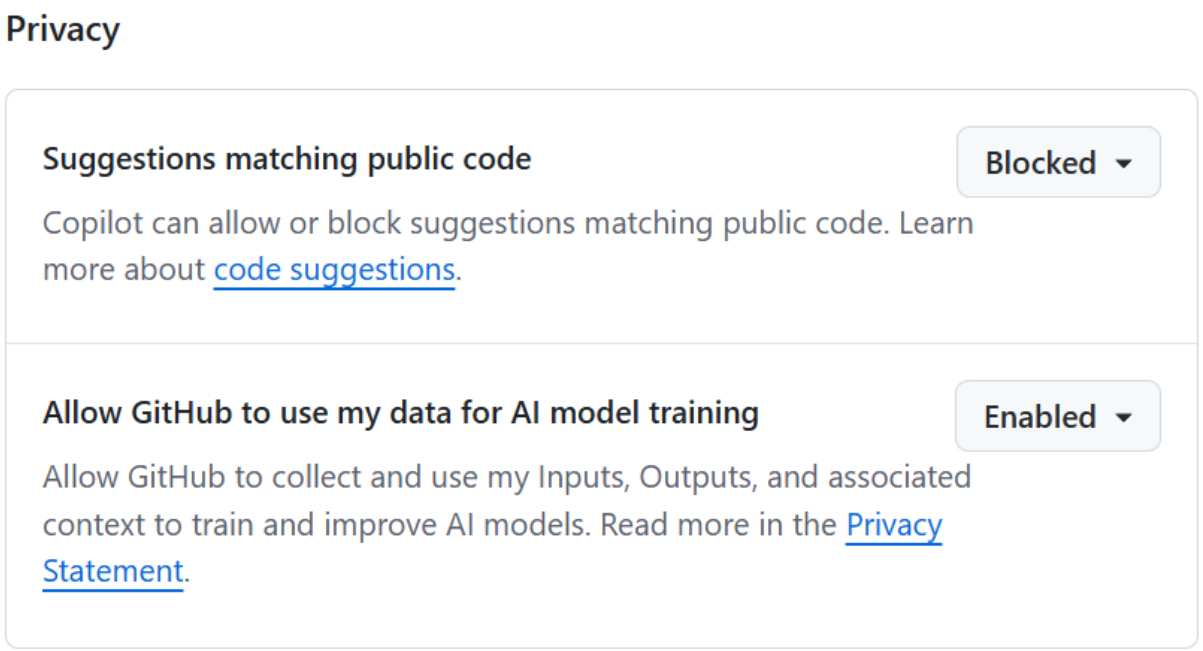
Suggestions matching public code: You can choose whether Copilot is allowed to show you suggestions that are similar to existing public code on GitHub. Blocking this reduces the chance of receiving verbatim code from public repositories.

Allow GitHub to use my data for product improvements: If enabled, GitHub may use your prompts, suggestions, and snippets to analyze how Copilot is used, fix issues, and improve features.

Allow GitHub to use my data for AI model training: If enabled, your prompts and snippets could also be used to further train GitHub's AI models. With it disabled, your data won't be included in training.

© 2026 by Innovation In Software Corporation

PRIVACY - 2

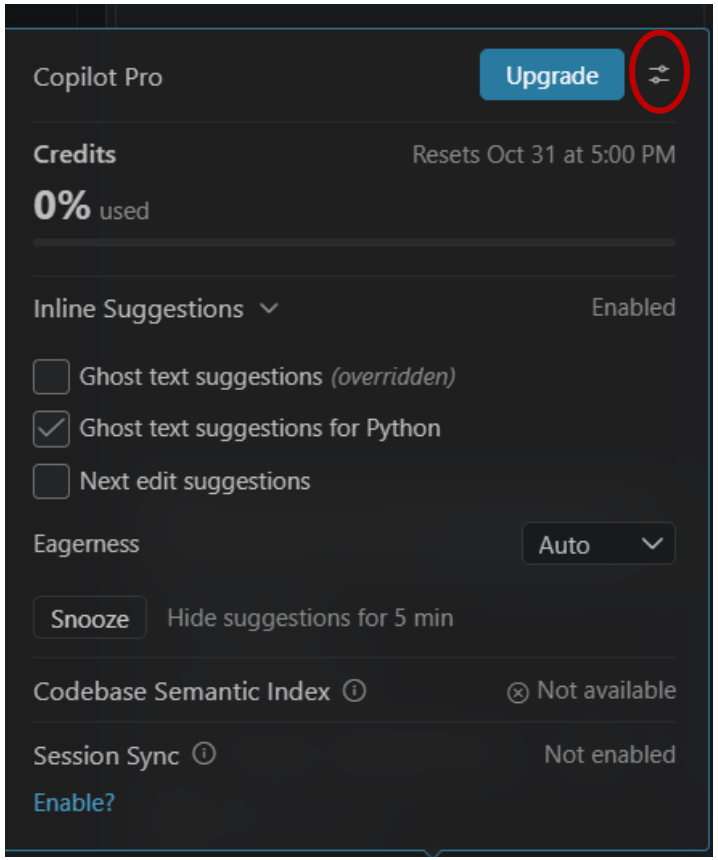


How to get to the Privacy panel.

- Log in to GitHub
- Go to github.com and sign in.
- Open Copilot settings
- Select Features
- Scroll down right panel to find Privacy

© 2026 Innovation In Software Corporation

PRIVACY - 3



Alternatively, set the Privacy options using the GitHub Copilot menu. From the Status Bar, select the GitHub Copilot menu and select the two horizontal bars, top-right corner of the menu.

© 2026 Innovation In Software Corporation

INLINE SUGGESTIONS

The inline suggestions completions toggle in GitHub Copilot controls whether ghost text suggestions appear while you type. You can set it globally, per language, or for specific file types.

When completions are enabled, you code faster because of boilerplate generation, context-aware help, and exposure to idiomatic patterns. It feels like a pair-programmer suggesting alternatives. The downside is potential over-reliance, clutter from too many suggestions, and the risk of accepting inaccurate or irrelevant code.

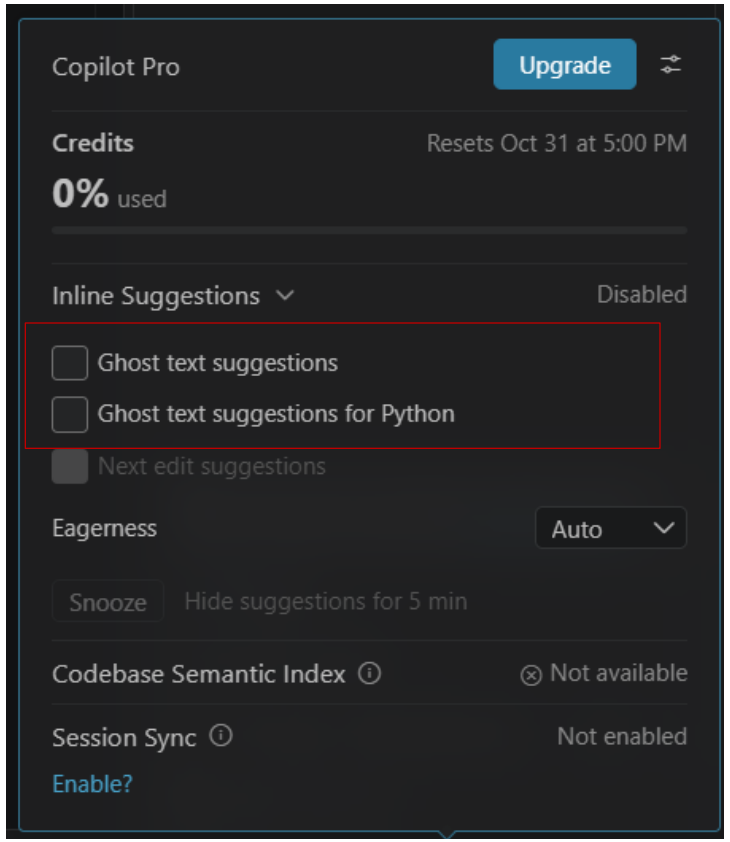
© 2026 by Innovation In Software Corporation

INLINE SUGGESTIONS – 2

When inline suggestions are disabled, you retain full control of your code with fewer distractions and better focus, which can be useful for teaching or training. However, you lose the productivity boost, brainstorming value, and language guidance Copilot provides.

© 2026 by Innovation In Software Corporation

INLINE SUGGESTIONS - 3

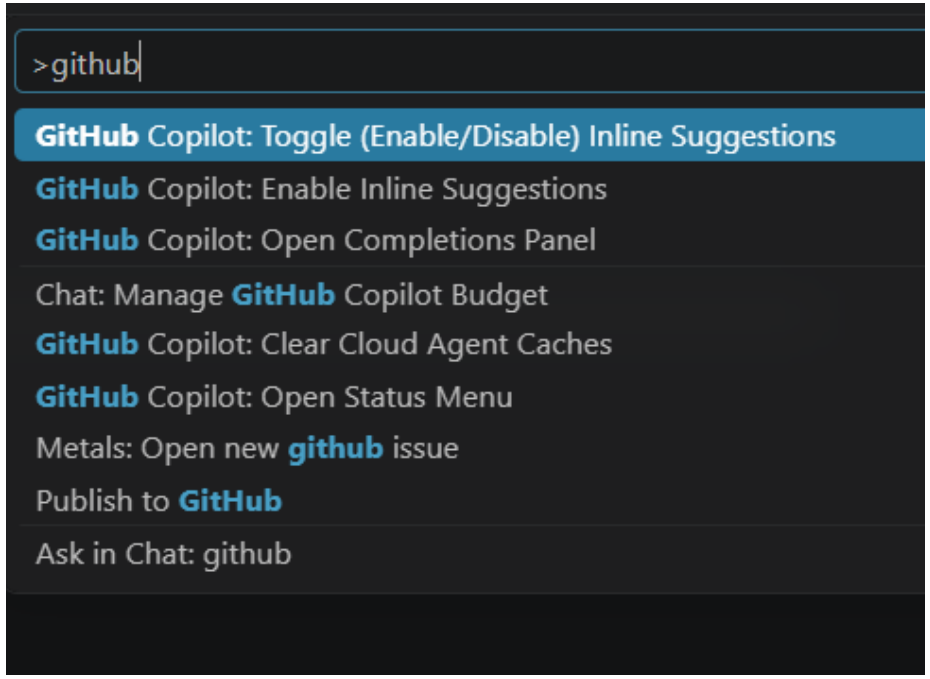


Ghost text suggestions is the global switch. It sets the "*" entry in github.copilot.enable, which applies to every language that has no rule of its own.

Ghost text suggestions for Python is the switch for the language of the file you have open. It sets the "python" entry, and that entry always wins over the global one for Python files.

© 2026 Innovation In Software Corporation

ENABLE / DISABLE COMPLETION – 4



Ctrl-Shift-P and GitHub Copilot: Toggle (Enable / Disable) Inline Suggestions.

© 2026 Innovation In Software Corporation

DISABLE FOR A LANGUAGE

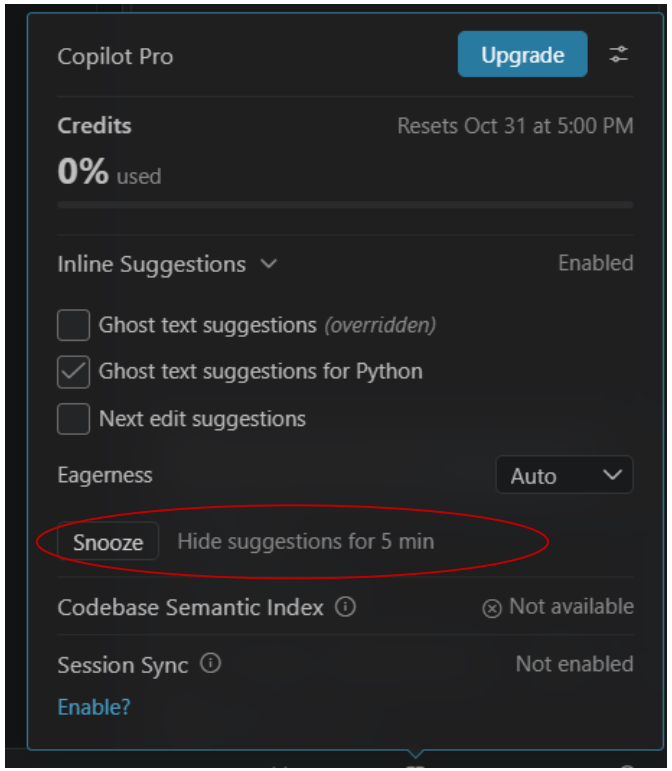
Disabling GitHub Copilot for certain languages is especially helpful when you need more control over security, focus, or code quality.

For sensitive or non-code files (such as .env, yaml, or markdown), it reduces the chance of leaking secrets, introducing misconfigurations, or cluttering documentation with unnecessary suggestions. In languages you already know well, it minimizes distractions and ensures that the code you write remains intentional. For teaching, learning, or secure environments, it helps avoid compliance risks, encourages independent problem-solving, and ensures AI isn't interfering where strict accuracy is required.

In short, turning Copilot off selectively gives you a better balance of productivity, safety, and intentional coding—leveraging AI where it's valuable

© 2026 by Innovation In Software Corporation

SNOOZE INLINE SUGGESTIONS

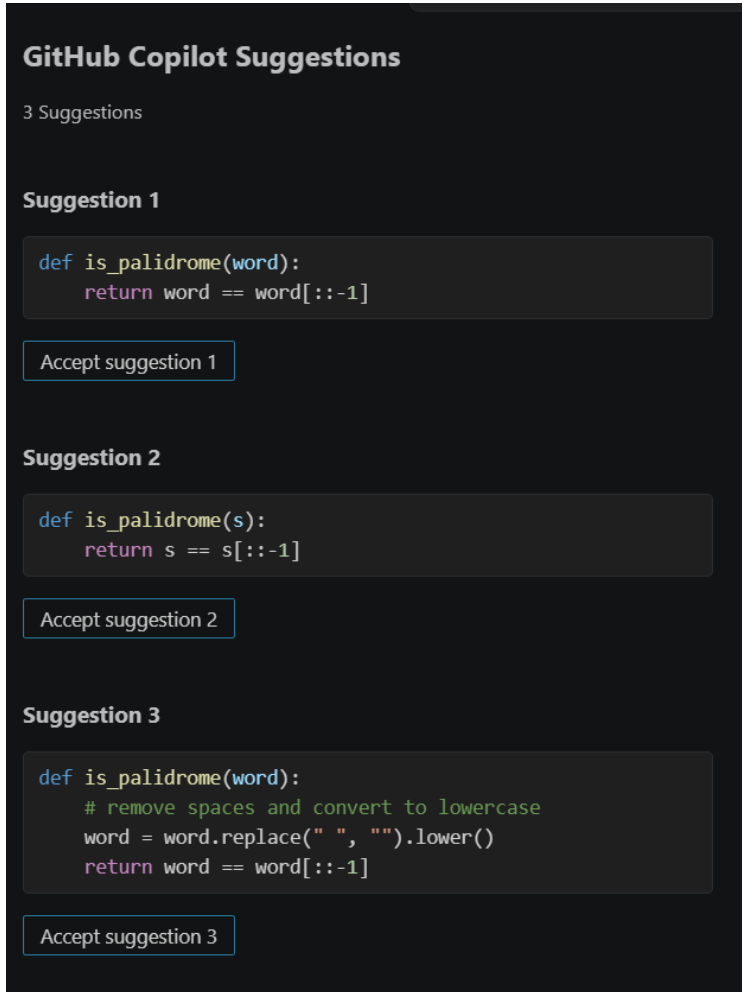


The Snooze feature temporarily prevents Copilot's ghost text suggestions in your editor.

When activated, Copilot pauses automatic suggestions for 5 minutes, then resumes automatically.

© 2026 Innovation In Software Corporation

OPEN COMPLETION PANEL

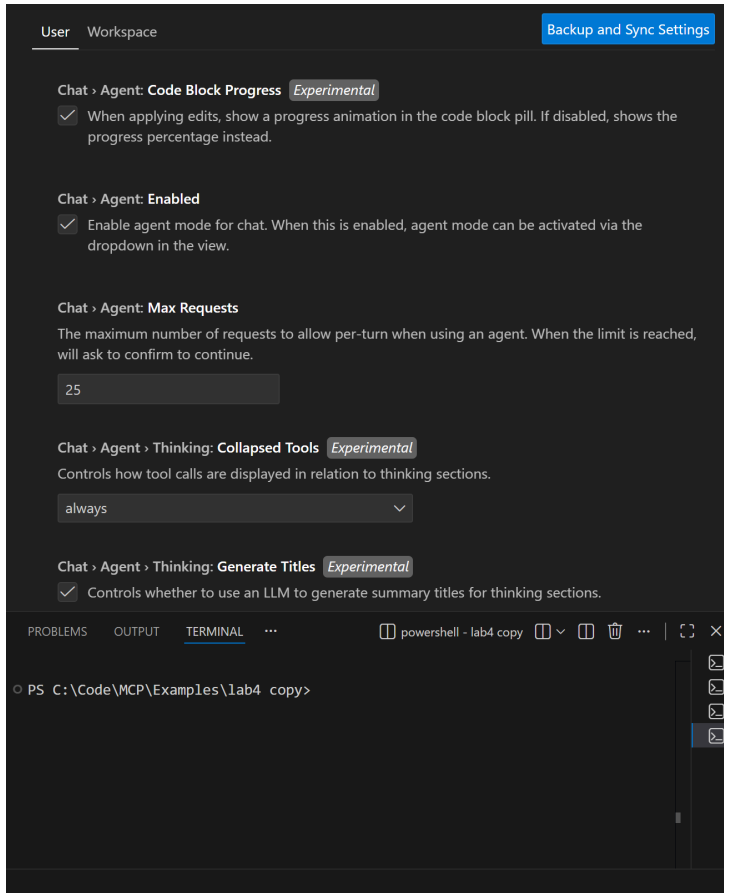


From the Inline Suggestions menu, the Open Completion Panel lets you view multiple Copilot suggestions instead of the single ghost text suggestion shown in the editor. You can compare alternatives and choose the best option for your task.

This is especially useful when ghost text suggestions are turned off, allowing you to manually request and review suggestions on demand.

© 2026 Innovation In Software Corporation

GITHUB COPILOT SETTINGS (VSCODE)



GitHub Copilot Chat settings are not group in VS code, which can make them difficult to find. To locate them easily, from the More menu (…) on the Chat window, select Chat Settings.

© 2026 Innovation In Software Corporation

RETROSPECTIVE

This module showed how GitHub Copilot works as a cloud-based AI assistant, using context and fill-in-the-middle to generate relevant suggestions. We walked through its workflow—from prompts and context gathering to secure cloud processing and IDE results—highlighting that Copilot is more than code completion; it adapts to the developer's environment.

We also stressed responsibility and safeguards. Copilot boosts productivity, but privacy, licensing, and security require careful configuration and adhering to best practices.

© 2026 by Innovation In Software Corporation

LAB 2 – PALINDROME

PALINDROME

A palindrome is a word, phrase, number, or sequence that reads the same forward and backward, such as “racecar”, “level”, or “12321”.

© 2026 Innovation In Software Corporation

SETUP

- Open VS Code
- Find and open the Lab2 folder. Within that folder, create a new folder: is_palindrome.
- Alternately, create a new folder called is_palindrome somewhere convenient.
- Within the is_palindrome folder, create a new file: is_palindrome.py.

© 2026 by Innovation In Software Corporation

CREATE THE PROGRAM

1. In is_palindrome.py, enter a comment to create a is_palindrome function with a string parameter.
2. Press enter and wait.
3. If necessary, tab again and wait to complete the function implementation.
4. If more than one suggestion provided, scroll (< >) the suggestions and tab to select one.

© 2026 by Innovation In Software Corporation

RUN THE PROGRAM

1. Add a comment to insert a main function to test the is_palindrome function.
2. *Save the code resulting code.*
3. Run the program from the terminal window.

© 2026 by Innovation In Software Corporation

IS_PRIME

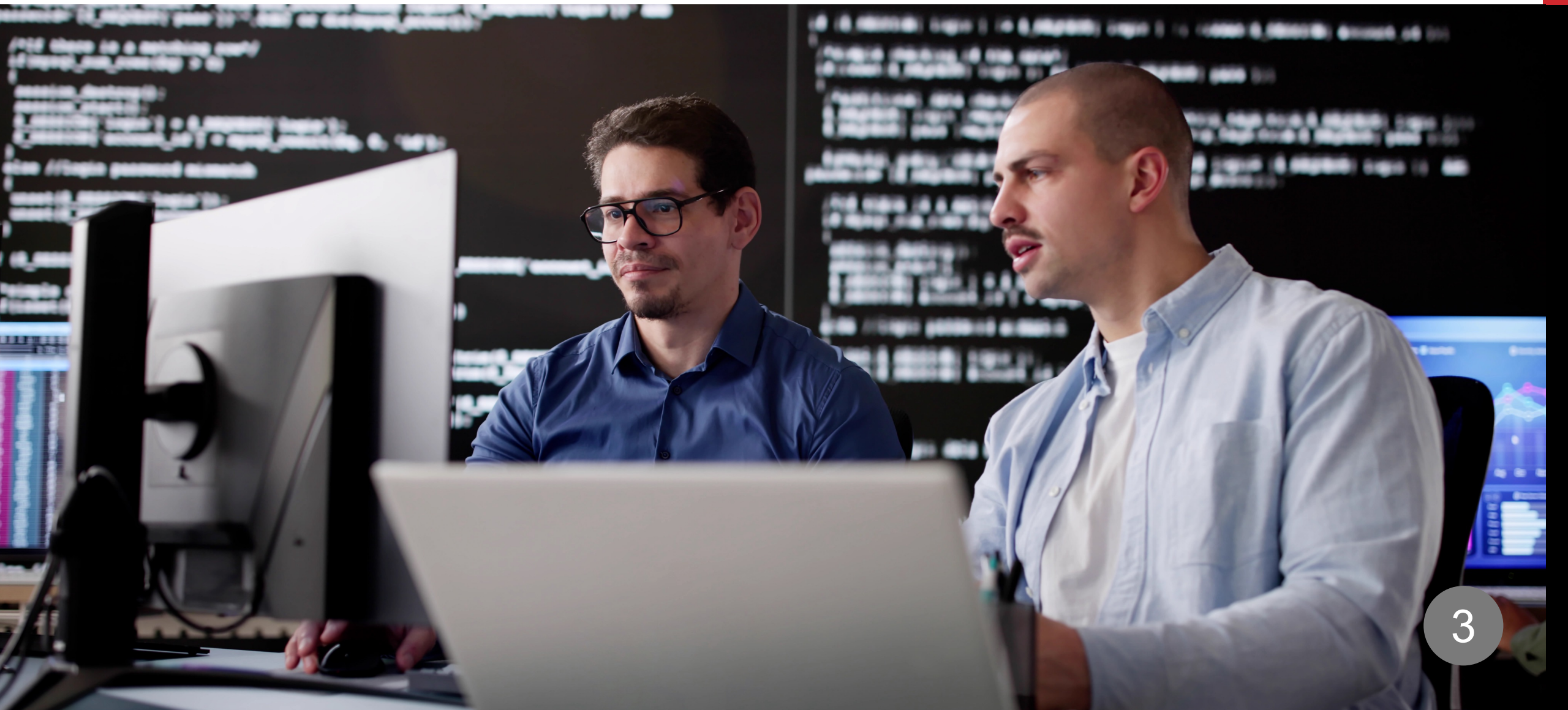
1. Add the function is_prime to your code, which returns whether a value is prime.
2. Generate the code to test the function.

© 2026 by Innovation In Software Corporation

Lab completed

INLINE SUGGESTIONS

BASICS AND BEST PRACTICES



INTRODUCTION

This module is a practical guide to using GitHub Copilot ghost text suggestions effectively. It explains ghost text and “getting started” habits (clear intent, pause for suggestions, accept/cycle), then shows how to sharpen results with context: meaningful names, comments/docstrings, and even adding imports to steer libraries.

It walks through scaffolding patterns, partial code guidance, and best-practices.

© 2026 by Innovation In Software Corporation

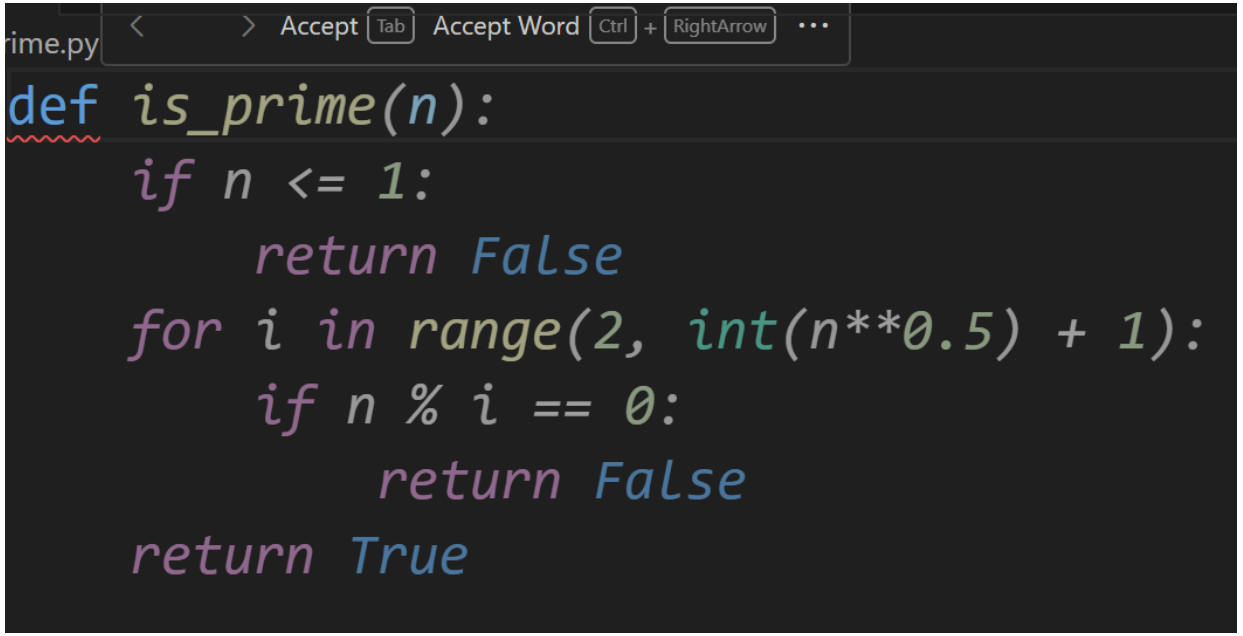
GETTING STARTED

Best practices for code completion:

- Start with a clear description: type a descriptive comment or a function signature (e.g., # print "hello, world" or def add(a, b):).
- Pause—Copilot will show ghost text. Press Tab to accept, or use the Inline Completion toolbar to cycle alternatives (if offered).
- Prefer small, incremental prompts: review the next line or two, accept, run, repeat. Short loops of type → accept → test keep suggestions on track.
- Use meaningful names (files, functions, variables). Better names → better suggestions.

© 2026 by Innovation In Software Corporation

GHOST TEXT



Copilot predicts what you might type next, showing it as italic gray text. It appears as faint, gray, italicized text that appears inline in your editor as a ghost text suggestion.

- The text isn't "real code" yet — it won't run, save, or commit until you accept it.
- Accepting with a keystroke (tab) can save you multiple lines of typing.
- Safe to ignore: If you keep typing, the italic ghost text disappears automatically.
- Options: You can cycle through alternatives (next/previous suggestion) if more than one is available.

© 2026 by Innovation In Software Corporation

CONTEXT

Provide context:

- Function and variable names (calculate_tax, db_client).
- Comments/docstrings that state what and how.
- Imports and existing patterns in your file.

If suggestions drift, narrow the context:

- Add a clarifying comment.
- Rename ambiguous identifiers.
- Close irrelevant files or move code into a focused file.

© 2026 by Innovation In Software Corporation

CONTEXT - COMMENTS

A second comment can help GitHub Copilot by narrowing the context and reducing ambiguity. While one comment may describe the general task, adding another can specify how it should be done, what tools or techniques to use, or what constraints to follow.

```
# Calculate factorial of a number
# Use recursion
def factorial(n):
```


© 2026 by Innovation In Software Corporation

PARTIAL CODE GUIDANCE

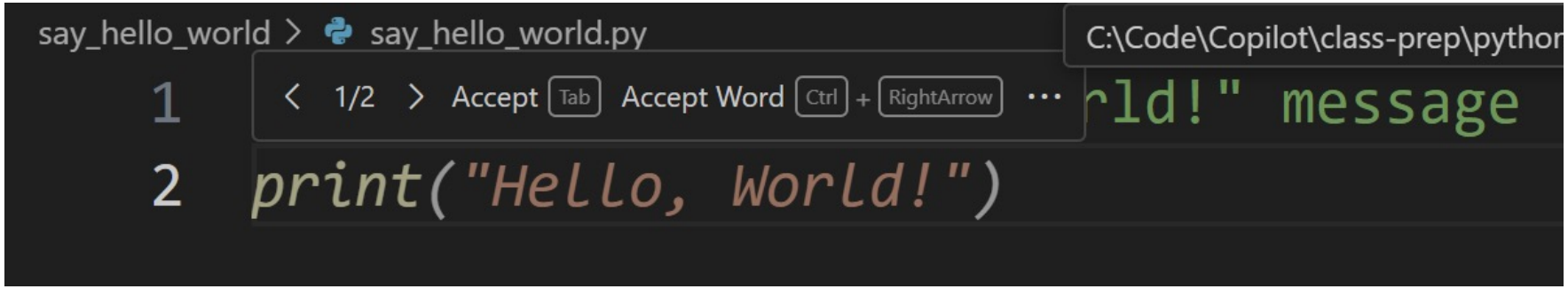
Partial code guidance means you don't have to fully describe or write the whole solution—just give GitHub Copilot a hint through a short comment, function signature, or partial line of code. Copilot will then try to complete it based on the context.

```
# Example 1
# Return the largest number in a list
def find_max(numbers):


# Example 2
# Print numbers 1 to 5
for i in range(
```


© 2026 by Innovation In Software Corporation

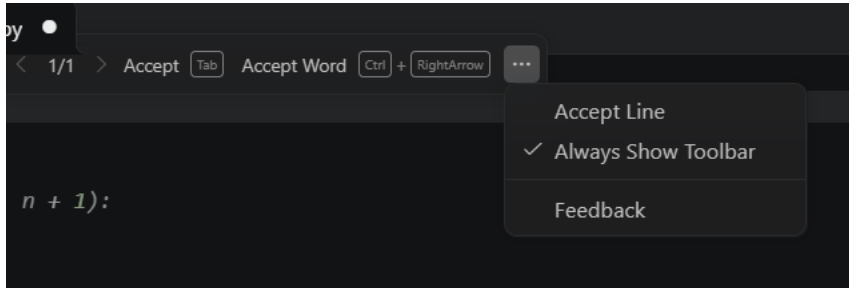
COPILOT INLINE SUGGESTION TOOLBAR



- Arrows (◀️ ▶️): to cycle through different completion suggestions.
- Accept (Tab): indicates you can press Tab (or Enter) to accept the current suggestion.
- Word: switches the suggestion to per word completion (within the current suggestion).

© 2026 by Innovation In Software Corporation

COPILOT GHOST TEXT SUGGESTION TOOLBAR - 2



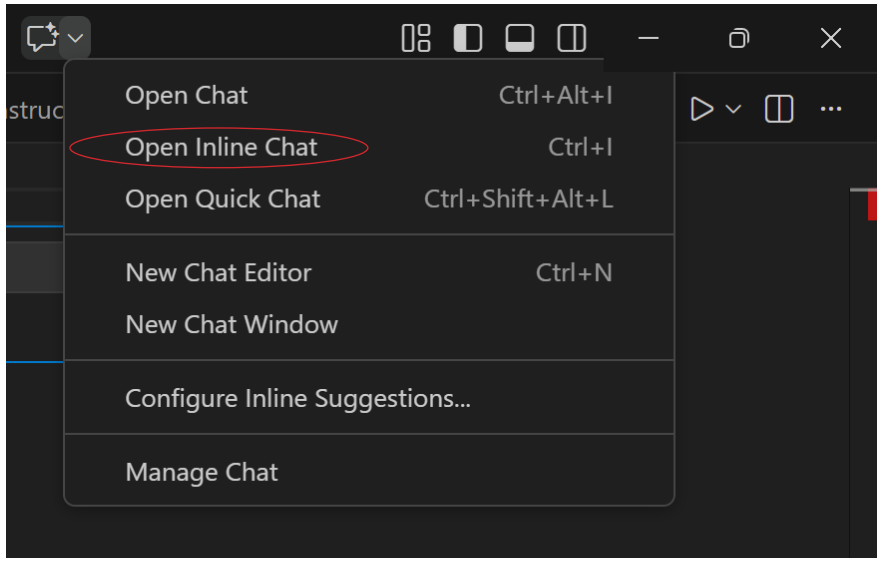
Accept Line. Let's you accept just the current line of Copilot's suggestion instead of an entire multi-line completion. This is handy when Copilot offers a large block of code.

Always Show Toolbar. Toggles whether the Inline Suggestion toolbar is always visible or only appears on **hover.**

Feedback. Opens Issue Reporter form to report an issue on GitHub Copilot for VS Code extension.

© 2026 by Innovation In Software Corporation

INLINE CHAT WINDOW

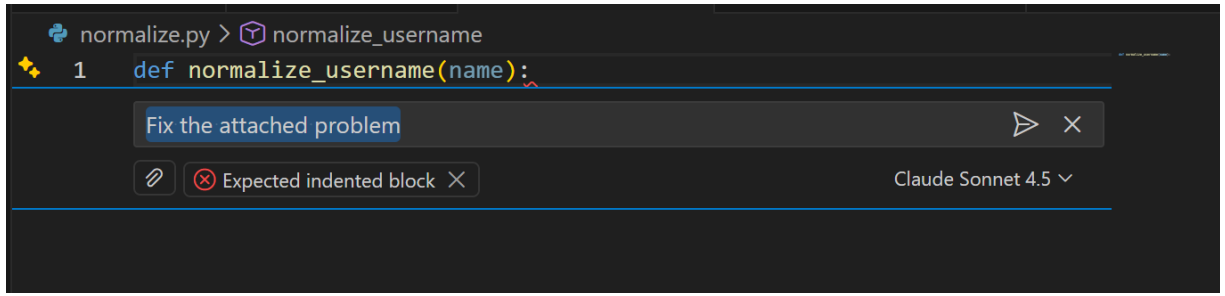


Adjacent to the Chat View button, open the Chat View menu. You can select *Open Inline Chat* to view the Inline Chat window.

Alternatively, open the window with Ctrl+I.

© 2026 by Innovation In Software Corporation

INLINE CHAT WINDOW - 2



Inline Chat window presents a prompt where you can interact with GitHub Copilot in the editor window.

Think of it as “Copilot, look at this exact spot in my code and do something here.”

Set the context before opening the Inline Chat window

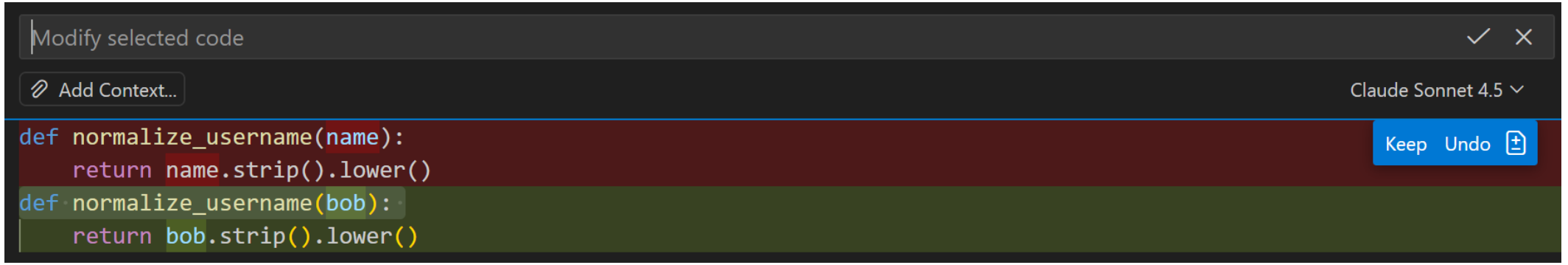
- Set the cursor location, or
- Select code.

© 2026 by Innovation In Software Corporation

INLINE CHAT WINDOW – 3

When you make code changes using the Inline Chat window, VS Code shows a before-and-after view in contrasting colors.

Click the Keep button to apply the changes to your code.



© 2026 by Innovation In Software Corporation

BEST PRACTICES

- Request alternatives: Add/adjust comments or partially type the target line to reshape the suggestion.
- Keep prompts safe: Don’t include sensitive data (passwords, keys, proprietary algorithms).
- Validate: Run, lint, and test. Copilot accelerates typing; you own the quality.
- Feedback loop: small steps improve the next suggestion iteratively.

© 2026 by Innovation In Software Corporation

RETROSPECTIVE

This module showed practical ways to use GitHub Copilot ghost text suggestions effectively. We saw how ghost text works and how intent signals—comments, docstrings, imports, and function signatures—improve results. Techniques like scaffolding, partial code hints, and test-driven scaffolding emphasized that small, clear prompts generate better completions.

Reviewing output for correctness, security, and style, while avoiding sensitive data in prompts.

© 2026 by Innovation In Software Corporation

LAB 3 – BANKING APPLICATION

OBJECTIVE

Use GitHub Copilot to build a simple banking application while reinforcing:

- Inline code completion
- Context-aware generation
- Code modification within the editor

You will create functions incrementally and enhance them using the features demonstrated in the previous module.

© 2026 by Innovation In Software Corporation

SETUP

- Open VS Code
- Find and open the Lab3 folder. Within that folder, create a new folder: banking.
- Alternately, create a new folder called banking somewhere convenient.
- Within the banking folder, create a new file: banking.py.

© 2026 by Innovation In Software Corporation

GENERATE FUNCTIONS

Open the file named:

banking.py

Begin with a short comment describing the purpose of the program.

> This simple banking application models basic transactions such as deposits, withdrawals, loans, and loan payments. Each function updates and returns the account balance to demonstrate clear and predictable financial operations.

© 2026 by Innovation In Software Corporation

CREATE BANKING FUNCTIONS

Create *one function at a time* using ghost text suggestions.

Each function must:

- Accept a balance parameter
- Accept an amount parameter
- Return the updated balance
- Remain a pure function (no global variables)

Create the following functions in this order:

- deposit
- loan
- withdrawal
- loan_payment

© 2026 by Innovation In Software Corporation

CREATE BANKING FUNCTIONS - 2

For each function:

- Type a descriptive comment (example: # create a deposit function)
- Press Enter
- Review the inline suggestion
- Press Tab to accept if appropriate
- Adjust as needed
- Do not manually write the full function. Use GitHub Copilot to generate each one.

© 2026 by Innovation In Software Corporation

UPDATE DEPOSIT AND WITHDRAWAL

In a single step, enhance the deposit and withdrawal functions so that when each function completes, it displays:

- A clear message indicating the transaction type
- The updated balance

Use the techniques demonstrated in the previous module. Avoid rewriting the functions manually.

© 2026 by Innovation In Software Corporation

TEST THE BANKING APPLICATION

Using ghost text suggestions, ask GitHub Copilot to generate a main function that test the four banking functions.

Test the application in the terminal window.

© 2026 by Innovation In Software Corporation

Lab completed

COPILOT CHAT

INTEGRATION

INTRODUCTION

The next module introduces GitHub Copilot Chat in VS Code as a conversational coding assistant. It expands beyond inline suggestions by supporting natural language queries, code edits, and even multi-step workflows through Interactive, Autopilot, and Plan modes

It also highlights practical tools such as checkpoints to track and roll back edits, action controls to apply or discard suggestions, and inline chat for quick changes. Features like slash commands, #context, and variable awareness show how Copilot integrates smoothly into the coding workflow.

© 2026 by Innovation In Software Corporation

COPILOT CHAT

Open the Chat window:

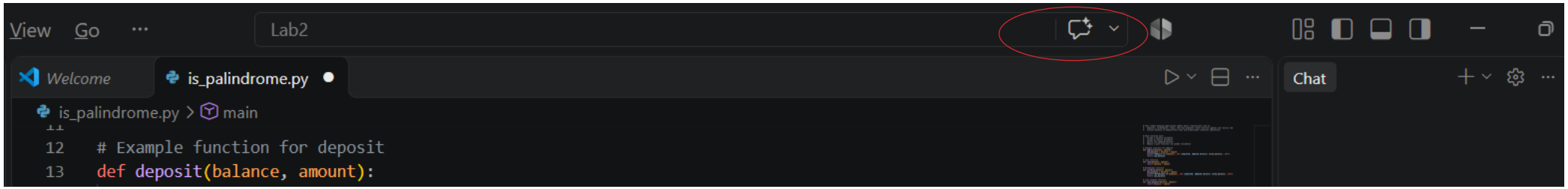
- If installed, you'll see a Chat View icon on top right of the VS Code window.

  Click it to open the Chat View.

- Alternatively, you can open chat directly with:

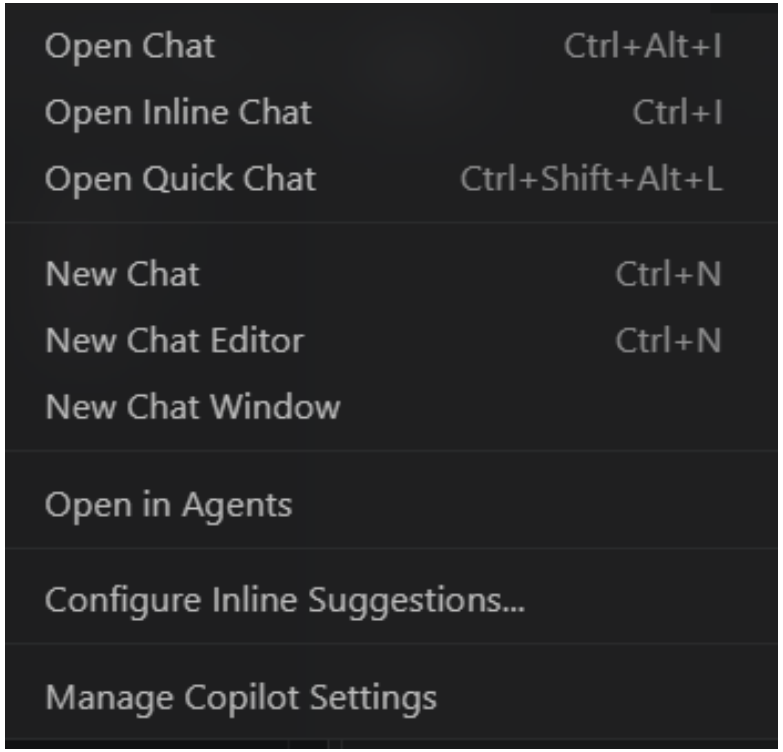
  Ctrl+Shift+I (Windows/Linux)

  ^⌘I (macOS).



© 2026 by Innovation In Software Corporation

CHAT VIEW MENU

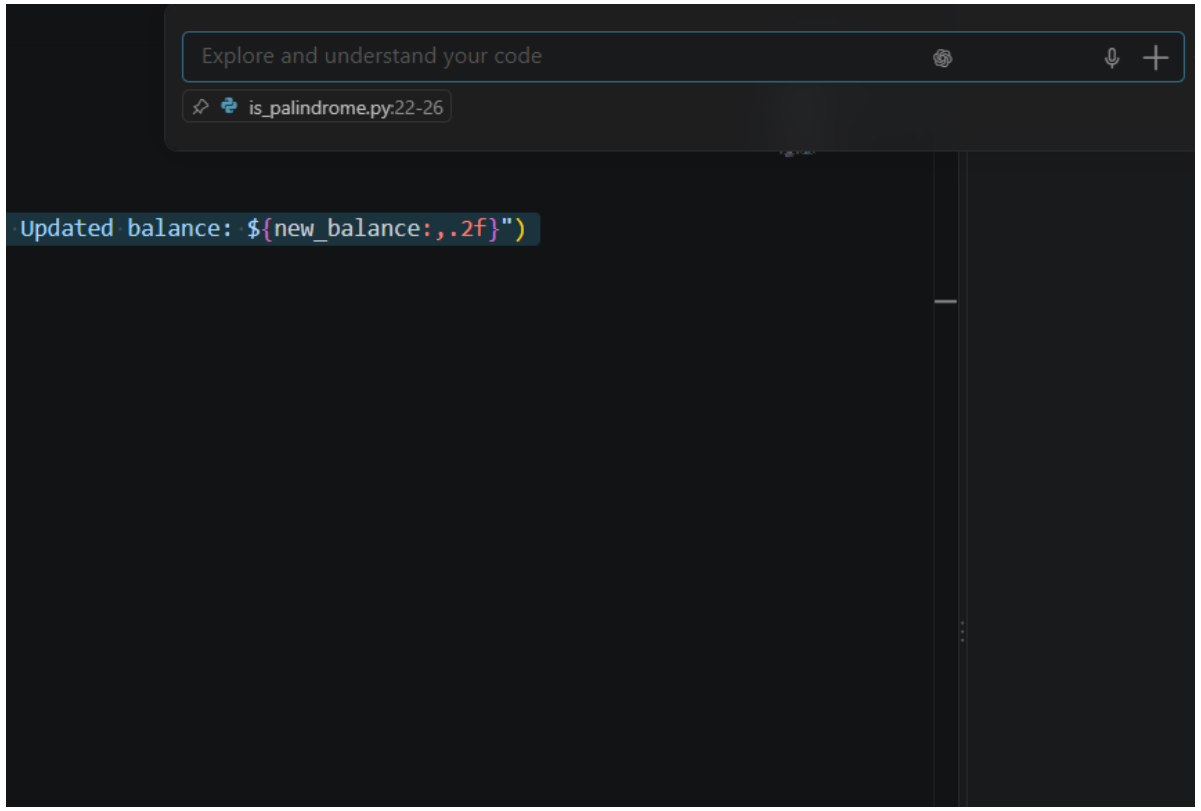


Open Chat (Chat View) opens the Chat window. This is the same as directly clicking on the button.

Open Inline Chat open a Prompt window within the VS Code source code editor at your cursor position, as shown before.

© 2026 by Innovation In Software Corporation

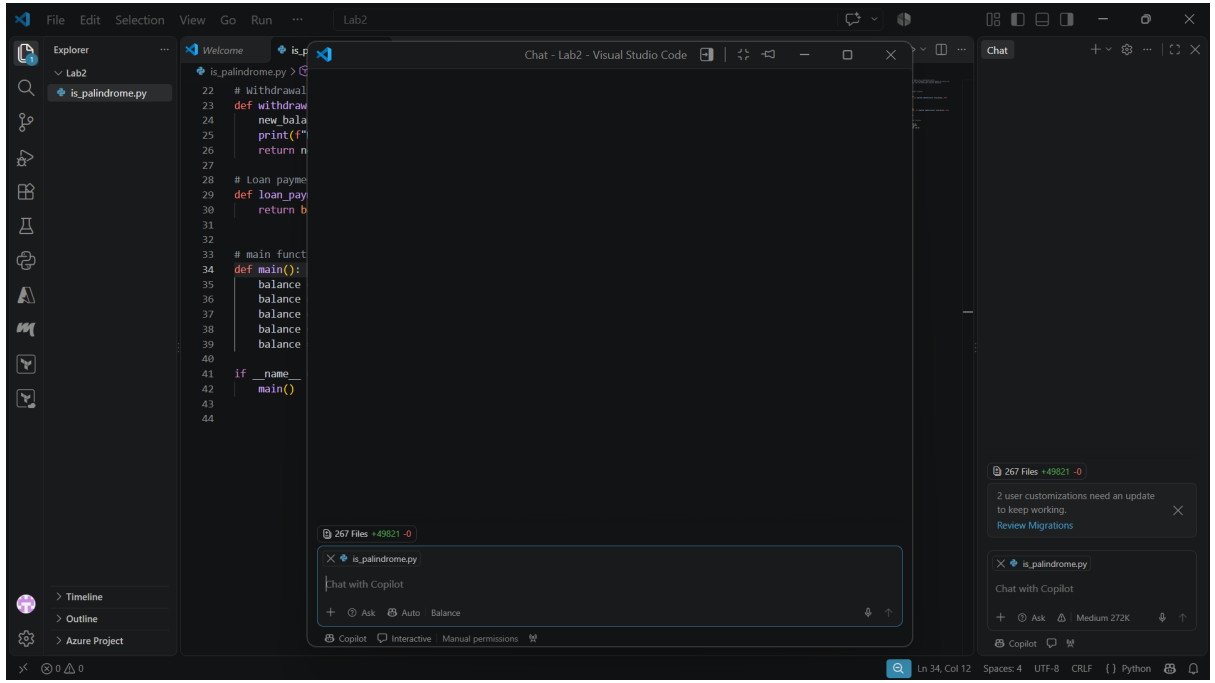
CHAT VIEW MENU - 2



From the Chat View menu, Open Quick Chat opens a lightweight GitHub Copilot chat panel directly in the editor, allowing you to submit context-aware prompts about your code, with history, without switching to the full Copilot Chat view.

© 2026 by Innovation In Software Corporation

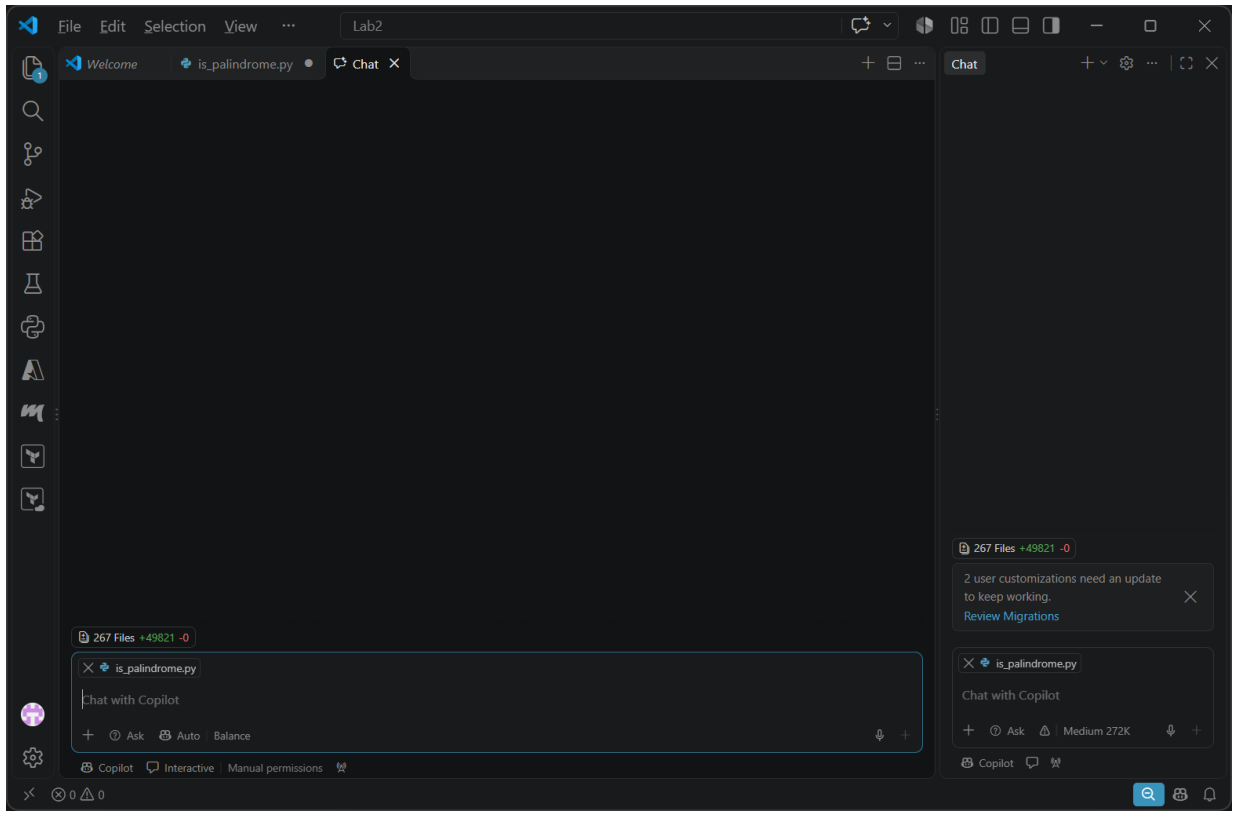
CHAT VIEW MENU - NEW CHAT WINDOW



Opens a Copilot chat session as a floating chat view. It is designed for quick, conversational interactions such as asking coding questions, generating small snippets, or explaining errors. This option is best when you need fast help without leaving your current file.

© 2026 by Innovation In Software Corporation

CHAT VIEW MENU - NEW CHAT EDITOR

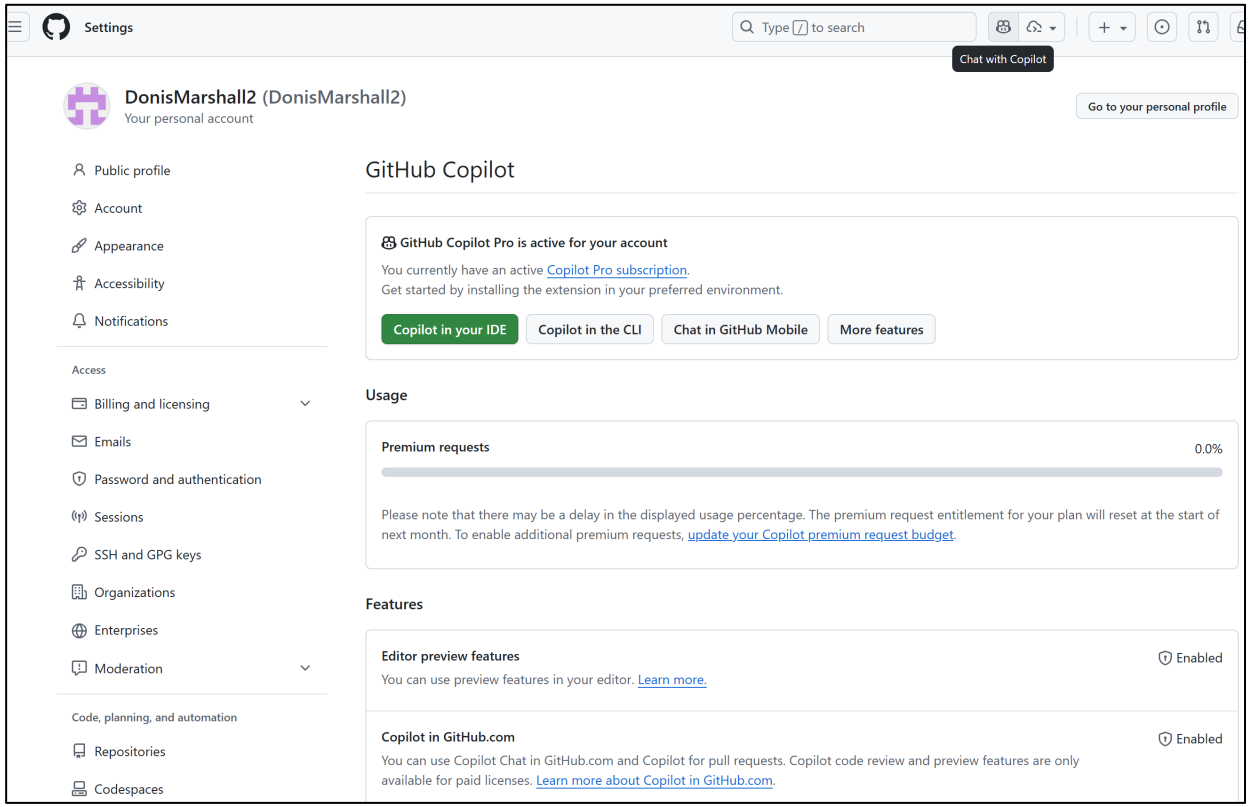


Opens a Copilot chat session as a new tab in the editor area, similar to opening a source file. This provides a larger, persistent workspace that is better suited for longer prompts, multi-step problem solving, code reviews, or architecture discussions where you want to scroll and revisit earlier responses.

© 2026 by Innovation In Software Corporation

CHAT VIEW MENU - 5

The Manage Copilot Settings menu item opens the Copilot settings (Features) for the current GitHub account.



© 2026 by Innovation In Software Corporation

OPEN IN AGENTS

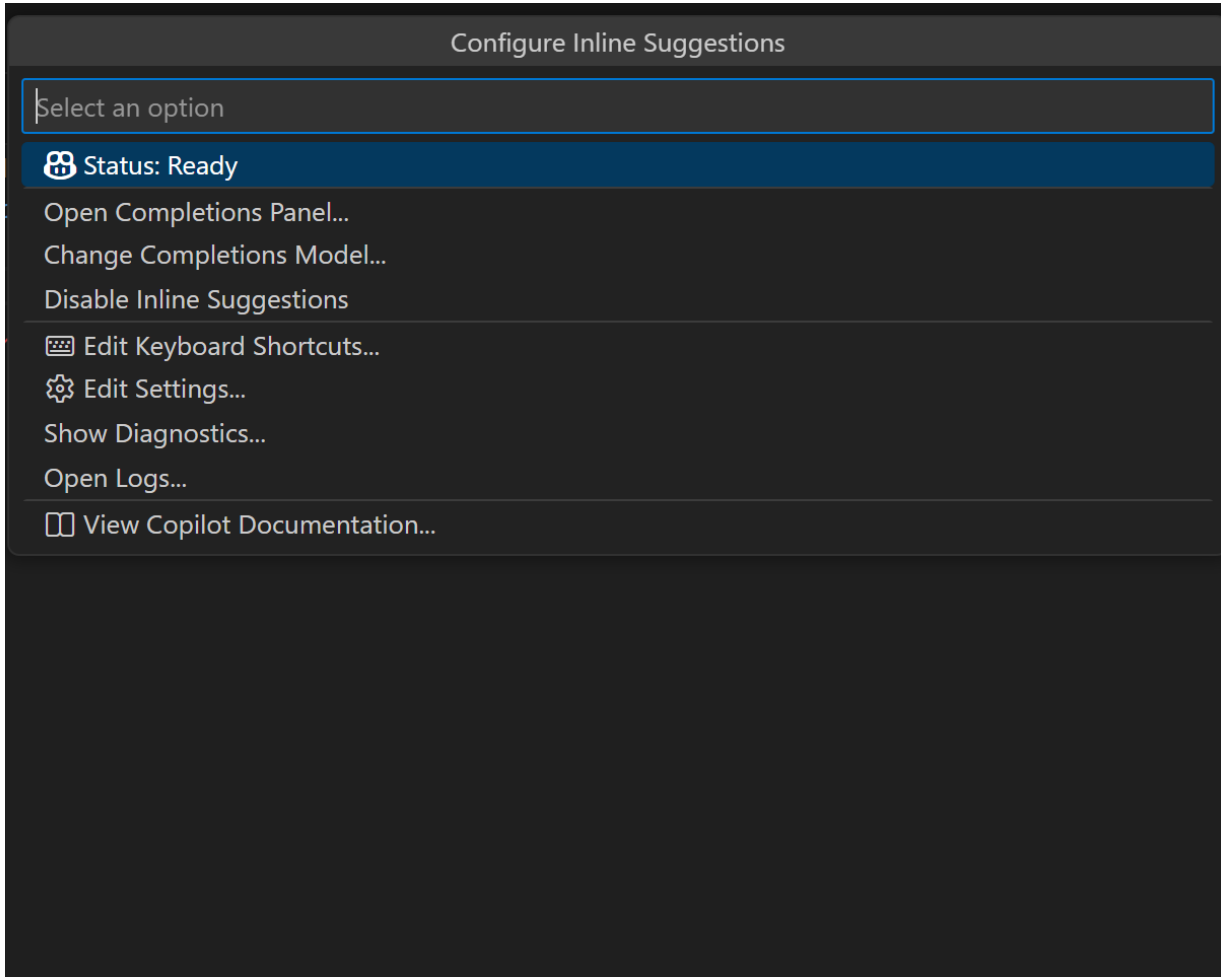
Open in Agents opens a second window where you hand jobs to Copilot instead of writing the code yourself.

In the Agents window, you give out jobs and check on them later. This window works like a to-do list of jobs. You can start one job, then start another right away, even in a different project. Each job runs on its own. The list on the left shows which jobs are working and which are done, and you click a job to see its changes. Use this when you have several tasks to run at once.

The Chat view is for coding *with* Copilot, and the Agents window is for handing jobs *to* Copilot

© 2026 by Innovation In Software Corporation

CONFIGURE INLINE SUGGESTIONS

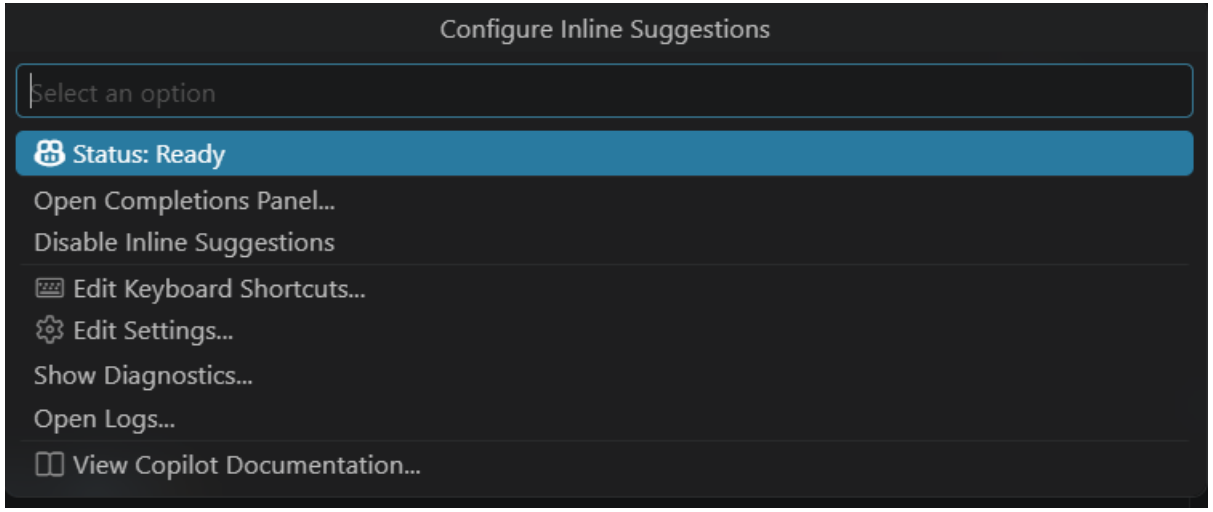


The Configure Inline Suggestions menu choice on the Chat View menu focuses on how inline suggestion behaves, including the current status.

- Status – Displays whether inline code completions is currently active.
- Open Completions Panel – Opens a dedicated panel where you can see multiple suggestions if available, as shown before.

© 2026 by Innovation In Software Corporation

CONFIGURE INLINE SUGGESTIONS - 2



- Disable Inline Suggestions: when ghost text suggestions are disabled, the Open Completions Panel becomes especially important because it allows you to request code suggestions on demand.

© 2026 by Innovation In Software Corporation

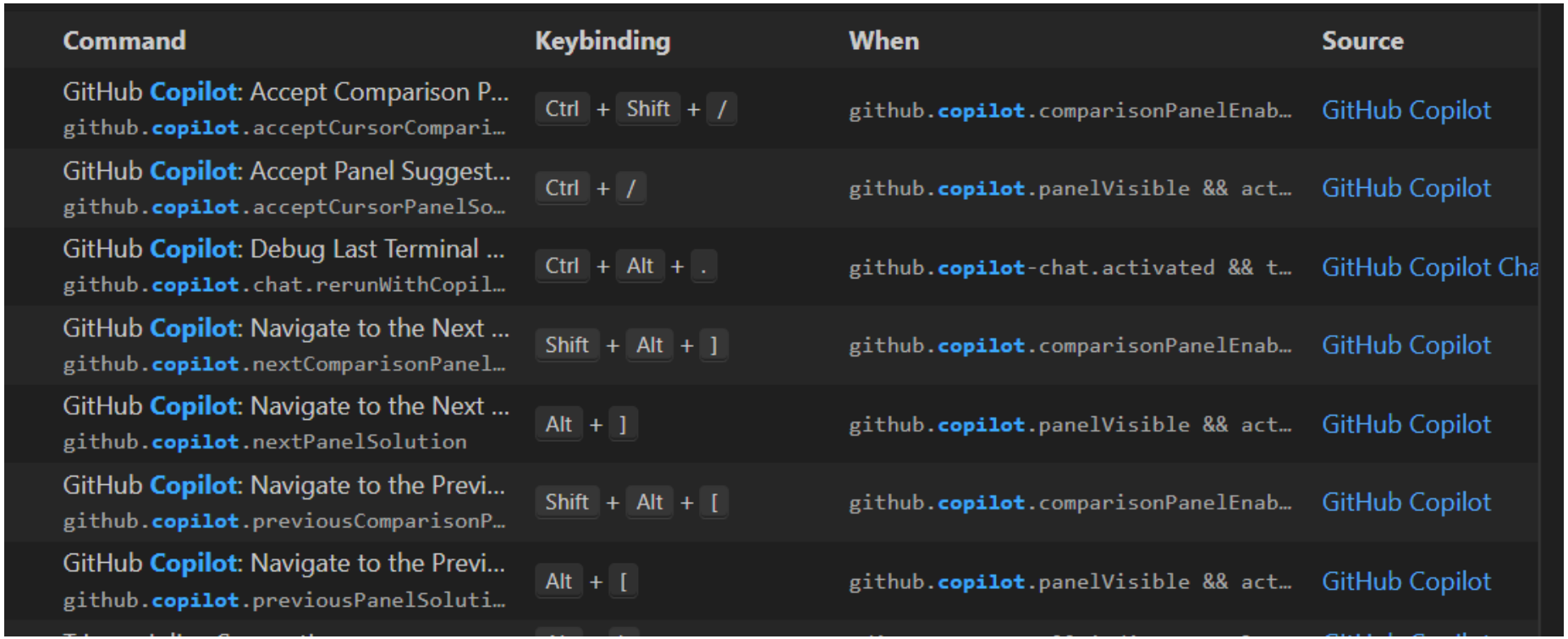
CONFIGURE INLINE SUGGESTIONS - 3

- Edit Keyboard Shortcuts – Opens the Keyboard Shortcuts editor with Copilot commands pre-filtered, so you can assign or change hotkeys for things like triggering completions or cycling through suggestions.
- Edit Settings – Opens the Copilot-related settings for VS Code where you can control behavior such as enabling completions by language, extensions, and so on.

© 2026 by Innovation In Software Corporation

EXAMPLE - EDIT KEYBOARD SHORTCUTS

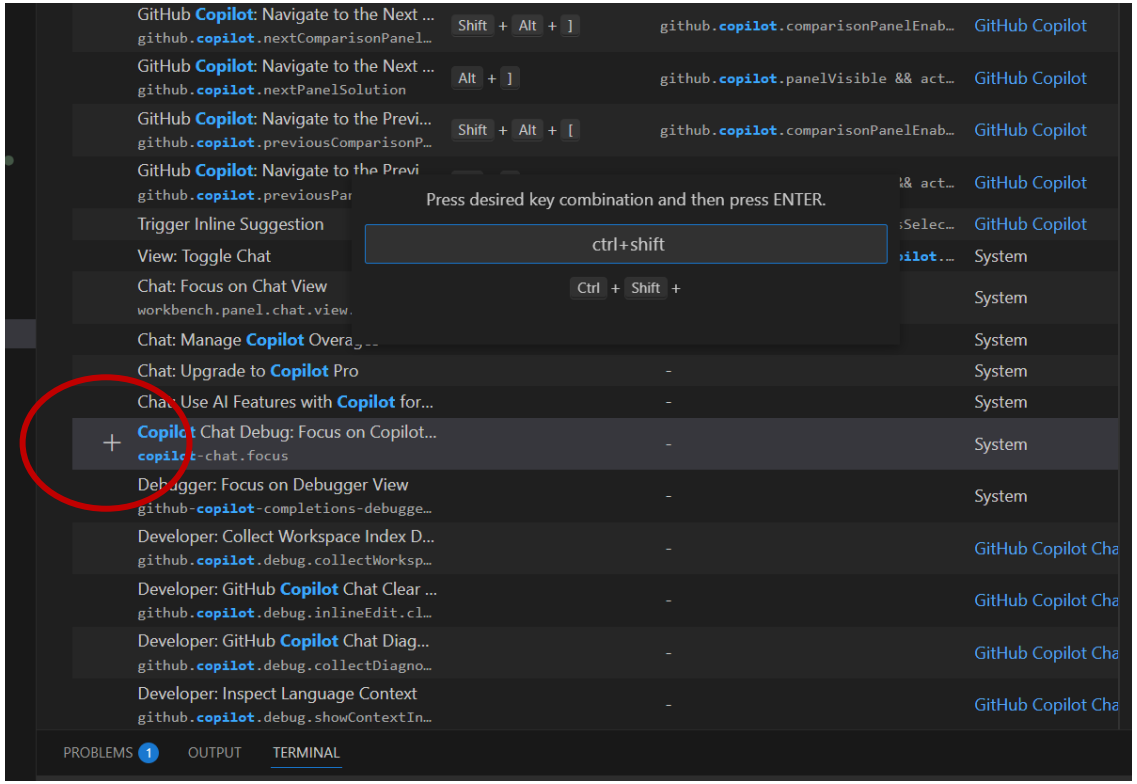
The Edit Keyboard Shortcuts option in the Copilot Chat menu opens VS Code's Keyboard Shortcuts editor with Copilot commands preselected. From there, you can assign, change, or remove shortcuts.



Command	Keybinding	When	Source
GitHub **Copilot**: Accept Comparison P... github.**copilot**.acceptCursorCompari…	Ctrl + Shift + /	github.**copilot**.comparisonPanelEnab…	GitHub Copilot
GitHub **Copilot**: Accept Panel Suggest... github.**copilot**.acceptCursorPanelSo…	Ctrl + /	github.**copilot**.panelVisible && act…	GitHub Copilot
GitHub **Copilot**: Debug Last Terminal ... github.**copilot**.chat.rerunWithCopil…	Ctrl + Alt + .	github.**copilot**-chat.activated && t…	GitHub Copilot Cha
GitHub **Copilot**: Navigate to the Next ... github.**copilot**.nextComparisonPanel…	Shift + Alt +]	github.**copilot**.comparisonPanelEnab…	GitHub Copilot
GitHub **Copilot**: Navigate to the Next ... github.**copilot**.nextPanelSolution	Alt +]	github.**copilot**.panelVisible && act…	GitHub Copilot
GitHub **Copilot**: Navigate to the Previ... github.**copilot**.previousComparisonP…	Shift + Alt + [	github.**copilot**.comparisonPanelEnab…	GitHub Copilot
GitHub **Copilot**: Navigate to the Previ... github.**copilot**.previousPanelSoluti…	Alt + [	github.**copilot**.panelVisible && act…	GitHub Copilot

© 2026 by Innovation In Software Corporation

COMMON CUSTOM KEY BINDINGS



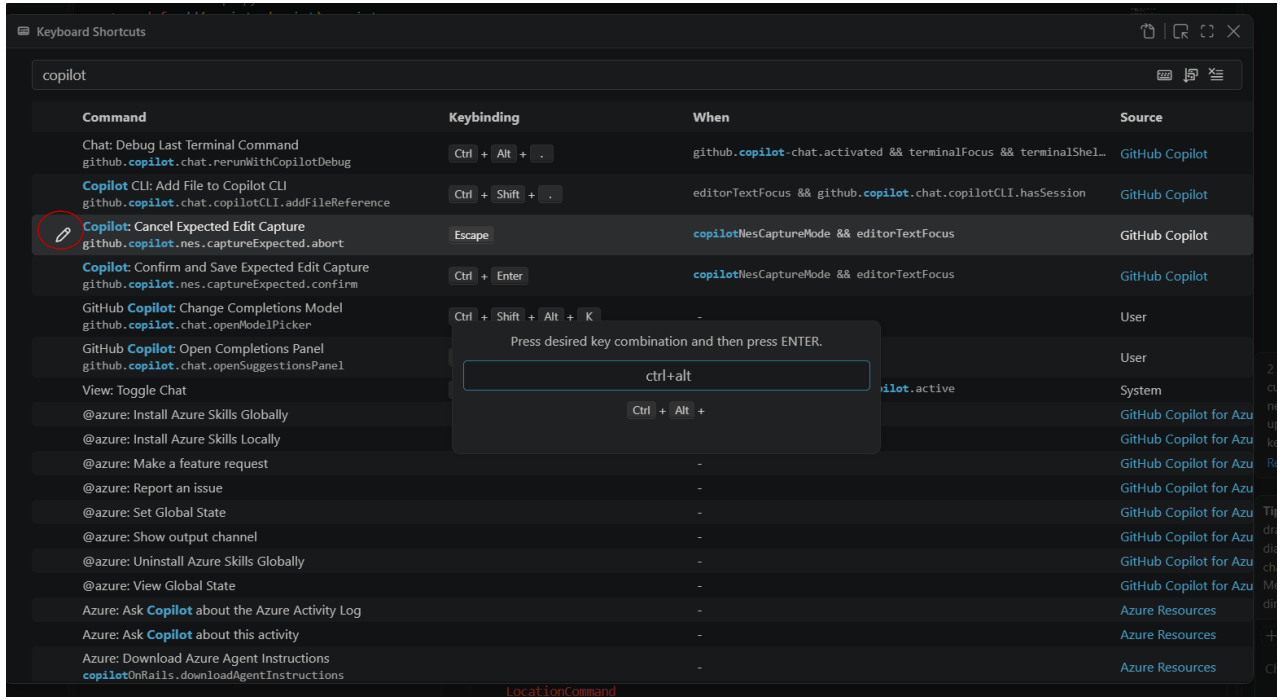
GitHub Copilot: Enable Inline Suggestions – Enables ghost text suggestions globally.

GitHub Copilot: Disable Inline Suggestions – Disable ghost text suggestions globally.

GitHub Copilot: Open Status Menu – open the Copilot status menu..

© 2026 by Innovation In Software Corporation

COMMON CUSTOM KEY BINDINGS - 2



GitHub Copilot: Open Completions Panel displays the completion panel.

Chat: Insert at Cursor – pastes a code block from a Copilot Chat response directly into your active file at the spot where your cursor is, exactly as written.

© 2026 by Innovation In Software Corporation

CONFIGURE INLINE SUGGESTIONS - 4

- Show Diagnostics (static) – Displays diagnostic information about Copilot's state and connectivity. This can help confirm whether the extension is connected properly and if requests are being sent and received.

- Open Logs (real time) – Opens the Output panel in VS Code filtered to GitHub Copilot logs. These logs capture startup messages, network errors, or authentication issues and are helpful for troubleshooting.

© 2026 by Innovation In Software Corporation

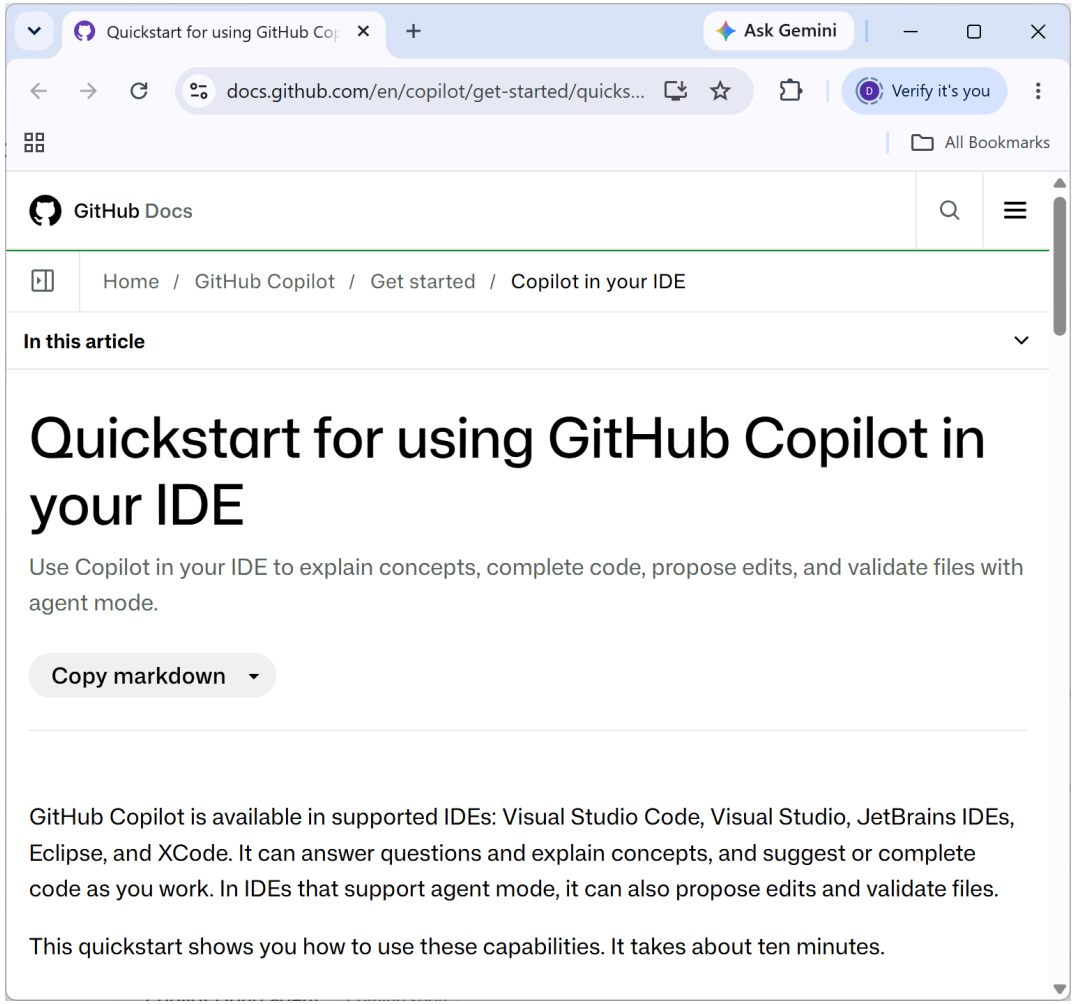
EXAMPLE - OPEN LOG

The Copilot log in VS Code is a diagnostic record of what the extension is doing behind the scenes. It captures events like when Copilot starts up, sends or receives a completion request, or encounters an error. You can view it by opening the Output panel and selecting GitHub Copilot from the dropdown list.

These logs are mainly for troubleshooting. If Copilot isn't generating suggestions or fails to connect, the log can reveal issues such as authentication errors, network timeouts, or extension crashes.

© 2026 by Innovation In Software Corporation

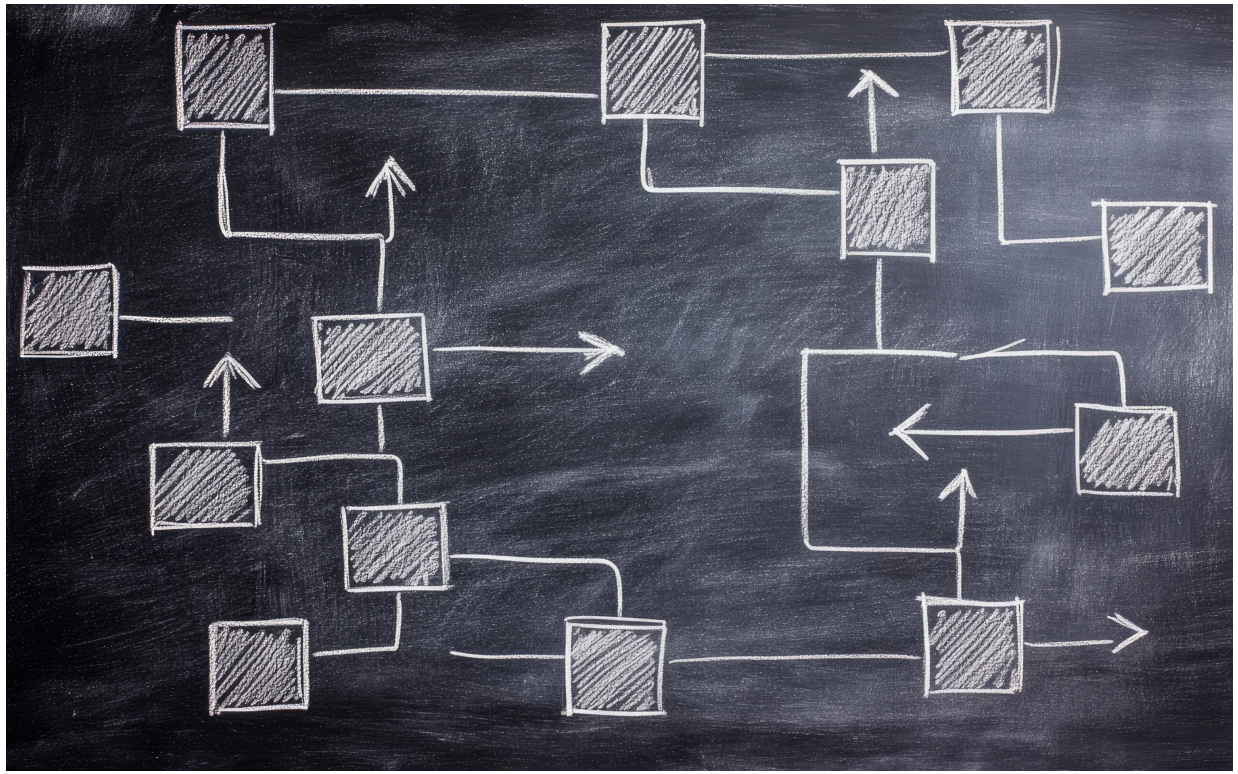
VIEW COPILOT DOCUMENTATION



The View Copilot Documentation menu item opens online help for GitHub Copilot, including a Quickstart.

© 2026 by Innovation In Software Corporation

MODES



These modes control how much the agent does on its own before checking with you. Think of hiring a painter:

- Interactive: you work together, one step at a time.
  The agent does a little, shows you, and waits for your next instruction. Like a painter who asks, “This wall next?” Best for beginners and for staying in control.

© 2026 by Innovation In Software Corporation

MODES – 2

- Plan: it makes a plan first, then waits for your OK.
  The agent writes a step-by-step plan but changes nothing yet. You review it, adjust it, and then tell it to go. Like a painter who gives you a quote before starting. Best for bigger tasks, so you catch mistakes early.
- Autopilot: it does the whole job by itself.
  The agent keeps working until the task is done, without stopping to ask questions. Like giving the painter your key and coming back later. Best for clear tasks you trust it with.

© 2026 by Innovation In Software Corporation

PERMISSIONS

These options decide when the agent must ask you before it acts, such as running a command or editing a file.

- Manual permissions (default): it asks you unless you already said yes.
  If you previously told VS Code "always allow this command" or "always allow this tool," the agent goes ahead. For everything else, it stops and asks you. This is the safest choice and the best one for beginners.

© 2026 by Innovation In Software Corporation

PERMISSIONS -2

- Assisted permissions (experimental): a second AI checks each action. Before each action, another AI judges whether it looks safe. Safe-looking actions run on their own; anything risky still comes to you for approval. Fewer interruptions, but it is still experimental and needs extra setup (folder isolation and a setting turned on), so it is not available in every session.
- Allow all: it never asks. The agent runs every command and edit without checking with you. It is fast, but only use it in a project you fully trust, ideally a throwaway folder.

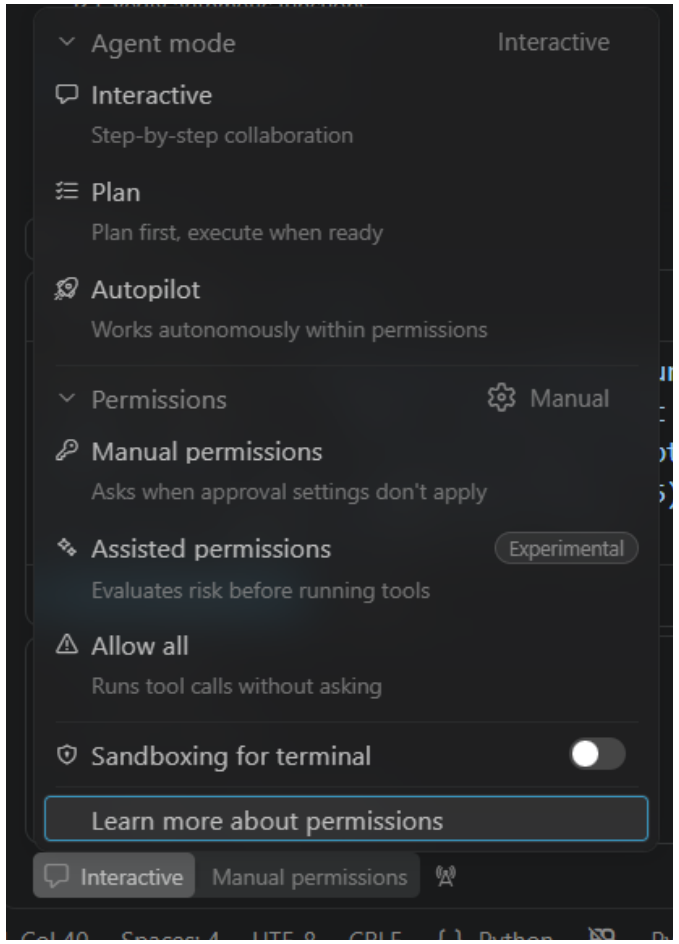
© 2026 by Innovation In Software Corporation

PERMISSIONS -3

- Sandboxing for terminal: a fence around commands.
  This is a separate on/off switch. When it is on, terminal commands the agent runs can only reach limited files and network locations. It still protects you even if you choose Allow all or Autopilot.

© 2026 by Innovation In Software Corporation

PERMISSIONS - 4



In the GitHub Copilot Chat window in VS Code, the default mode is Interactive. To change it, click the mode selector at the bottom of the chat panel and choose another mode such as Plan from the dropdown. The chat session then switches to the selected mode.

© 2026 by Innovation In Software Corporation

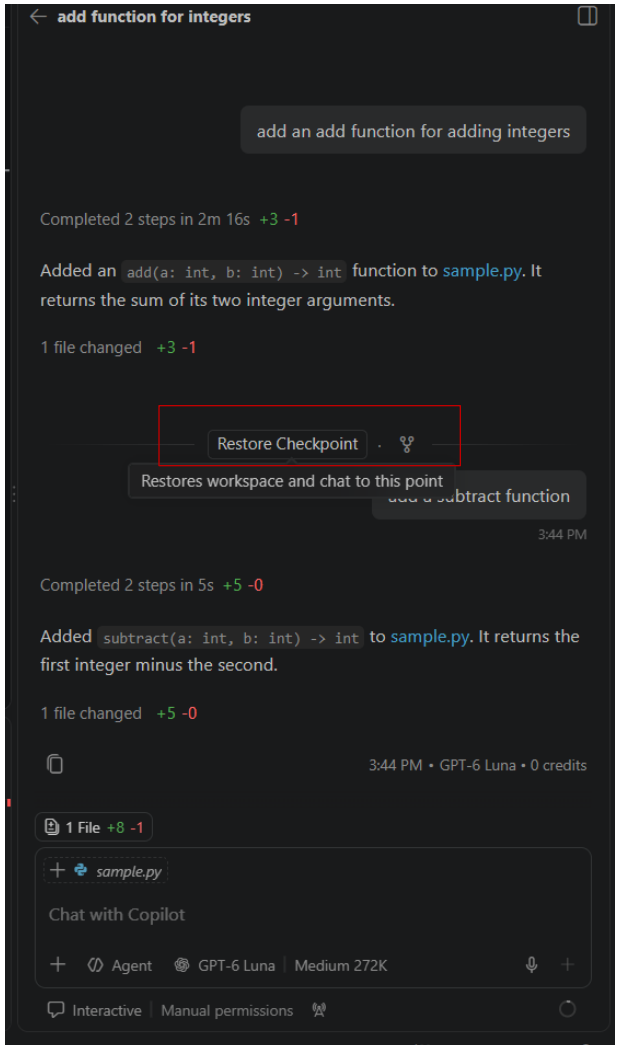
CHECKPOINTS

In the Copilot Chat side panel, the history of your interactions is preserved in the chat conversation. Each request and Copilot's corresponding suggestion are shown in sequence, making it easy to scroll back and review prior edits or additions without losing context. This gives you a running log of how your code evolved through chat-based instructions.

At the same time, checkpoints are placed directly in your code editor as a thin separated. These indicate where Copilot made a change. If you hover over the line, you'll see the checkpoint, which allows you to quickly roll back (Restore Checkpoint) to the earlier version of your code. This provides a convenient safety net, ensuring that edits suggested through the chat panel are reversible.

© 2026 by Innovation In Software Corporation

CHECKPOINTS - 2



In the Copilot Chat pane, checkpoints appear when hovering over the intersection of prompts.

© 2026 by Innovation In Software Corporation

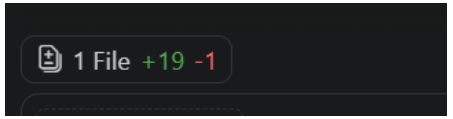
CHECKPOINTS - 3

VS Code saved a snapshot of your files and the chat right before you sent this message.

- Selecting Restore Checkpoint rolls both back to that snapshot: any code changes the agent made afterward are undone, and the later chat messages are removed. Branch icon (fork the conversation): try a different path, keep the original.
  The small branch icon next to it starts a new chat that copies the conversation up to this point. Your original chat stays untouched, so you can try a different request in the new chat and compare.

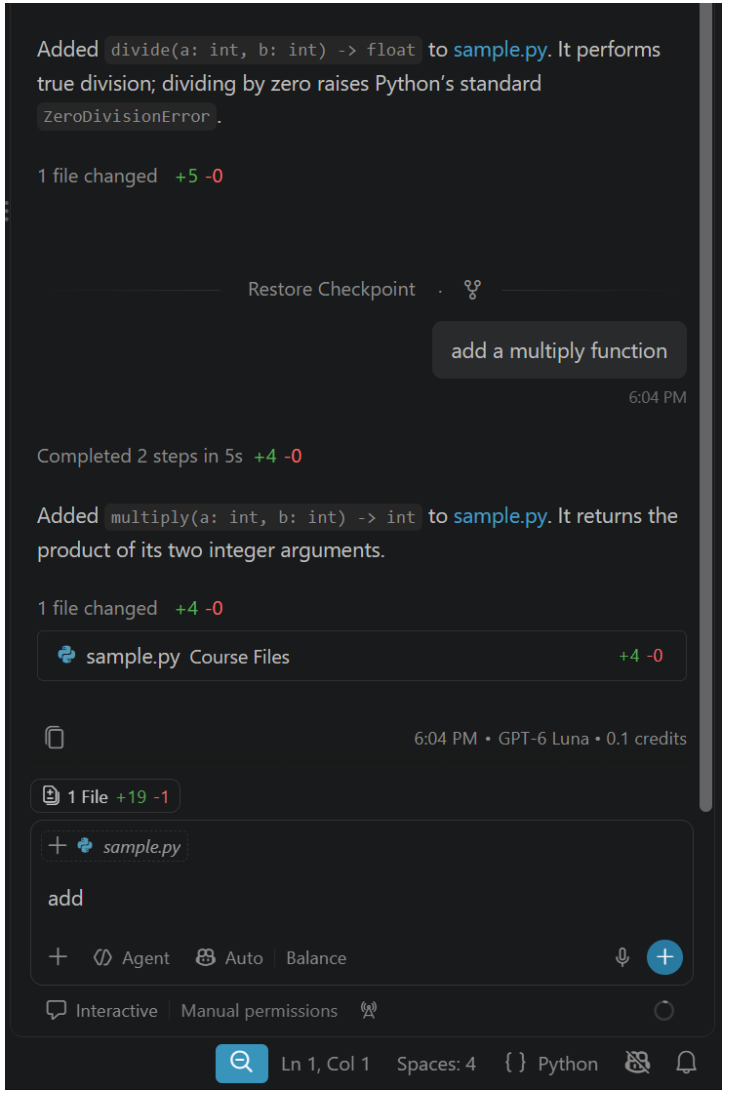
© 2026 by Innovation In Software Corporation

CHANGED-FILES SUMMARY



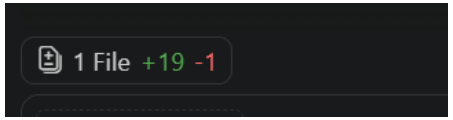
Copilot now edits your file right away. There is no Apply or Keep step anymore. When you prompt “add a multiply function,” Copilot writes it straight into sample.py. The Review Changes Pane (“1 file changed +4 -0”) is just a report of what it already did.

You review changes afterward. Select 1 File +19 -1 above the chat box to see every line Copilot added or remove



© 2026 by Innovation In Software Corporation

CHANGED-FILES SUMMARY - 2



That button is the changed-files summary. It shows how many files Copilot changed in this chat (1 File), how many lines it added (+19, green), and how many it removed (-1, red).

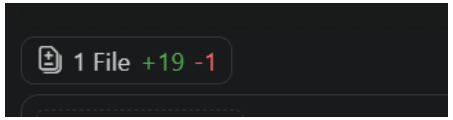
To see the diff view:

1. Select 1 File +19 -1. VS Code opens a diff editor showing all of Copilot's changes in one place.
2. In the diff editor, read it like a before-and-after:
   - Green lines were added by Copilot.
   - Red lines were removed.
   - Unchanged code appears in plain color around them for context.

If Copilot changed several files, scroll down: each file gets its own section in the same view.

© 2026 by Innovation In Software Corporation

CHANGED-FILES SUMMARY - 3



sample.py Course Files

11 hidden lines

```
12      """Return the result of dividing the first integer by th
13      return a / b
14
15
```

11 hidden lines

```
12      """Return the result of dividing the first integer by th
13      return a / b
14
15+
16+ def multiply(a: int, b: int) -> int:
17+      """Return the product of two integers."""
18+      return a * b
19
```


© 2026 by Innovation In Software Corporation

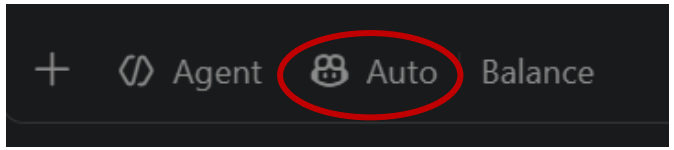
OTHER FEATURES

© 2026 by Innovation In Software Corporation

AUTO LLM

The Auto selection is the model picker,

- Auto: Copilot picks one model for you.
  For each request, Copilot chooses a single model it thinks fits best, so you do not have to pick one yourself. Think of it as a receptionist sending you to the right expert. It is the simplest choice and the best default for beginners.
- HydraFusion: Copilot makes several models work as a team.
  Instead of picking one model, HydraFusion picks a *way of working* and uses several models within the same answer. It uses one of three approaches:



© 2026 by Innovation In Software Corporation

AUTO LLM - 2

Auto May Choose the Wrong Model

Auto may sometimes select a model that is not ideal for the task. For example, it might prioritize a faster general-purpose model instead of a specialized coding model, which can produce different results or cause certain workflows to behave differently.

Overriding Auto

Developers can override Auto by manually selecting a specific model in the Copilot Chat model selector. Choosing a model such as GPT-5.2-Codex or GPT-5.3-Codex forces Copilot to use that model for the session and provides more consistent behavior for coding tasks.

© 2026 by Innovation In Software Corporation

VARIABLES

Copilot fully understands variables you've declared in the codebase.

Example:

```
total = price * quantity
```

If you then write a comment like

```
# calculate tax on total
```

Copilot knows total is a variable and will suggest code like:

```
tax = total * 0.08
```

Copilot's completions are context-aware. It looks at variable names, types, and surrounding code when generating suggestions.

© 2026 by Innovation In Software Corporation

VARIABLES - 2

Copilot can sometimes get confused if you have several variables with similar or the same name. To disambiguate in a Copilot prompt, you give it extra clues.

1. Mention the Type or Purpose

If you have both user (funca) and user (funcb), you can guide Copilot by saying:

Instead of:

“Print the user variable"

Say:

“Print the user variable from funcb"

© 2026 by Innovation In Software Corporation

VARIABLES - 3

3. Use Comments Near the Code

Copilot pays attention to comments. Right above where you're working, add:

```
# Use the username variable, not the user object
print(username)
```

Copilot will now autocomplete correctly because you've clarified in plain English.

© 2026 by Innovation In Software Corporation

VARIABLES - 4

4. Use Explicit Prompt Language

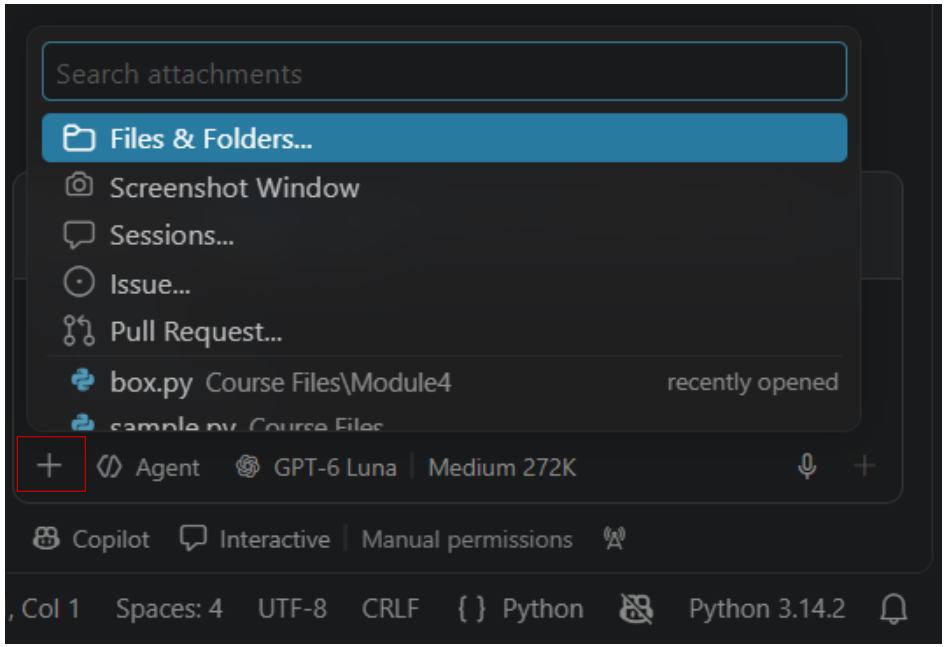
Instead of just saying “fix this,” be very specific:

Good: “Loop through customer_list (the array of names), not customer (the object).”

Bad: “Loop through customers.” (too vague)

© 2026 by Innovation In Software Corporation

CONTEXT BUTTON



The Add Context button, the plus (+) button, is provided to add items to the context, such as files.

It will open a list of resources that can be added to the context.

The current file is automatically included in the context.

© 2026 by Innovation In Software Corporation

#CONTEXT

The # context items let you give Copilot Chat extra context by referencing files, your whole workspace, or web content directly in your prompt. Type #, then pick from the list that appears:

- #codebase tells Copilot to search your entire workspace.
- #<filename> points to a specific file. For example, #style.css adds style.css to the context.
- #fetch pulls in content from a web page, or you can paste a URL directly.

Copilot also finds relevant files on its own, so use # when you want to point it at something specific.

© 2026 by Innovation In Software Corporation

#CONTEXT EXAMPLE

Here's a short example of using file path in Copilot Chat: Here is the project structure:

```
project/
  src/
    utils/
      math.py
  styles/
    main.css
```

Prompt in Copilot Chat:

Explain how the functions are organized #file:src/utils/math.py.

Suggest improvements to button styling #file:styles/main.css.

Result:

Copilot looks at math.py to describe your utility functions. It also uses main.css to suggest design tweaks.

© 2026 by Innovation In Software Corporation

#SELECTION

```python
def process_data(records):
    cleaned = []
    for r in records:
        if r is not None and r != "" and r.strip() != "":
            cleaned.append(r.strip().lower())
    # ... more logic for filtering, transforming, exporting ...
    return cleaned
```

You can focus suggestions on selected code with #selection, such as a portion of a function.

For example:

#selection optimize

© 2026 by Innovation In Software Corporation

CODEBASE

When you type #codebase in Copilot Chat, you're explicitly telling Copilot:

> "Use my entire project as context for this request."

Normally, Copilot only looks at:

- The current file,
- Recently opened files, or

By mentioning #codebase, you override that and attach the whole project folder (everything in your workspace index) as extra context.

© 2026 by Innovation In Software Corporation

COMMANDS

You can use the slash commands to steer Copilot into specific behaviors.

Common Copilot Chat Commands

- /fix: Suggest fixes for errors or issues in the selected code.
- /explain: Explain what the selected code does in plain language.
- /tests: Generate unit tests for the selected code.

© 2026 by Innovation In Software Corporation

HOW TO FIX

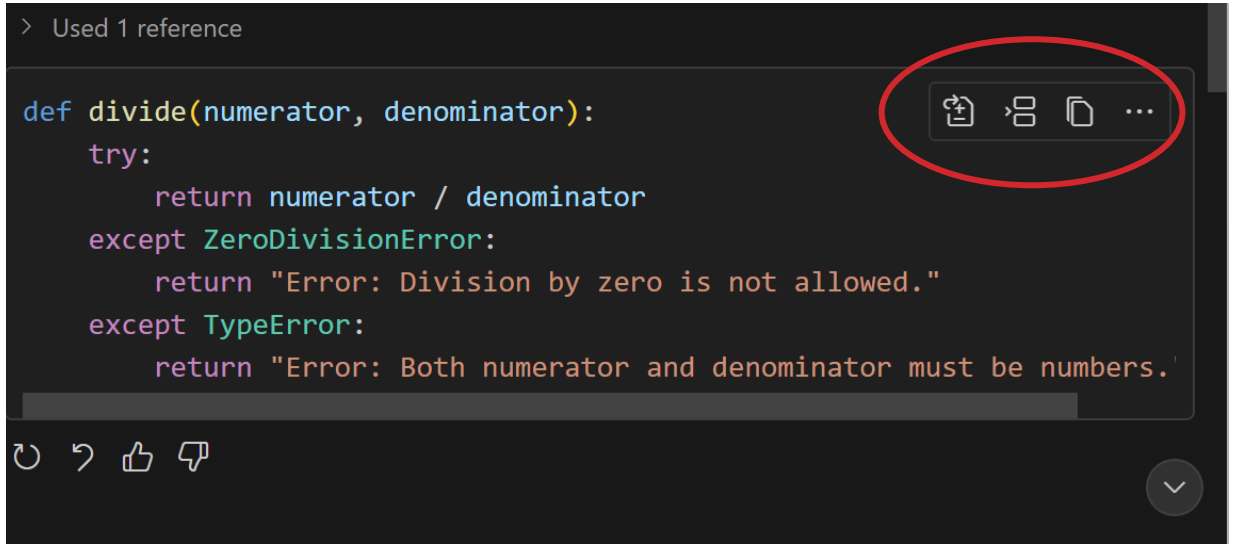
Here are steps to fix:

- Open filter_values.py, select the function, and ask Copilot to show the fix in chat instead of editing the file. For example:
- Fix the bug in the selected code. Show the corrected code in chat as a code block. Don’t edit any files.
- Hover over the code block, and the toolbar appears at the top-right. Select Apply (the first icon), and Copilot merges the fix into filter_values.py. You can then review the change and keep it, or undo it with Ctrl+Z.
- The key words are “Show … in chat” and “Don’t edit any files.” Without them, the agent fixes the file directly, and no Apply button appears.

© 2026 by Innovation In Software Corporation

ACTION BAR

When in Copilot Chat, suggestions are first presented in the Chat window. Hovering over that suggestion reveals four action buttons that let you decide how to handle (action) the suggestion.



© 2026 by Innovation In Software Corporation

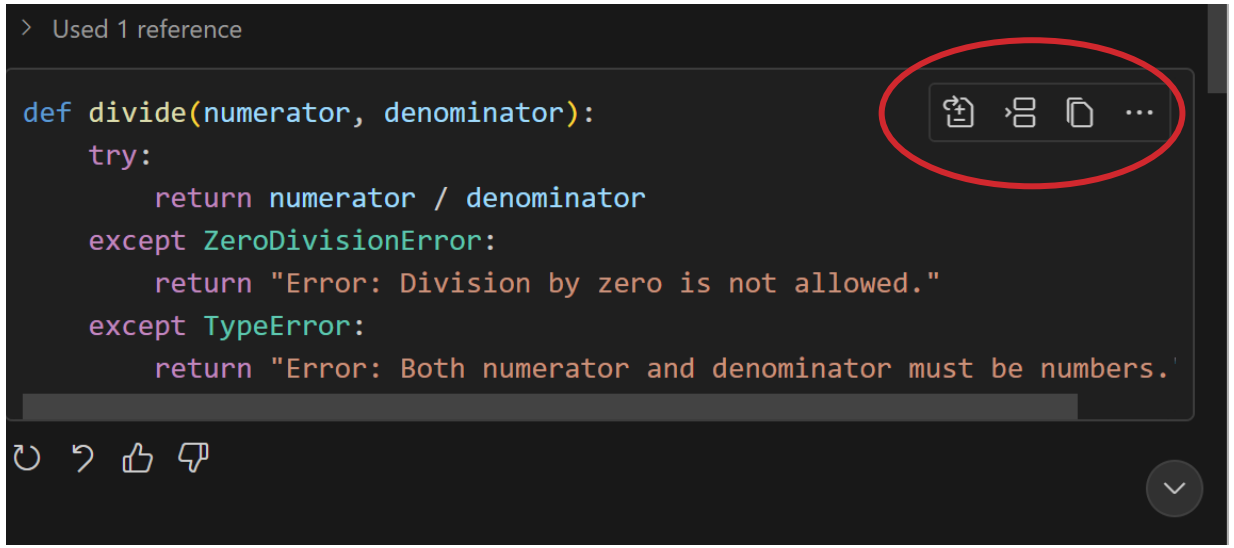
ACTIONS

Apply To: Applies the suggestion directly to the code you had selected in the editor.

Insert At: Lets you insert the suggestion at your current cursor location in the open file.

Copy: Copies the suggestion to your clipboard.

More menu: . . .



© 2026 by Innovation In Software Corporation

/EXPLAIN

```
def calculate_prime(n: int) -> bool:
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Explains the code you have selected or the file that's active in the editor. Great for onboarding to an unfamiliar area or validating logic before changes.

© 2026 by Innovation In Software Corporation

/FIX

```
def divide_numbers(a, b):
    return a / b


print(divide_numbers(10,
0))
```

Analyzes the selected code and proposes a patch for errors or suspicious patterns. You'll get a diff you can accept or tweak.

© 2026 by Innovation In Software Corporation

/TESTS

```
def factorial(n: int) -> int:
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
```

Generates unit tests for your active file or selection, including typical and edge cases. You can specify frameworks or constraints.

© 2026 by Innovation In Software Corporation

/FIXTESTFAILURE

```
def parse_int_list(s: str) -> list[int]:
  return   [int(x) for x in s.split("")]
```

Targets a failing test and suggests code changes to make it pass. Use when your test runner (or CI) is red and you want a focused remedy.

© 2026 by Innovation In Software Corporation

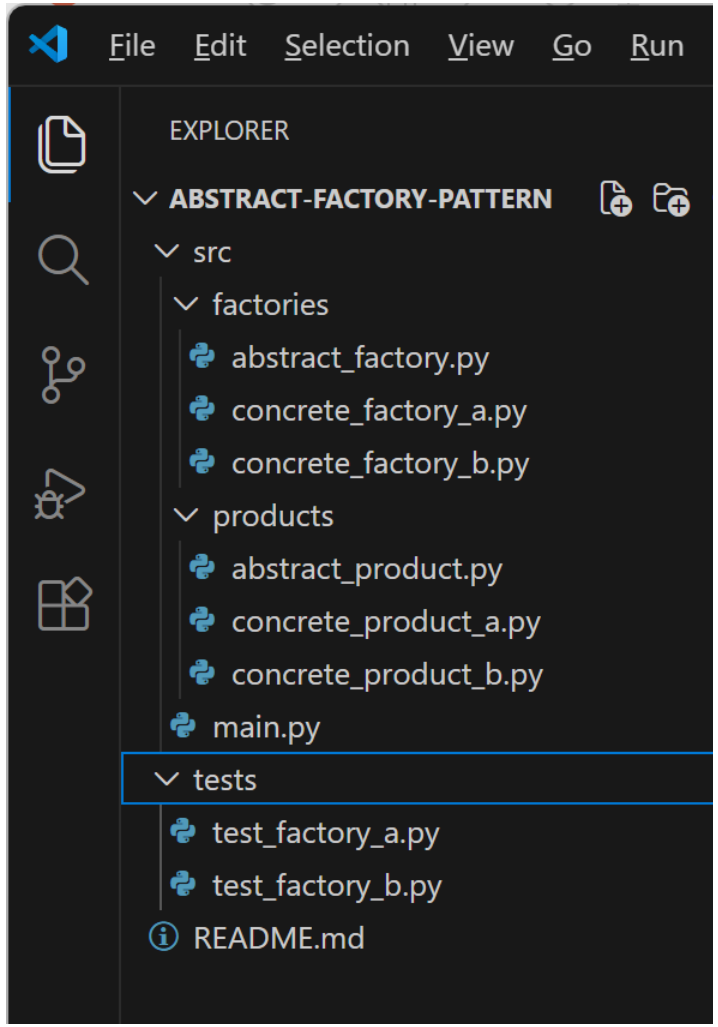
/NEW

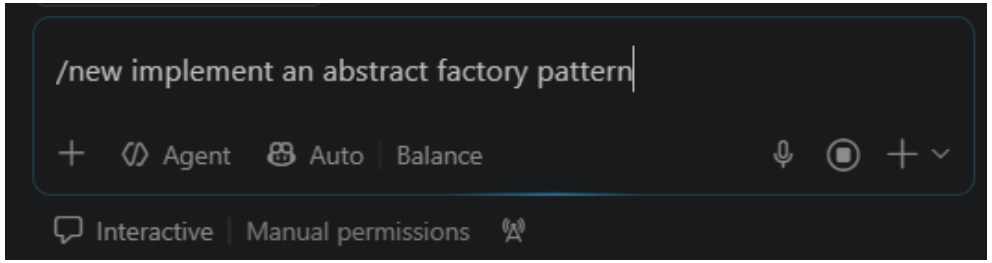
The /new command scaffolds a new project or starter code in your workspace. Type /new followed by a description of what to generate, for example: */new Create a Python REST API application.*

The /new command does not reset chat history. It creates new files or starter code based on your instructions.

© 2026 by Innovation In Software Corporation

/NEW - 2





© 2026 by Innovation In Software Corporation

/CLEAR

In GitHub Copilot Chat, the /clear command is used to reset the current chat session. When you enter /clear, Copilot removes the existing conversation history from the active chat window, allowing you to start fresh without prior context influencing responses.

This command only clears the chat memory, not your workspace, files, or project. It is useful when switching topics, correcting earlier misunderstandings, or ensuring previous prompts do not affect new requests.

© 2026 by Innovation In Software Corporation

POP QUIZ: ARE THEY THE SAME?

Is the /fix command and fix prompt the same?

10 MINUTES

© 2026 by Innovation In Software Corporation

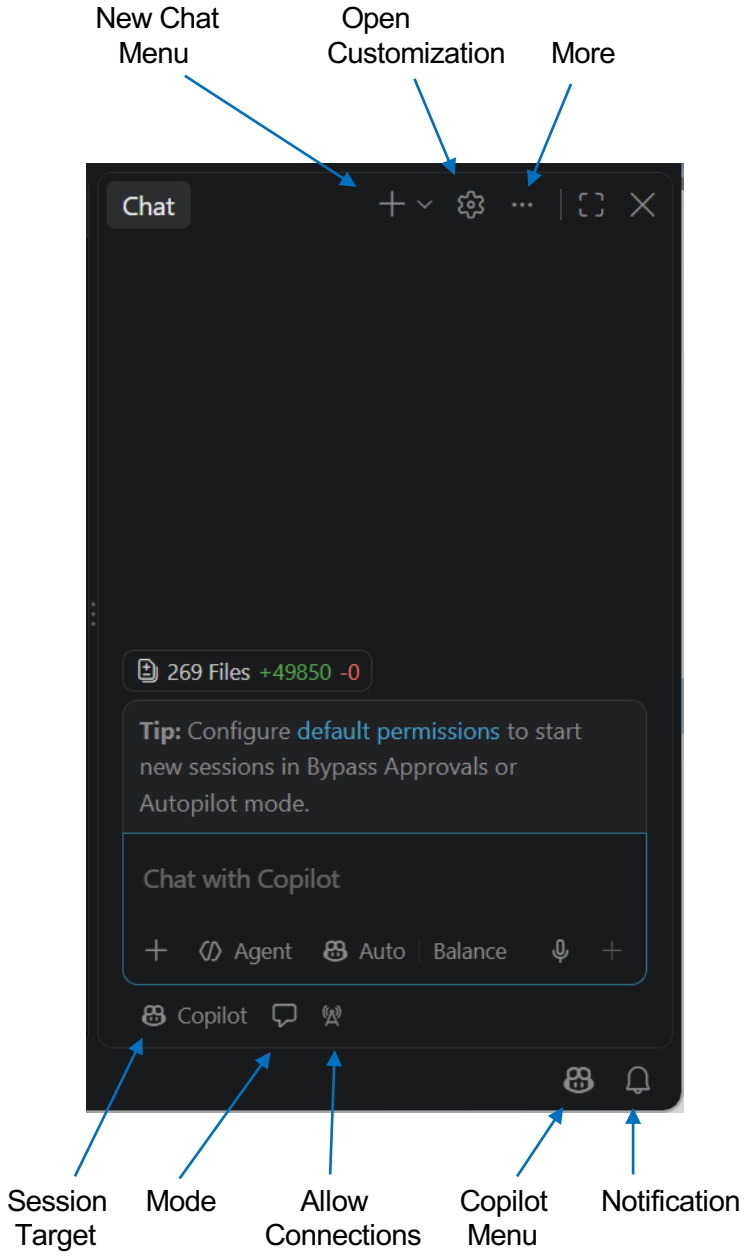
FIX VERSUS /FIX

Slash commands like /fix and /explain are basically shortcuts that tell GitHub Copilot what role you want it to play. When you type /fix, you're saying "this code is broken — please correct it," and Copilot tries to make the smallest, safest changes it can. When you type /explain, you're saying "don't change anything — just tell me what this does," so Copilot focuses on describing the code instead of rewriting it.

If you type "fix this" or "explain this" in plain English, Copilot is usually smart enough to figure out what you mean – the result often appears the same. The slash commands make Copilot more careful and predictable.

© 2026 by Innovation In Software Corporation

CHAT INPUT BOX



© 2026 by Innovation In Software Corporation

SHORTCUT KEYS

Action	Windows/Linux	macOS
Open Copilot Chat panel	`Ctrl` + `Alt` + `I`	`Ctrl` + `Cmd` + `I`
Open Inline Chat (editor overlay)	`Ctrl` + `I`	`Cmd` + `I`
Open Quick Chat	`Ctrl` + `Shift` + `Alt` + `L`	`Shift` + `Option` + `Cmd` + `L`
New Chat	`Ctrl` + `N` (in Chat view)	`Cmd` + `N` (in Chat view)
Accept suggestion	`Tab`	`Tab`
Dismiss suggestion	`Esc`	`Esc`
Submit prompt in chat	`Enter`	`Enter`
Newline in chat prompt	`Shift` + `Enter`	`Shift` + `Enter`
Explain selected code	Select code, `Ctrl` + `I`, then `/explain`	Select code, `Cmd` + `I`, then `/explain`

© 2026 by Innovation In Software Corporation

RETROSPECTIVE

This module explains how to use GitHub Copilot Chat in VS Code across Chat View, Inline Chat, and Quick Chat, and it helpfully walks through the Copilot Chat/Completions menus (status, open completions panel, disable completions, shortcuts, settings, diagnostics/logs).

It also covers modes (Interactive, Plan, and Autopilot), core slash commands (/fix, /tests, /explain, /edit), checkpoints and the action bar

© 2026 by Innovation In Software Corporation

LAB 4 – MOBY DICK

The goal of this lab is file processing – starting with Moby Dick.

MOBY DICK

Call me Ishmael. Some years ago—never mind how long precisely—having little or no money in my purse, and nothing particular to interest me on shore, I thought I would sail about a little and see the watery part of the world. It is a way I have of driving off the spleen and regulating the circulation. Whenever I find myself growing grim about the mouth; whenever it is a damp, drizzly November in my soul; whenever I find myself involuntarily pausing before coffin warehouses, and bringing up the rear of every funeral I meet; and especially whenever my hypos get such an upper hand of me, that it requires a strong moral principle to prevent me from deliberately stepping into the street, and methodically knocking people's hats off—then, I account it high time to get to sea as soon as I can. This is my substitute for pistol and ball…

© 2026 by Innovation In Software Corporation

SETUP

- Open VS Code
- Find and open the Lab4 folder. Within that folder, create a new folder: mobydick.
- Alternately, create a new folder called mobydick somewhere convenient.
- Within the mobydick folder, create a new file: textsummary.py.

© 2026 by Innovation In Software Corporation

FILE PROCESSING

Goal: Use Copilot Chat to perform text processing

1. Open textsummary.py.
2. Add a comment:

- # Read a file and summarize the number of lines, words, and characters
- Accept Copilot's suggestion.

3. Copy mobydick.txt from the Module4 directory for sample files to the mobydick directory.
4. Run the script and verify output.

© 2026 by Innovation In Software Corporation

REVISING

Ask Copilot Chat:

- Suggest improvements for handling empty files
- Add error handling if mobydick.txt is missing
- Display the run time of the script in milliseconds

2. Refactor the script to support command-line arguments so different file names can be provided.

3. Use the /tests command to create unit tests, using the unittest library.

4. Run the unit test(s)

© 2026 by Innovation In Software Corporation

Lab completed

COMMON TASKS

ERROR HANDLING, SYNTAX CHECKING, AND MORE

INTRODUCTION

This module introduces GitHub Copilot Chat as a practical debugging partner: it spots and explains common mistakes (e.g., removing items while iterating a list), scaffolds real error handling with clear try/except patterns, helps fix syntax slips across languages, and—most importantly—catches “silent” logic errors that still run but produce wrong results.

© 2026 by Innovation In Software Corporation

EXAMPLE

```
def remove_negatives(numbers):
    for n in numbers:
        if n < 0:
            numbers.remove(n)
    return numbers

data = [3, -1, -2, 5, 7]
print(remove_negatives(data))
```


© 2026 by Innovation In Software Corporation

POP QUIZ: PROBLEM

Does the previous code have a syntactical or logical error?

10 MINUTES

© 2026 by Innovation In Software Corporation

COPILOT CHAT – THE FIX

```
def remove_negatives(numbers):
    return [n for n in numbers if n >= 0]

data = [1, -1, -2, 3, -4, -5, 6]

print(remove_negatives(data))
```

Yes, there is an unseen error.

The issue is that you're modifying the list while iterating over it. When you remove an element during iteration, Python shifts the remaining items left, so the loop skips over some elements.

© 2026 by Innovation In Software Corporation

ERROR HANDLING

Error handling is an area where Copilot shines by quickly generating try-except or try-catch structures. Many developers overlook robust error handling in the early stages of coding.

Copilot can encourage better practices by suggesting error handling blocks automatically.

© 2026 by Innovation In Software Corporation

EXAMPLE

```
def compute_average(filename):
    with open(filename, "r") as f:
        lines = f.readlines()
        total = sum(int(line.strip()) for line in lines)
        avg = total / len(lines)
    return avg
```


© 2026 by Innovation In Software Corporation

WITH ERROR HANDLING

```
def compute_average(filename):
    try:
        with open(filename, "r") as f:
            lines = f.readlines()
            if not lines:
                raise ValueError("The file is empty.")
            total = sum(int(line.strip()) for line in lines)
            avg = total / len(lines)
            return avg
    except FileNotFoundError:
        print(f"Error: The file '{filename}' was not found.")
    except ValueError as ve:
        print(f"Error: {ve}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
```

Error handling is an example of repetitive code, such as try-except or try-catch structures, that GitHub Copilot handles exceptionally well.

© 2026 by Innovation In Software Corporation

SYNTAX ERRORS

Syntax errors are among the most common issues developers encounter, from missing colons in Python to mismatched brackets in JavaScript. Copilot helps resolve these by correcting the syntax based on language rules and best practices. For example, if you mistakenly write if (x > 10) in Python, Copilot may recognize the missing colon and suggest if x > 10:. Similarly, if you open an HTML tag and forget to close it, Copilot often generates the closing tag for you.

Another example is in Java: if you write System.out.println("Hello World") without a semicolon, Copilot can highlight the issue by suggesting the corrected line. These corrections are particularly useful for beginners or for developers working in a language they do not use frequently.

© 2026 by Innovation In Software Corporation

MUST BE GOOD!

This is especially important for syntax errors that do not cause compilation errors. In languages like Python, certain mistakes still produce valid code, so the program runs but behaves incorrectly.

These errors are much harder to detect because they don’t trigger obvious failures — they just produce wrong results.

© 2026 by Innovation In Software Corporation

EXAMPLE

```
def check_value(x):
    message = ""
    magic_number = 10
    if (y := magic_number):
        if x > y:
            message = f"x is greater than {y}"
        elif x < y:
            message = f"x is less than {y}"
        else:
            message = f"x is equal to {y}"
    else:
        message = "Condition failed"
    return message
```

```
print(check_value(5))
print(check_value(10))
print(check_value(20))
```

© 2026 by Innovation In Software Corporation

FIXED

```
def check_value(x):
    message = ""
    magic_number = 10

    if x > magic_number:
        message = f"x is greater than {magic_number}"
    elif x < magic_number:
        message = f"x is less than {magic_number}"
    else:
        message = f"x is equal to {magic_number}"

    return message
```

GitHub Copilot would quickly find and fixed the syntax / logical error.

© 2026 by Innovation In Software Corporation

RETROSPECTIVE

This module clearly shows how Copilot Chat helps you catch and fix everyday mistakes: mutating a list while iterating, undefined or incorrect calls, and common syntax slips; it then levels up practice with real error handling (file I/O guards, empty-file checks via try/except) and highlights “silent” logic errors

© 2026 by Innovation In Software Corporation

LAB 5 – PERFORMANCE

This module focuses on debugging application performance.

OVERVIEW

The module primarily focuses on using GitHub Copilot to identify and fix common programming mistakes, such as logic errors, missing error handling, and syntax problems. Performance optimization is not specifically covered in this module. However, GitHub Copilot can also be a valuable tool for analyzing and improving the performance of algorithms.

In this lab, you will use Copilot to examine an inefficient algorithm, understand its time complexity, and develop a more efficient implementation. This exercise demonstrates how Copilot can assist not only with debugging correctness issues but also with identifying opportunities to improve algorithm performance.

© 2026 by Innovation In Software Corporation

SAMPLE CODE

The find_duplicates function finds values that appear more than once in a list. It compares elements using nested loops and adds matches to a duplicates list if they have not already been recorded.

```
def find_duplicates(items):
    duplicates = []
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j] and items[i] not in duplicates:
                duplicates.append(items[i])
    return duplicates
```


© 2026 by Innovation In Software Corporation

SETUP

- Open VS Code
- Open the Lab5 folder.
- Open duplicates.py in a VS Code source window.

© 2026 by Innovation In Software Corporation

ANALYZE THE ALGORITHM

Review the provided code.

Tasks:

- For the find_duplicates function, determine what the algorithm is intended to do.
- Estimate the algorithm's time complexity.

Use a Copilot prompt that asks the assistant to:

- Analyze the algorithm (find_duplicates)
- Explain how it works
- Estimate its time complexity.

© 2026 by Innovation In Software Corporation

IMPROVE THE ALGORITHM

Refactor the algorithm to improve performance.

Tasks:

- Identify the inefficient portion of the code.
- Implement a more efficient approach.

Use a prompt that asks Copilot to:

- Propose a more efficient implementation
- Rewrite the algorithm with better performance.

© 2026 by Innovation In Software Corporation

EVALUATE FLEXIBILITY

Determine whether the improved algorithm works for different input types.

Tasks:

- Consider whether the algorithm depends on numeric operations.
- Determine whether it also works for strings.

Use a prompt that asks Copilot to:

- Evaluate whether the algorithm works for both numeric and string inputs
- Explain the reasoning.

© 2026 by Innovation In Software Corporation

GENERICS

In Python, generics provide limited practical benefit at runtime, because Python's dynamic typing already allows the function to work with many types (such as integers, strings, or tuples). The algorithm only relies on equality comparison, so it naturally works with any compatible type.

However, using generics can still provide some advantages:

- Improved readability – It clearly indicates that the function is designed to work with any type of list elements.
- Better static type checking – Tools like mypy or Pyright can verify correct usage.
- Better IDE support – Editors can provide more accurate type hints and warnings.

© 2026 by Innovation In Software Corporation

GENERALIZE THE IMPLEMENTATION

Improve the design so the function clearly supports multiple data types.

Tasks:

- Refactor the function using type hints or generics.
- Improve readability and maintainability.

Use a prompt that asks Copilot to:

- Refactor the function so it works with generic types
- Apply Python type hints.

© 2026 by Innovation In Software Corporation

EXAMPLE PROMPTS

Compare your prompts with these minimal solutions.

- @selection explain and analyze the time optimization of this algorithm
- rewrite this algorithm with better time optimization
- would this algorithm work for strings as well as numbers
- rewrite this function using Python generics
- compare the complexity of the original and optimized implementations

© 2026 by Innovation In Software Corporation

BONUS - BENCHMARKING

Create actual benchmarking metrics for the find_duplicates function.

Tasks:

- Select the function.
- Ask for benchmarking data

Use a prompt that asks Copilot to:

- Implement and display performance data for the selected function.
- Answer all follow-up questions.

© 2026 by Innovation In Software Corporation

Lab completed

CODE DOCUMENTATION

INTRODUCTION

This module is a hands-on guide to using GitHub Copilot Chat for fast, consistent documentation and clearer code understanding. You'll practice context-aware generation of docstrings and README documentation.

© 2026 by Innovation In Software Corporation

CONTEXT AWARE

Copilot now uses context-aware generation, meaning it considers the function signature, surrounding code, and even related files to suggest meaningful comments or documentation.

For example, when you begin typing a docstring in Python or a Javadoc comment in Java, Copilot can propose a full template that includes a description, parameter explanations, and return values.

© 2026 by Innovation In Software Corporation

HATE DOCUMENTATION

Many developers find documentation less engaging than writing code, especially under tight deadlines. Repetitive boilerplate and the effort required to explain technical details clearly can make documentation feel like overhead rather than value.

As a solution, many promote “self-documenting code.” Clear function names, descriptive variables, and clean structure reduce the need for excessive comments. However, readable code alone cannot replace higher-level documentation such as architecture decisions, business rules, and API contracts.

© 2026 by Innovation In Software Corporation

DOCUMENTATION IS CODE QUALITY

Documentation is part of software quality, not an afterthought. Clear documentation reduces cognitive load, accelerates onboarding, and preserves architectural intent over time.

Strong documentation improves collaboration and long-term maintainability, while weak documentation increases technical debt and slows debugging and feature delivery.

© 2026 by Innovation In Software Corporation

WHERE TO DOCUMENT

GitHub Copilot accelerates documentation at multiple levels of a project.

- Function and class docstrings
- Module summaries
- Project README files
- API reference documentation
- Refactoring explanations

Copilot acts as a drafting assistant that speeds up initial creation. Human review ensures accuracy and business alignment.

© 2026 by Innovation In Software Corporation

HIGH QUALITY COMMENTS

Copilot can generate structured docstrings directly from function definitions. When prompted clearly, it can include parameter descriptions, return values, assumptions, and validation rules.

To improve output quality:

- Specify documentation style (Google, NumPy, Javadoc)
- Request edge cases
- Ask for usage examples

Clear prompts produce clearer documentation.

© 2026 by Innovation In Software Corporation

REVERSE ENGINEERING

Copilot is especially useful for explaining unfamiliar or complex code. It can summarize algorithms, describe control flow, and highlight assumptions embedded in the logic.

This is particularly valuable when working with legacy systems or onboarding new team members. Generated explanations should be refined to reflect actual business intent.

© 2026 by Innovation In Software Corporation

README

Copilot can scaffold project-level documentation quickly, including setup instructions, configuration details, and usage examples.

- Providing context improves quality:
- Target audience
- Deployment environment
- Dependencies and constraints

Well-prompted outputs reduce the time spent drafting boilerplate documentation.

© 2026 by Innovation In Software Corporation

API DOCUMENTATION

Copilot can generate endpoint descriptions, example payloads, and structured API documentation. This accelerates internal and external documentation workflows.

However, developers should validate:

- Field accuracy
- Data types
- Authentication rules
- Error codes

© 2026 by Innovation In Software Corporation

ARCHITECTURAL SUMMARIES

For larger systems, Copilot can summarize repository structure and describe service interactions. It can help draft architecture overviews that are useful in design reviews or knowledge transfer sessions.

While helpful, it cannot fully infer business rationale behind architectural decisions. That context must be added manually.

© 2026 by Innovation In Software Corporation

DOCUMENTATION DURING REFACTORING

When code changes, documentation often becomes outdated. Copilot can regenerate docstrings, update parameter descriptions, and suggest improved naming conventions.

A strong workflow includes:

- Refactor code
- Regenerate documentation
- Compare for accuracy
- Manually validate edge cases

This reduces documentation drift.

© 2026 by Innovation In Software Corporation

RISKS AND LIMITATIONS

AI-generated documentation must always be reviewed. Copilot may hallucinate behaviors, misinterpret edge cases, or describe logic that does not exist.

It does not understand compliance requirements, performance constraints, or nuanced business rules unless explicitly prompted.

Documentation generated by AI is a draft — not authoritative.

© 2026 by Innovation In Software Corporation

ADVANCED PROMPTS

High-quality documentation depends on high-quality prompts. Developers should request structure, specify format, and ask for examples and edge cases.

For example:

Generate a NumPy-style docstring including validation rules and a usage example.

The more specific the instruction, the more precise and useful the documentation will be.

© 2026 by Innovation In Software Corporation

DOCUMENTATION - BEST PRACTICES

- Write the docstring first
- Constrain scope with context: use #selection (only selected code) or #file to feed the right amount of code; smaller context → tighter docs.
- Seed with style hints: a first line like “Use imperative mood, ≤80-char lines, include examples” steers output.

© 2026 by Innovation In Software Corporation

DOCUMENTATION - BEST PRACTICES - 2

docstring:

- #selection #utils.py Create a concise docstring: 1-line summary, args, returns, edge cases. Use imperative mood.

README snippet:

- #codebase Generate a minimal README section: install, run, test. Keep steps numbered and under 7 lines.

© 2026 by Innovation In Software Corporation

PROMPT - BEST PRACTICES - 3

Template reuse via prompt files:

- Keep a docstyle.md (tone, sections, examples). Reference it in prompts so Copilot stays consistent.

Ask for contrasts/examples:

- “Add a 1-line example and a common pitfall” produces practical docs without bloat.

© 2026 by Innovation In Software Corporation

RETROSPECTIVE

This module showed how to use GitHub Copilot for fast, consistent documentation: it demonstrated context-aware generation of docstrings that consider signatures, surrounding code, and related files.

© 2026 by Innovation In Software Corporation

Lab completed

ADVANCED

SETUP, REFERENCE, AND MORE

INTRODUCTION

In this module, we examine how GitHub Copilot manages your codebase. You'll learn the differences between local, workspace, and remote indexing, how context depth affects suggestion quality, and how premium requests are measured. We'll also review how @ references and # injections shape Copilot's understanding.

By the end, you'll understand how Copilot "sees" your environment, what improves suggestion accuracy, when to refresh indexing, and how to manage premium usage effectively.

© 2026 by Innovation In Software Corporation

TOKENS

Copilot chat, agent tasks, and code review use GitHub AI Credits, which are charged based on how much text (tokens) each request uses and which model handles it. Each paid plan includes a monthly credit allowance. Code completions and next edit suggestions are included and do not use credits.

© 2026 by Innovation In Software Corporation

GHOST TEXT SUGGESTIONS

Ghost text suggestions (and next edit suggestions) are included in your Copilot plan and do not use AI Credits. Accepting inline suggestions never adds a usage-based charge.

© 2026 by Innovation In Software Corporation

GITHUB AI CREDITS

GitHub AI Credits are how GitHub Copilot keeps track of how much you use its AI. Every time you chat with Copilot or ask the agent to do a task, it uses some credits. Bigger tasks and more powerful AI models use more credits, while quick questions use only a few. Code suggestions that appear as you type (ghost text) are always included and never use credits.

Each paid plan gives you a set amount of credits every month, which resets automatically. You can see how many you have used in the Copilot menu in VS Code. If you run out, you can buy more, or your company's administrator decides what happens next.

© 2026 by Innovation In Software Corporation

GITHUB AI CREDITS - 2

Tokens are the small pieces of text that AI models read and write, roughly a word or part of a word each. Every time you use Copilot, it counts the tokens in your request, including your prompt and any code it reads, plus the tokens in its answer. The more text that goes in and comes out, the more tokens the request uses.

GitHub AI Credits are what you pay for those tokens. Each AI model has its own price per token, so the same request costs more credits on a powerful model than on a simpler one. In short, tokens measure how much work Copilot does, and credits are the cost of that work deducted from your monthly allowance.

© 2026 by Innovation In Software Corporation

PREMIUM REQUESTS - 3

Each Copilot request is charged in GitHub AI Credits based on the tokens it uses, and the price per token depends on the AI model, as shown below for ten popular models (USD per 1 million tokens, where 1 AI credit = $0.01).

Provider	Model	Input	Cached input	Output
OpenAI	GPT-6 Luna	$0.10	$0.01	$0.50
Anthropic	Claude Haiku 5.5	$0.10	$0.01	$0.50
OpenAI	GPT-5 mini	$0.25	$0.025	$2.00
Google	Gemini 3.8 Flash*	$0.75	$0.075	$3.75
OpenAI	GPT-6.1 Sol	$2.00	$0.10	$10.00
Anthropic	Claude Sonnet 5.5	$2.00	$0.10	$10.00
Google	Gemini 4 Argon*	$2.00	$0.10	$10.00
Anthropic	Claude Opus 5.5	$4.00	$0.20	$20.00
OpenAI	GPT-6 Astra	$10.00	$1.00	$50.00
Anthropic	Claude Fable 5.1	$10.00	$0.25	$50.00

© 2026 by Innovation In Software Corporation

PREMIUM REQUESTS - EXAMPLE

Here is an example using a typical agent request, such as “Add a multiply function to sample.py.” Assume Copilot reads 20,000 input tokens (your prompt plus the code it looks at) and writes 2,000 output tokens.

How the cost is calculated (Claude Sonnet 5.5):

Input: 20,000 tokens × $2.00 per million = $0.04

Output: 2,000 tokens × $10.00 per million = $0.02

Total: $0.06 = 6 AI credits

© 2026 by Innovation In Software Corporation

CHAT MESSAGE

A chat message is a single turn in a Copilot Chat conversation. Each prompt you type is one message, and each reply from Copilot is another.

© 2026 by Innovation In Software Corporation

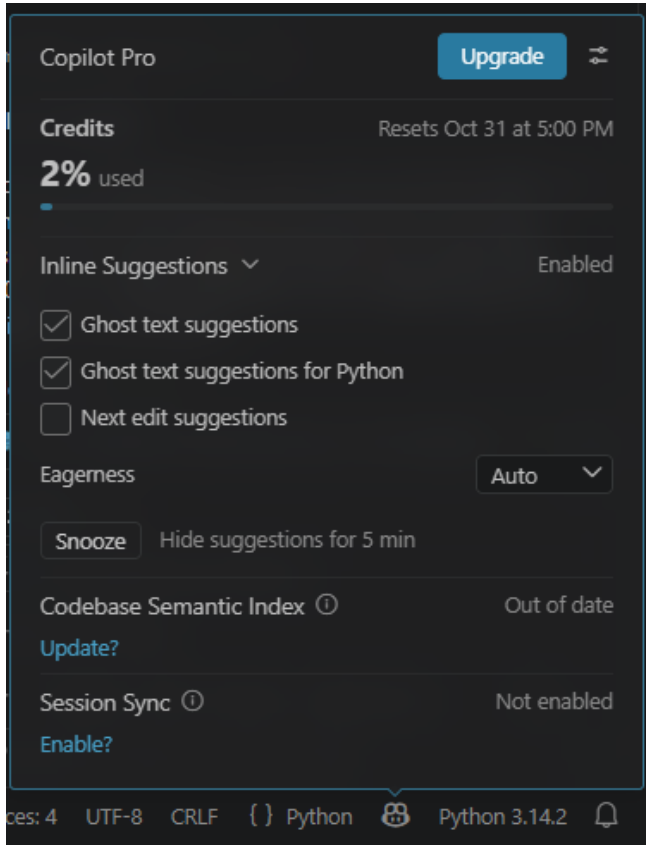
CHAT MESSAGE - 2

You type chat messages, but GitHub Copilot measures usage in tokens, which are billed as GitHub AI Credits. Every chat message you send uses some credits.

A short question on a small model costs very little, while a request that involves a lot of code, several agent steps, or an advanced model costs more. For that reason, a chat message is just the interface; tokens are what Copilot measures, and AI Credits are what you pay.

© 2026 by Innovation In Software Corporation

VIEW CREDIT INFORMATION

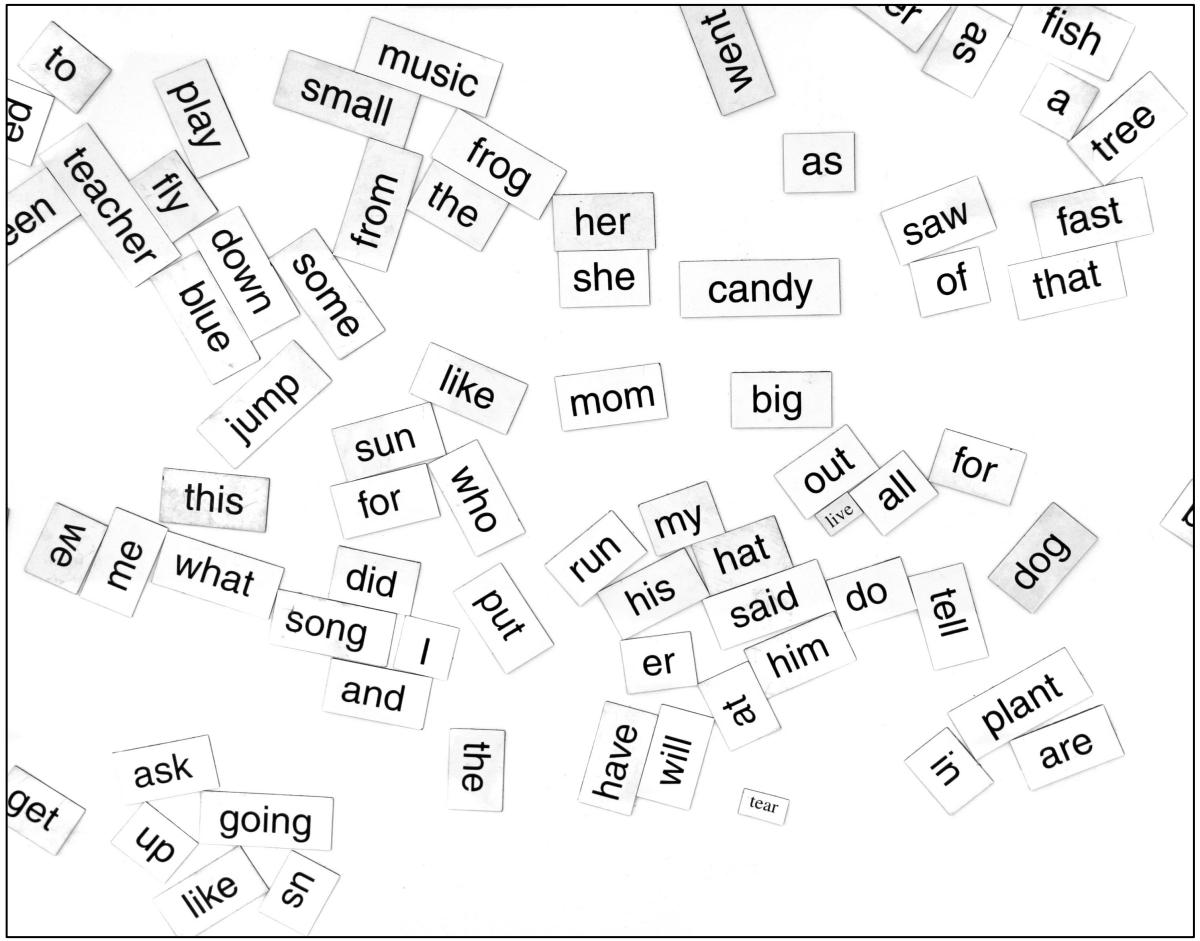


The Copilot status menu, opened by selecting the Copilot icon in the VS Code status bar, shows how many of your monthly GitHub AI Credits you have used. The Credits section displays your plan, the percentage used, a progress bar, and the date your allowance resets.

This menu only displays usage. To buy more credits or see a detailed breakdown, you will go to your GitHub account's billing settings on GitHub.com. To make credits last longer, choose a smaller model or Auto for simple tasks, and start a new chat for each new task.

© 2026 by Innovation In Software Corporation

TOKEN



Think of Tokens as “Word Pieces”

When you type something into GitHub Copilot, it doesn’t see full sentences the way humans do.

It breaks everything into small chunks called tokens.

A token can be:

- A word
- Part of a word
- A number
- A punctuation mark

© 2026 Innovation In Software Corporation

TOKEN - 2

Instead of reading:

> “Create a function that calculates interest.”

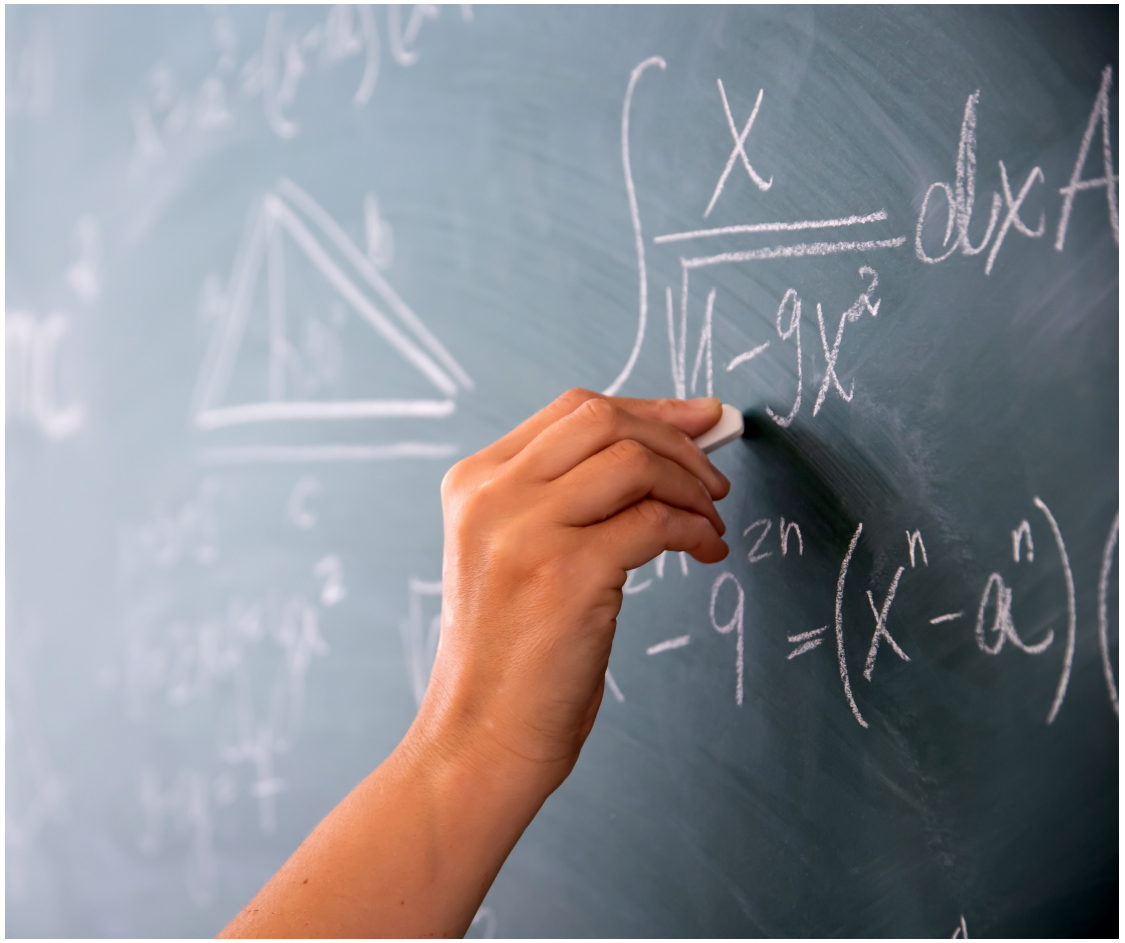
Copilot sees something more like:

> “Create” | “ a” | “ function” | “ that” | “ calculates” | “ interest” | “.”

Each of those pieces is a token.

© 2026 by Innovation In Software Corporation

WHY IMPORTANT

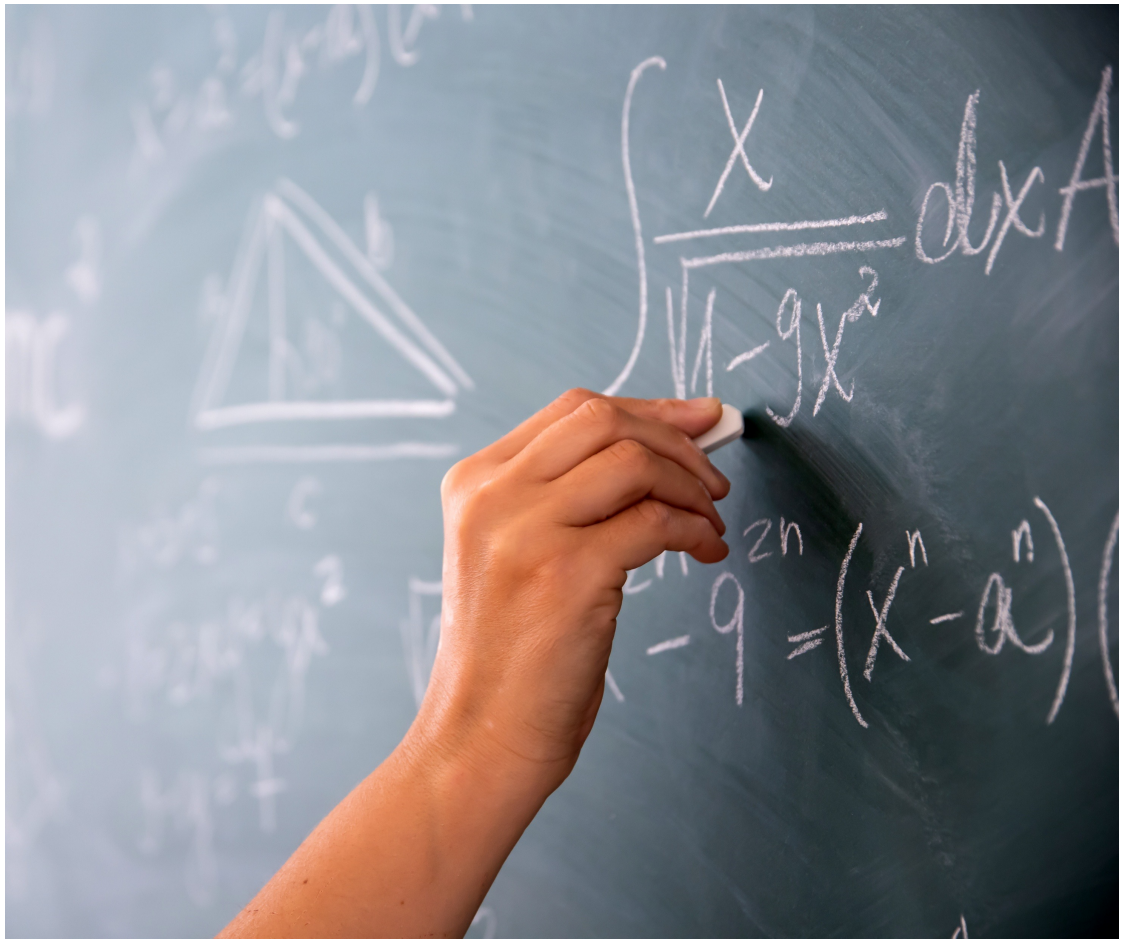


Copilot has a limit on how many tokens it can look at at one time. This is called the context window. Envision Copilot has a whiteboard. The whiteboard can only hold a certain number of word pieces (tokens). When it gets full, Copilot condenses older parts into a short summary to make room, so details from early in a long chat may be lost.

Starting a new chat for each new task gives Copilot a clean whiteboard.

© 2026 Innovation In Software Corporation

WHY IMPORTANT - 2



Otherwise, tokens determine:

- How much of your file Copilot can “see”
- How much of your prompt it remembers
- How long of a response it can generate

© 2026 Innovation In Software Corporation

BOTTOM LINE

Using too many tokens causes problems:

- Copilot forgets. When the context window fills up, older messages are condensed, and details can be lost.
- Quality drops. Long files make it harder for Copilot to focus on what matters.
- Costs rise. Every token uses your monthly AI Credits.

f Copilot seems confused, the important context was likely condensed or buried.

To avoid it: write short, focused prompts, point Copilot at specific code, and start a new chat for each task.

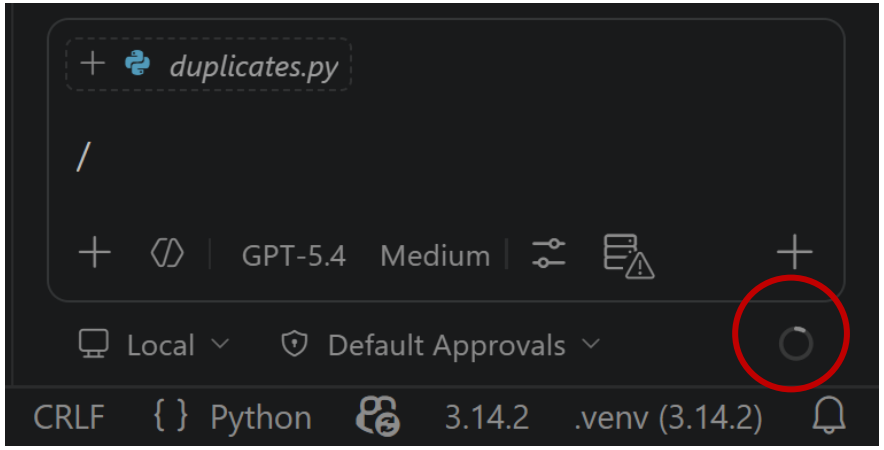
© 2026 by Innovation In Software Corporation

REDUCE TOKENS

- Keep prompts focused – Ask one clear, specific question instead of multiple tasks at once.
- Close unused files – Only keep relevant files open in your editor.
- Avoid large copy/paste blocks – Share only the part that matters.
- Start a new chat – Resetting clears accumulated conversation history.
- Trim long instruction or rule files – Keep custom instructions concise and relevant.

© 2026 by Innovation In Software Corporation

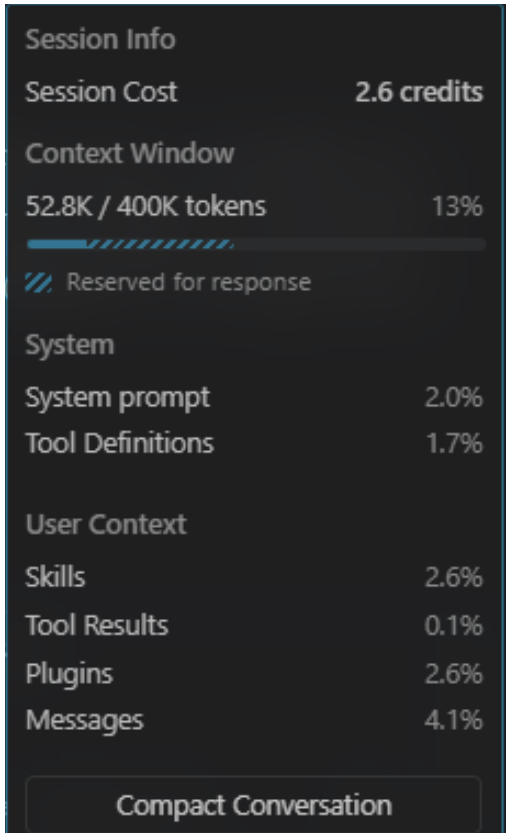
CONTEXT WINDOW



Open GitHub Copilot Chat in VS Code. The token usage indicator is at the bottom right of the status bar. Click this indicator to open the Context Window panel.

© 2026 Innovation In Software Corporation

CONTEXT WINDOW - 2



The panel displays:

- Total tokens used vs. available
- System instructions usage
- Tool definitions
- User messages

© 2026 Innovation In Software Corporation

INDEXING

For questions about your whole project, Copilot needs a quick way to find the right code, and it uses an index to do that.

- Codebase Semantic Index = a searchable map of your project that helps Copilot find relevant code.
- GitHub or Azure DevOps repos = the map is built and maintained for you.
- Other local folders = Copilot builds the map on your computer and keeps it updated.
- No index = Copilot falls back to regular text and file search.

© 2026 by Innovation In Software Corporation

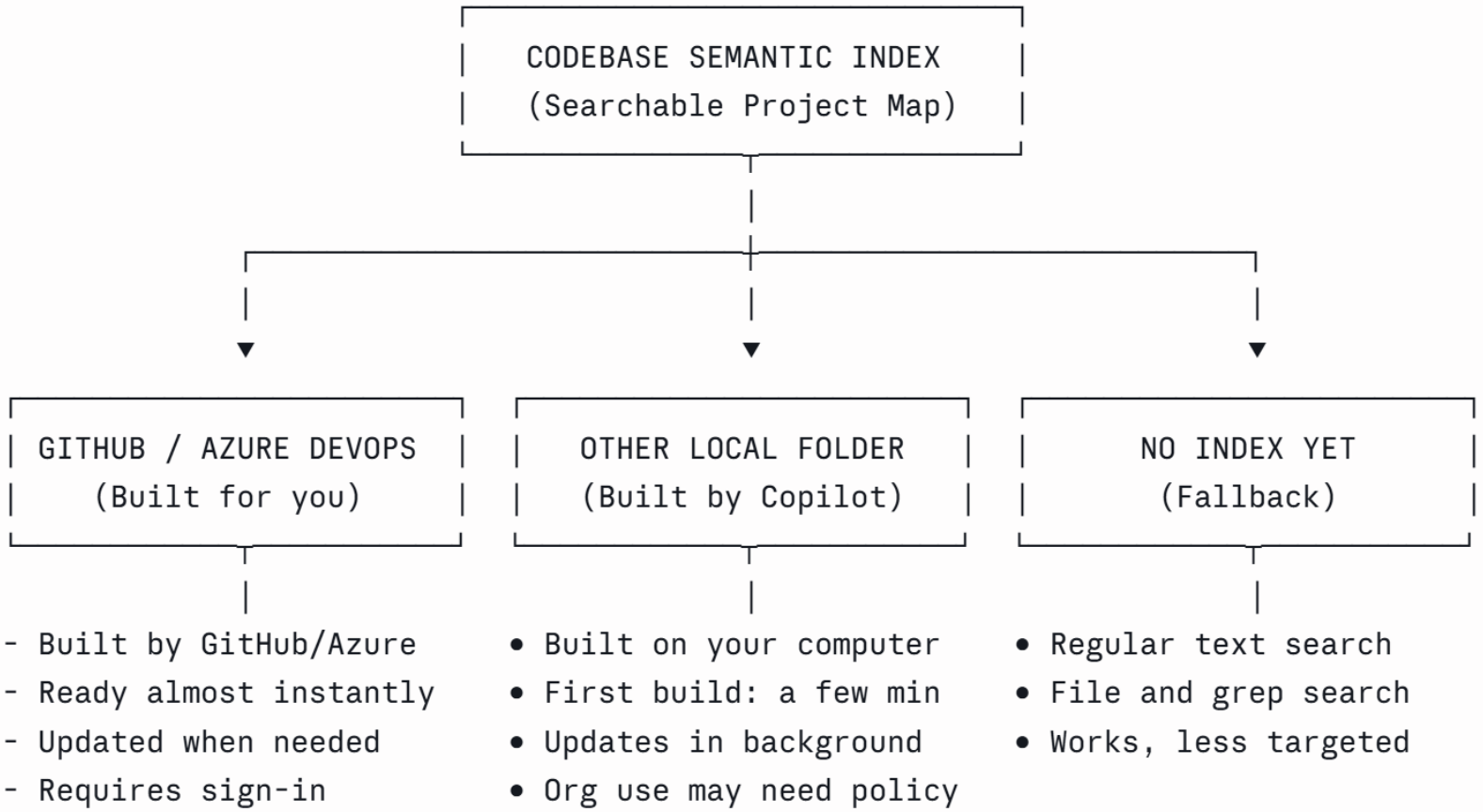
ORDER OF PREFERENCE

How Copilot builds the Codebase Semantic Index depends on where your project lives. If your project is a GitHub or Azure DevOps repository, the index is built and maintained for you, so it is often ready right away. For other folders, Copilot builds the index on your computer, which takes a few minutes the first time and then updates in the background. If no index is available yet, Copilot still finds code by using regular text and file search.

- GitHub or Azure DevOps repo: index built and maintained for you
- Other local folder: index built by Copilot on your computer
- No index yet: Copilot uses regular text and file search

© 2026 by Innovation In Software Corporation

ORDER OF PREFERENCE - 2



© 2026 by Innovation In Software Corporation

REMOVAL

Copilot's semantic index skips:

- Files ignored by .gitignore
- Files hidden by the VS Code files.exclude setting
- Binary files, such as images and PDFs
- Common low-value file types, such as .tmp and .out

The search.exclude setting removes files from text search but not from the index.

Note: .gitignore is bypassed when you have an ignored file open or text selected in it.

© 2026 by Innovation In Software Corporation

REBUILD

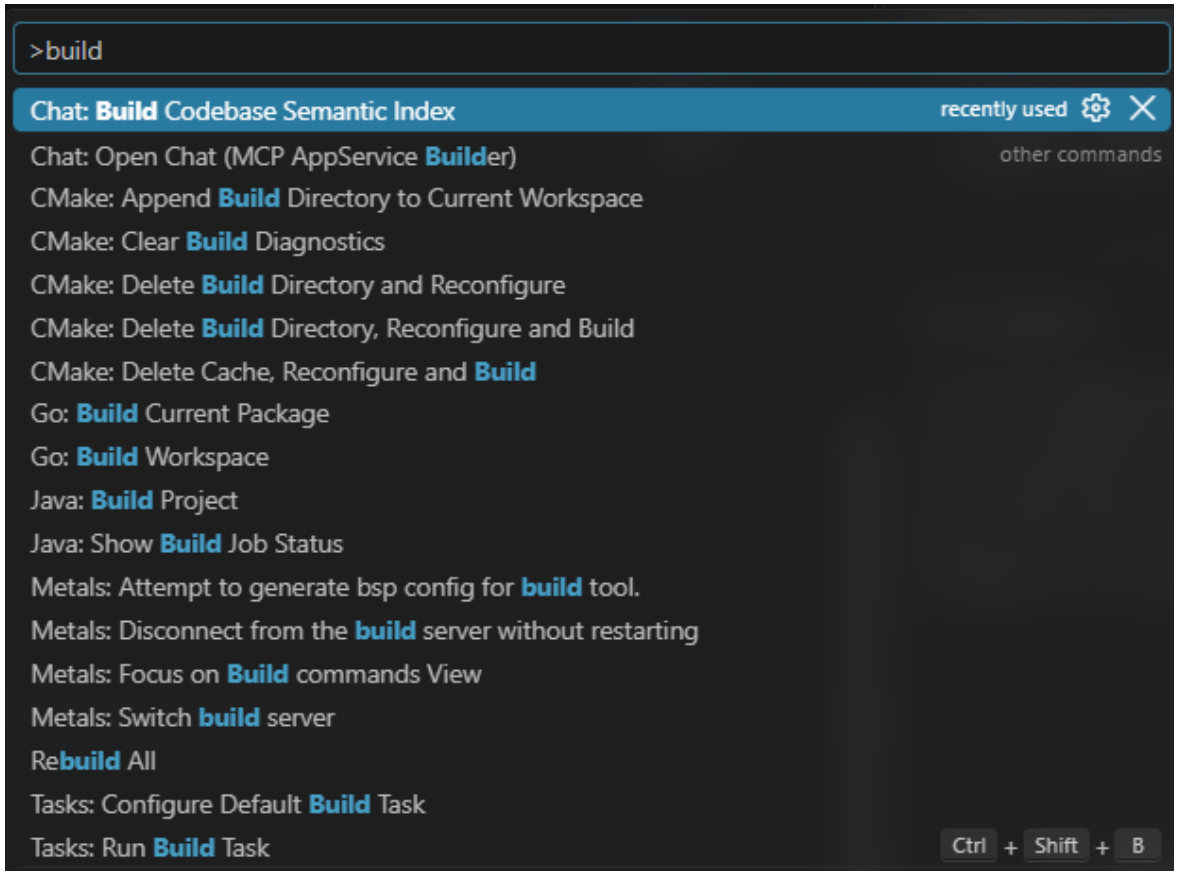
The Codebase Semantic Index normally updates on its own. Rebuild it manually when:

- The Copilot status menu shows Out of date or an indexing error
- Copilot’s answers seem outdated or reference code that no longer exists
- You made large changes, such as a major refactor, adding or removing many files, renaming key folders, or switching to a very different branch

To rebuild, select Update? in the Copilot status menu, or run Build Codebase semantic index from the Command Palette.

© 2026 by Innovation In Software Corporation

REBUILD - 2



How to select Build Codebase semantic index from the Command Palette

© 2026 Innovation In Software Corporation

REFERENCES (@)

GitHub Copilot Chat is context-aware, meaning it can pull from different sources depending on your task. You can guide it to the right context to improve accuracy when needed:

- @terminal: terminal commands and errors
- @vscode: VS Code features and settings
- @github: GitHub issues, pull requests, and repositories

Note: In Agent mode, Copilot gathers much of this context on its own, so @ and # are now ways to steer it rather than requirements.

© 2026 by Innovation In Software Corporation

REFERENCES (@) - 2

Here is a detailed explanation:

Copilot @ Context References

Feature	`@terminal`	`@vscode`	`@github`
Primary Purpose	Help with terminal commands and shell tasks	Answer questions about VS Code features and settings	Work with GitHub data
Context Source	The VS Code terminal and shell	VS Code commands, settings, and documentation	Issues, pull requests, and repositories
Best For	Finding the right command, understanding terminal output	Learning how to do something in VS Code	Repo insights and collaboration
Typical Prompt	`@terminal how do I list files by size?`	`@vscode how do I turn on word wrap?`	`@github list open pull requests`

© 2026 by Innovation In Software Corporation

RETROSPECTIVE

In this module, we examined how GitHub Copilot uses layered context — local, workspace, and remote indexes — to generate accurate, project-aware suggestions. We clarified the difference between chat messages and premium requests, and how usage is measured.

We also explored how @ references direct Copilot to context sources.

© 2026 by Innovation In Software Corporation

LAB 7 – CONTEXT

This lab uses #codebase and @terminal within Python

OBJECTIVE

Use GitHub Copilot Chat to:

- Understand a Python project with #codebase
- Run the application
- Diagnose and fix errors using @terminal

© 2026 by Innovation In Software Corporation

SETUP

Find and open Lab7 as a simple Python project in Visual Studio Code. It contains these files:

- app.py
- utils.py
- requirements.txt

© 2026 by Innovation In Software Corporation

#CODEBASE TO UNDERSTAND PROJECT

In Copilot Chat, type:

#codebase explain the purpose of this project

Then:

#codebase identify the main files and how they interact

Then:

#codebase how do I run this Python application?

© 2026 by Innovation In Software Corporation

RESULTS

So far, Copilot should:

- Describe what app.py does
- Explain supporting files like utils.py
- Suggest a command such as:

    python app.py

© 2026 by Innovation In Software Corporation

ERROR

In the terminal, run an incorrect command:

```
python missing_file.py
```

Use @terminal to Diagnose. In Copilot Chat:

```
@terminal why did this fail?
```

Then:

```
@terminal suggest the correct command
```

© 2026 by Innovation In Software Corporation

RESULTS

Copilot should:

Identify the error: *file not found*

Suggest running:

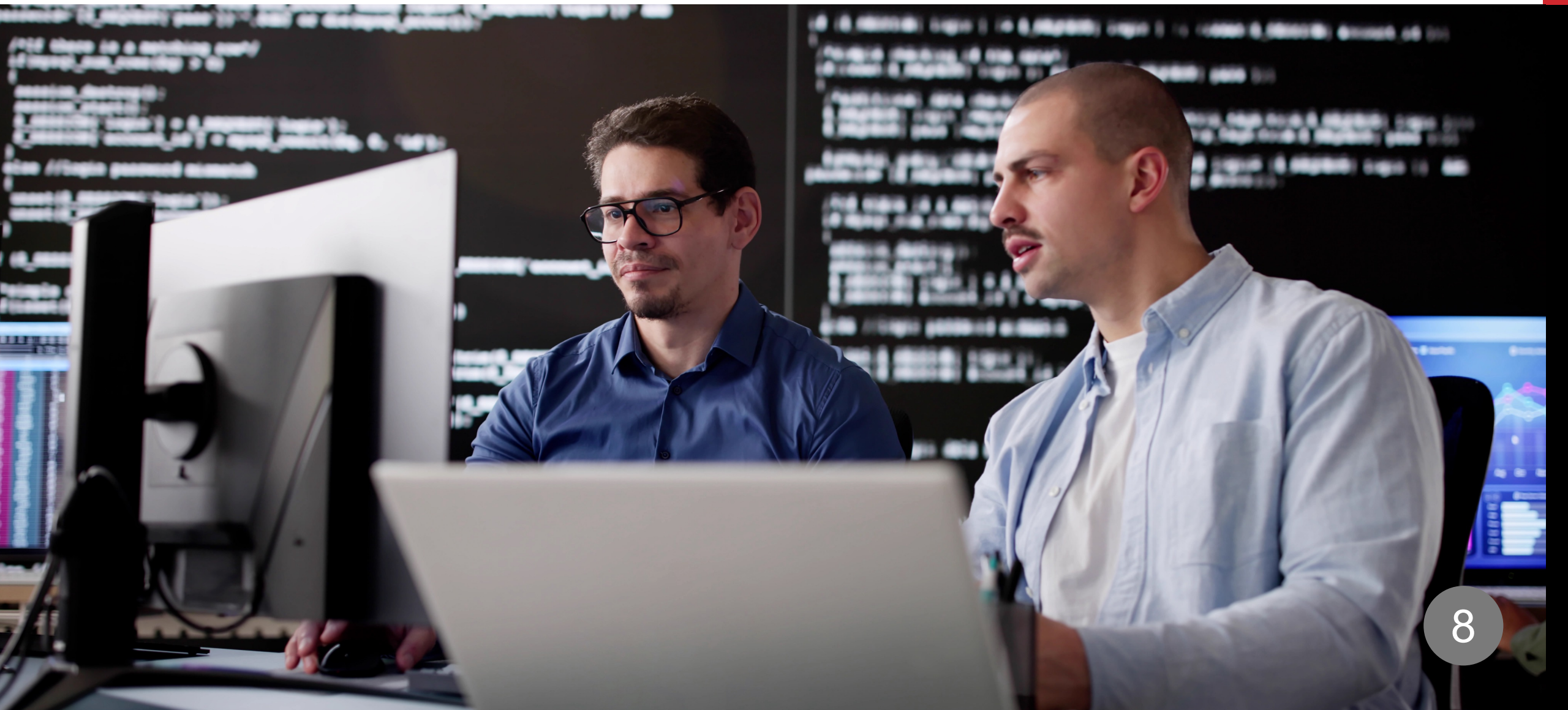
python app.py

© 2026 by Innovation In Software Corporation

Lab completed

CUSTOMIZATION

PROMPTS, INSTRUCTIONS, AND MCPS

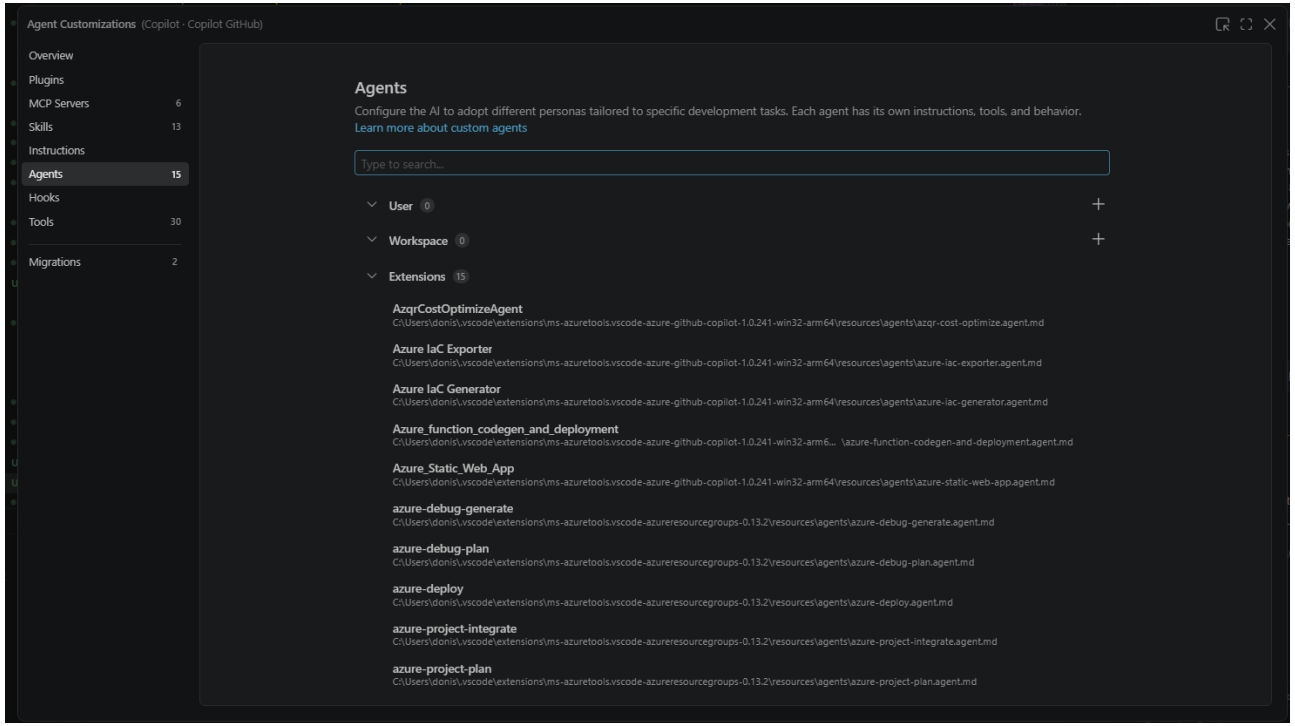


INTRODUCTION

GitHub Copilot is more than inline code completion — it is a configurable AI assistant that lets you customize your development environment, enforce standards, and extend beyond the workspace. This module introduces the mechanisms that control Copilot's behavior, including Skills, Instruction Files, Hooks, and MCP servers.

© 2026 by Innovation In Software Corporation

OPEN CUSTOMIZATION MENU



The Open Customization menu is at the top-right of the GitHub Copilot Chat side panel in VS Code. The menu items — Custom Agents, Prompt Files, Skills, Instructions & Rules, Hooks, MCP Servers, and Tool Sets—each offer different ways to customize Copilot Chat.

© 2026 by Innovation In Software Corporation

REUSABLE PROMPTS

A Prompt File is a Markdown file (.prompt.md) that stores a reusable prompt template.

Instead of retyping the same instructions in Copilot Chat, you put them into a *command*.prompt.md file and run it whenever needed. *Command* is the name of the task used to reference the prompt later.

Metadata (front-matter at the top) is included to indicate the type of prompt file.

Think of Prompt files as macros for use with GitHub Copilot, but written in plain text/markdown.

© 2026 by Innovation In Software Corporation

PROMPT FILES

A Prompt File is a Markdown file (.prompt.md) that stores a reusable prompt template.

Instead of retyping the same instructions in Copilot Chat, you put them into a *command*.prompt.md file and run it whenever needed. *Command* is the name of the task used to reference the prompt later.

Metadata (front-matter at the top) is included to indicate the type of prompt file.

Think of Prompt files as macros for use with GitHub Copilot, but written in plain text/markdown.

© 2026 by Innovation In Software Corporation

PROMPT FRONT-MATTER

Front-matter metadata is YAML data between --- lines. For GitHub Copilot, this section indicates how the prompt should be interpreted and which mode Copilot should use when responding.

```
---
agent: agent
---
```


© 2026 by Innovation In Software Corporation

PROMPT FRONT-MATTER - 2

Agent

The agent setting controls how Copilot behaves. When set to agent: agent, Copilot operates in a “do the work” mode rather than simple question-and-answer. This enables planning, multi-step reasoning, and more autonomous code changes instead of just explanations.

applyTo

The applyTo setting defines where the instructions apply. It scopes the prompt to specific files, languages, or folders (for example, only Python files or the entire workspace). This makes the prompt activate consistently instead of being applied randomly or ignored.

© 2026 by Innovation In Software Corporation

PROMPT FRONT-MATTER - 3

Priority

Priority determines which instructions win when multiple prompt or instruction files exist. Higher-priority rules override lower-priority ones and can supersede Copilot's defaults. Without priority, custom instructions may be silently overridden by other rules.

Example:

```
---
agent: agent
applyTo: "**/*.py"
priority: 100
---
```


© 2026 by Innovation In Software Corporation

PROMPT BODY

The prompt body follows after the front matter and describes the role, if any, and how the is prompt applied.

```
---
name: factorial
description: Describe when to use this prompt
---
create the factorial function in Python.
The function should be non-recursive and should handle
edge cases such as negative numbers and zero.
```


© 2026 by Innovation In Software Corporation

HOW TO USE

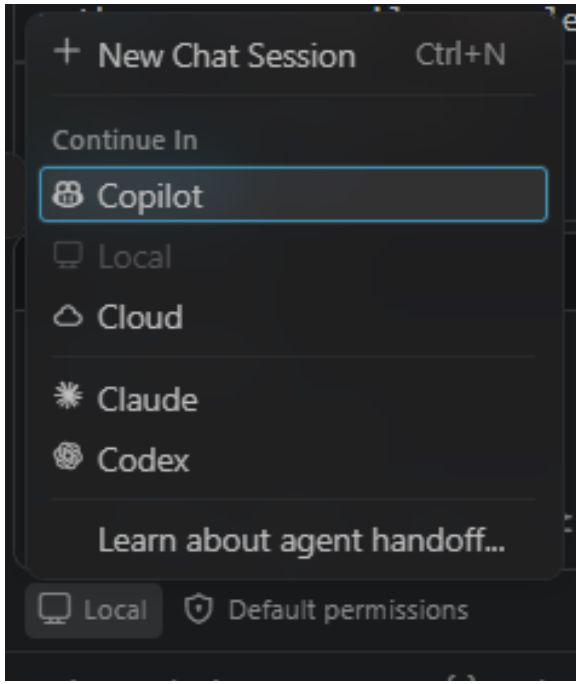
Execute directly in a chat prompt with a slash command:

/promptcommand

Note: Prompts are no longer represented in the Open Customizations window, but remain valid. Must be run in a local session.

© 2026 by Innovation In Software Corporation

DELEGATE SESSION



New Chat Session starts fresh, while *Continue In* hands your *current* conversation to a different kind of agent (VS Code calls this a handoff). The conversation history comes along, but the tools, permissions, and models may change.

Appears in the Copilot Chat window probably initially as a Copilot icon. At the bottom-left.

© 2026 Innovation In Software Corporation

DELEGATE SESSION - 2

- Copilot: the everyday choice. It runs on your machine, and the session keeps running even if you close the editor, so you can pick it up later. This is what you've been using.
- Local: runs inside VS Code's built-in agent, in your current window. It's the most hands-on option, but closing the window stops the work. It's also the only type that supports prompt files.
- Cloud: runs on GitHub's servers against your GitHub repository and hands back a pull request when it's done. It can't see your open files or local editor context, so give it a clear, self-contained task. You can also send a task there from a Copilot session by typing /delegate.
- Claude and Codex: the same idea, but powered by Anthropic's and OpenAI's own agent tools.

© 2026 by Innovation In Software Corporation

INSTRUCTIONS & RULES

To start a new Instruction or Rule in VS Code, open GitHub Copilot Chat, click the ⚙ Gear menu, and select Instructions & Rules. From there, choose New Instruction to create a global instructions.md file at the workspace root, or choose New Rule to create a {name}.instructions.md file inside .github/instructions/ with optional path-based metadata. The editor will generate the template for you, and once saved, Copilot will automatically apply it.

© 2026 Innovation In Software Corporation

INSTRUCTION VERSUS RULES

An Instruction uses a file named exactly instructions.md, typically placed at the workspace root. This file provides persistent, global guidance for Copilot across the entire project. It is always loaded and defines overall behavioral expectations, such as “Use Java 21” or “Avoid recursion.” It contains no metadata or path targeting — it simply sets project-wide preferences.

A Rule uses the naming convention {name}.instructions.md and is stored in .github/instructions/. Unlike instructions.md, Rule files include metadata (such as paths) to apply conditionally to specific files or directories. Multiple Rule files can exist and load selectively. In short: instructions.md defines an Instruction (global guidance), while {name}.instructions.md defines scoped Rules.

© 2026 by Innovation In Software Corporation

RULES – *NAME*.INSTRUCTION.MD

```
---

applyTo: "**/*.java"

---


Follow these security rules when generating or modifying
code:


- Never hardcode secrets or credentials.

- Validate and sanitize all external input.

- Use parameterized queries for database access.

- Prefer modern cryptographic algorithms.

- Flag potential OWASP Top 10 risks when detected.
```


© 2026 by Innovation In Software Corporation

RULES – *NAME*.INSTRUCTION.MD - 2

```
Follow these security rules when generating or modifying code:


- Never hardcode secrets or credentials.
- Validate and sanitize all external input.
- Use parameterized queries for database access.
- Prefer modern cryptographic algorithms.
- Flag potential OWASP Top 10 risks when detected.
```


© 2026 by Innovation In Software Corporation

GHOST TEXT SUGGESTIONS

Ghost text ignores custom instructions. Plainly stated, custom instructions are not taken into account for inline suggestions as you type in the editor.

© 2026 by Innovation In Software Corporation

INSTRUCTION FILES

The supported instruction files are:

- .github/copilot-instructions.md: project-wide rules that apply to every Copilot chat and agent request in the project
- *.instructions.md files in .github/instructions: targeted rules that apply only to specific files, folders, or tasks
- AGENTS.md: shared project guidance that Copilot and other AI agents, such as OpenAI Codex, can all read

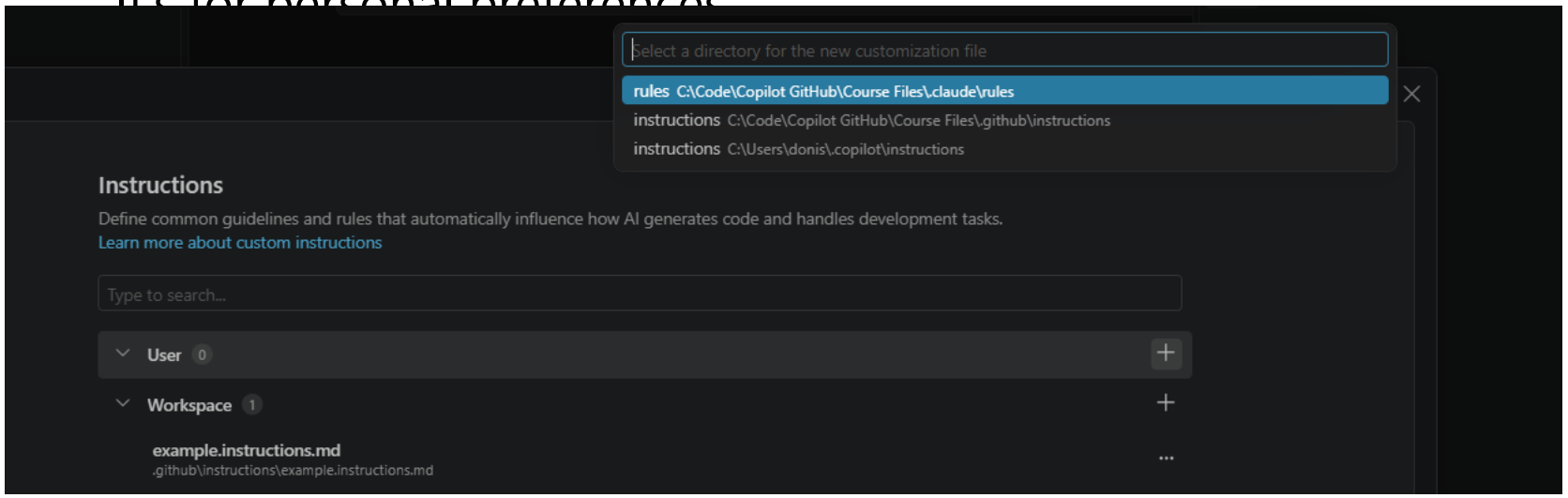
© 2026 by Innovation In Software Corporation

INSTRUCTION TEMPLATE

Selecting + starts a wizard that creates a new, ready-to-edit instruction file in the right folder.

Step 1: Choose User or Workspace. Select the + next to the section you want:

- Workspace: the file lives in your project and is shared with anyone who uses the repository.
- User: the file lives in your home folder and applies to all your projects. It's for personal preferences.



© 2026 by Innovation In Software Corporation

INSTRUCTION TEMPLATE - 2

Step 2: Pick a folder. The list you see (in your screenshot) matches each folder to the agent that reads it:

- instructions …\Course Files\.github\instructions: Copilot, workspace. Pick this one for your course.
- rules …\Course Files\.claude\rules: Claude agent, workspace.
- instructions C:\Users\donis\.copilot\instructions: Copilot, user (all your projects).

Step 3: Enter a file name, for example python-style. VS Code adds the .instructions.md extension and opens the new file.

Step 4: Fill in the template. The file starts with optional frontmatter, followed by your rules in plain Markdown.

© 2026 by Innovation In Software Corporation

SKILL

A Skill in GitHub Copilot for VS Code is a reusable workflow you invoke to perform a specific task. Unlike instruction files that automatically guide behavior, a Skill runs only when selected and follows defined steps, edits, or commands.

Skills can coordinate code changes, generate documentation, run approved terminal commands, or use connected tools. They standardize repeatable tasks.

© 2026 Innovation In Software Corporation

SKILLS FEATURES

Features:

- A reusable instruction or workflow
- Invoked intentionally (not automatically like instructions files)
- Designed to perform a specific task
- Often parameterized (can accept inputs)

It turns Copilot from a passive assistant into a task executor for defined workflows.

© 2026 by Innovation In Software Corporation

CAPABILITY

They can orchestrate any capability that Copilot Chat itself can access — which includes code editing, terminal commands, workspace context, tools, and (in Agent mode) external integrations.

- Code & Workspace Tasks
- File System Operations
- Terminal / Command Execution
- Tool-Based Tasks
- Documentation
- Multi-Step Orchestrated Workflows

© 2026 by Innovation In Software Corporation

SKILLS - SETUP

Skills created through the Skills menu (New Skill) are stored as Markdown files, typically named skill.md.

When you create a Skill using the UI, VS Code generates a folder for that Skill under:

.github/skills/<skill-name>/

The skill.md file contains structured frontmatter (YAML metadata at the top) followed by the workflow instructions written in Markdown.

© 2026 by Innovation In Software Corporation

SKILLS – EXAMPLE

---

name: add-prefix

description: Adds a specified prefix to all files in a chosen directory.

---

1. Ask the user for:

   - Target directory

   - Prefix to apply

2. List the files in the directory.

3. Show a preview of renamed files.

4. Ask for confirmation.

5. Rename the files.

6. Report completion.

Use filesystem tools when available.

© 2026 by Innovation In Software Corporation

HOOKS

In GitHub Copilot Chat within Visual Studio Code, hooks run your own commands or scripts at specific points in an agent session, such as when a session starts, before or after the agent uses a tool (like editing a file or running a terminal command), or when the agent finishes.

Hooks can help enforce quality and control. You can block risky commands before they run, format or test code after the agent edits it, or log every action for auditing. They make Copilot a controlled, policy-aware part of your workflow. (Hooks are currently in Preview.)

© 2026 Innovation In Software Corporation

HOOKS - FEATURES

A hook allows you to:

- Run a script before the agent edits files or runs a command
- Run a command after the agent makes changes
- Block risky actions or require approval automatically
- Integrate with local tools, such as linters, formatters, tests, and security scans
- Log agent activity for auditing

Instead of manually running commands after every AI change, hooks let you automate that process.

© 2026 by Innovation In Software Corporation

HOOKS – ACTIONS

Here are the primary actions:

- **Before file edits are applied**
  Run a script or command before the agent writes changes to your files.
- **After file edits are applied**
  Trigger validation steps, such as tests, linters, and formatters, once the changes are made.
- **Before executing a terminal command**
  Check, require approval for, or block a command the agent is about to run.
- **After executing a terminal command**
  Analyze the output, trigger follow-up checks, or log the results.

© 2026 by Innovation In Software Corporation

HOOKS – ACTIONS - 2

- **Before tool execution**
  Intercept calls to tools, including built-in tools, extension tools, and MCP server tools.
- **After tool execution**
  Process tool output, enforce policies, react to failures, or trigger additional automation.
- **When a session starts**
  Prepare the environment, load project context, or initialize dependencies.
- **When a subagent starts or stops**
  Track, check, or log work the agent hands off to a helper agent.
- **When the agent finishes**
  Run cleanup tasks, test suites, or security scans.

© 2026 by Innovation In Software Corporation

HOOKS - SETUP

Create hooks in the Open Customization window, similar to Instructions. User hooks are placed in this directory.:

.github/hooks/

In the current version, hook definitions are stored as JSON files, typically using a descriptive naming convention such as:

pre-file-edit.json

post-tool-use.json

pre-terminal-command.json

Each JSON file defines the trigger event and the command or behavior that runs when that event fires.

© 2026 by Innovation In Software Corporation

HOOKS – FILE

```
{
    "hooks": {
        "PostToolUse": [
            {
                "type": "command",
                 "command": "powershell -NoProfile -Command
\"$inputJson = [Console]::In.ReadToEnd(); $ts = (Get-
Date).ToString('o'); Add-Content -Path .copilot-hook.log -Value ('['
+ $ts + '] PostToolUse fired');\""
            }
        ]
    }
}
```


© 2026 by Innovation In Software Corporation

HOOKS KEYWORDS

The actions are:

- PreToolUse
- PostToolUse
- PreFileEdit
- PostFileEdit
- PreTerminalCommand
- PostTerminalCommand
- SubagentStart
- SubagentStop
- OnError

© 2026 by Innovation In Software Corporation

CUSTOM AGENTS

Custom agents in GitHub Copilot for Visual Studio Code let you define a persistent role and behavior that Copilot applies automatically to your prompts. Instead of rewriting detailed instructions each time, you create an agent once (e.g., “security reviewer” or “test generator”), and it consistently controls tone, focus, and output style whenever selected.

© 2026 Innovation In Software Corporation

CUSTOM AGENTS - STEPS

1. Open your project in VS Code.

2. Create this folder in the project root:

   .github/agents

3. Create a file such as:

   .github/agents/{python-reviewer}.agent.md

© 2026 by Innovation In Software Corporation

CUSTOM AGENTS – STEPS - 2

4. Add agent instructions:

---

description: Demo agent that proves it is being used

You are a strict reviewer.

Rules:

- ALWAYS start your response with: "AGENT ACTIVE ✅"
- Keep responses under 2 sentences
- Be slightly critical in tone

© 2026 by Innovation In Software Corporation

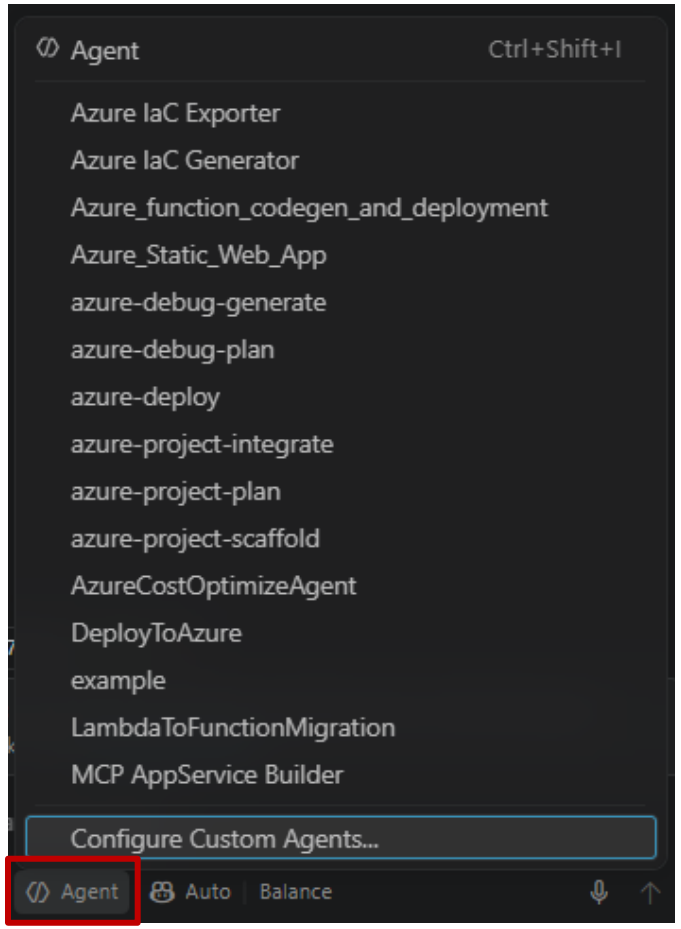
CUSTOM AGENTS – STEPS - 3

5. Save the file.
6. Open Copilot Chat.
7. Open the agent/mode dropdown in the chat window.
8. Select your custom agent, for example:

   {Python Reviewer}
9. Test it with a prompt:
10. Review app.py for major Python issues.

© 2026 by Innovation In Software Corporation

CUSTOM AGENTS



Custom agents in GitHub Copilot for Visual Studio Code let you define a persistent role and behavior that Copilot applies automatically to your prompts. Instead of rewriting detailed instructions each time, you create an agent once (e.g., “security reviewer” or “test generator”), and it consistently controls tone, focus, and output style whenever selected.

© 2026 Innovation In Software Corporation

MCP SERVER

MCP (Model Context Protocol) servers use a standard protocol that allows AI assistants to securely communicate with external tools, data sources, and services. Instead of limiting the model to only the local editor or repository, an MCP server acts as an intermediary layer that enables structured requests and responses between the AI and additional capabilities.

These additional capabilities expand what the LLM can reason about, not just in terms of files in a project, but in terms of live systems, enterprise data, and external services. As a result, responses become more relevant, actionable, and integrated with operational environments rather than limited to local code or pre-trained knowledge alone.

© 2026 by Innovation In Software Corporation

INNOVATION

Model Context Protocol (MCP) drives innovation by providing a standard interface between AI models and external tools. Without it, developers must build custom integrations for data, tools, and model interactions—spending time on technical plumbing instead of creating new capabilities.

Rather than hard-wiring integrations into the model, MCP establishes a clear contract between the LLM (or agent) and MCP servers.

© 2026 Innovation In Software Corporation

MCP SERVER WHERE?

GitHub ships some built-in ones (filesystem, terminal, GitHub Issues/PRs). Enter @MCP when searching for a VS Code extension to uncover various available MCP servers.

You can also write a MCP server in any language that publishes an interface that adheres to the protocol.

© 2026 by Innovation In Software Corporation

MCP – THE PROTOCOL

The Model Context Protocol (MCP) is like USB-C for AI apps. Just as any USB-C device can plug into any USB-C port, any AI app that supports MCP can connect to any MCP server.

In MCP terms, the AI app, such as GitHub Copilot in VS Code or Claude Desktop, is the client. The outside programs it connects to are servers. Each server can offer three things:

- Tools: actions the AI can perform, such as creating a GitHub issue or querying a database
- Resources: data the AI can read, such as files, documents, or records
- Prompts: ready-made templates that help you ask the AI for common tasks

© 2026 by Innovation In Software Corporation

MCP – THE PROTOCOL - 2

Why this matters: without a standard, every AI app would need its own custom integration for every tool. With MCP, a server is built once and works with any MCP client: one standard instead of many adapters.

© 2026 by Innovation In Software Corporation

FASTMCP

FastMCP is a developer-friendly Python library that makes it easy to stand up an MCP server without having to write the protocol interface yourself.

Instead of manually handling JSON-RPC, message passing, and lifecycle management, you just define your tools, resources, and prompts as ordinary Python functions with simple decorators. FastMCP then exposes them through MCP automatically, so any MCP client, such as GitHub Copilot in VS Code, can discover and use them.

© 2026 by Innovation In Software Corporation

FASTMCP - 2

Features of FastMCP:

- Quick server creation: register functions as MCP tools with decorators.
- Auto-wiring: handles protocol messages, introspection, and dependencies automatically.
- Python-native: you define tools in Python and FastMCP takes care of wrapping them in MCP-compliant responses.
- Integration with clients: servers built with FastMCP can be found with GitHub Copilot Chat, ChatGPT (with MCP support), or other MCP-capable assistants.

© 2026 by Innovation In Software Corporation

FASTMCP - INSTALL

Standard install:

- pip install fastmcp

Verify the installation

- fastmcp --help

© 2026 by Innovation In Software Corporation

SAMPLE CODE

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("hello-mcp")

@mcp.tool()
def ping() -> str:
    """Returns 'pong'."""
    return "pong"

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Creating a MCP Server is outside the scope of this class. Importantly, MCP is language-agnostic—it's just JSON-RPC over transports like stdio or HTTPS—so you can write a server in any language that can read/write JSON on those transports.

© 2026 by Innovation In Software Corporation

SAMPLE CODE – CONFIGURATION FILE – MCP.JSON

```
{
  "servers": {
    "helloMcp2": {
      "type": "stdio",
      "command": "python",
      "args": ["${workspaceFolder}/server.py"]
    }
  }
}
```

Tells VS Code/Copilot which MCP servers to use and how to start them: choose the transport (stdio or http), set the launch command and args (or a remote url), and optionally define inputs (e.g., API keys).

Where to put it:

./.vscode/mcp.json

© 2026 by Innovation In Software Corporation

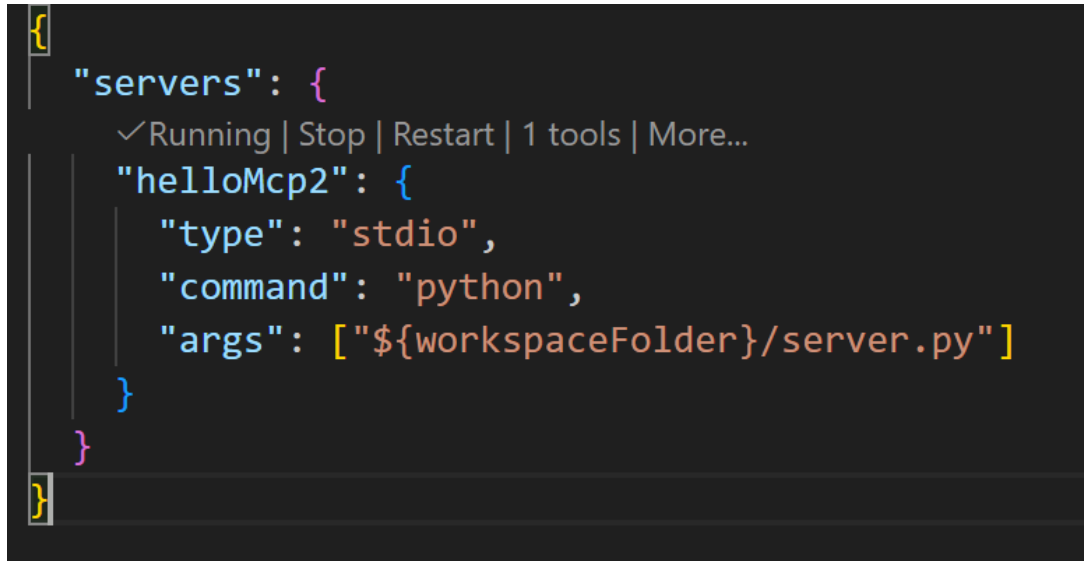
PROJECT STRUCTURE

```
your-project/
├─ .vscode/
│   └─ mcp.json
├─ server.py
```

The mcp.json file lives in the .vscode directory as VS Code–specific configuration, while server.py at the project root contains the MCP server logic; when the workspace opens, VS Code automatically discovers .vscode/mcp.json and can start the server when requested.

© 2026 by Innovation In Software Corporation

STARTING SERVER



Running | Stop | Restart | 1 tools | More…

This toolbar is an indicator that VS Code has successfully started the MCP server.

- Running: Server is active
- Stop: Terminates the MCP server process
- Restart: Restarts server.py
- 1 tools: The server has registered one MCP tool
- More…: Logs, diagnostics, advanced actions

© 2026 by Innovation In Software Corporation

EXECUTE AN MCP SERVER

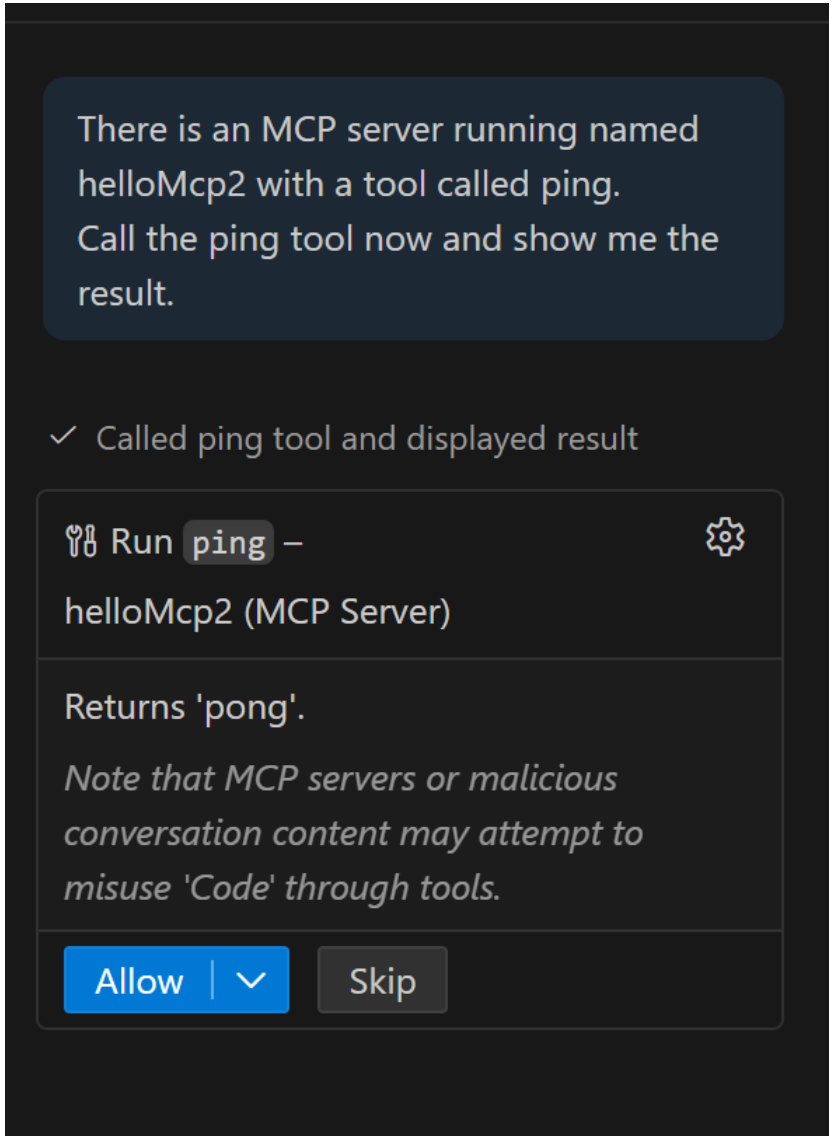
You don’t call MCP tools like commands or slash actions. They are functions that only Copilot can run. Your job is to describe what you want in NLP. Copilot then decides whether a tool aligns with the request, prepares the inputs, runs it, and shows you the result.

Think of MCP as giving Copilot extra capabilities, not new buttons. If Copilot can answer without using the tool, it may do so, which is why testing sometimes requires explicit wording like “you must call the *MCP tool*.” The key shift is simple: you state the intent; Copilot chooses and runs the tool.

You could call the ping tool within the helloMcp2 MCP server with this prompt – “Call the ping tool now and show me the result.”

© 2026 by Innovation In Software Corporation

EXECUTE AN MCP SERVER



© 2026 by Innovation In Software Corporation

MCP GALLERIES

There is an official MCP registry that serves as a public catalog of MCP servers, similar in spirit to npm or PyPI. It hosts metadata such as server descriptions, exposed tools, and connection details, enabling MCP clients and IDEs to discover and use servers more easily.

https://registry.modelcontextprotocol.io

https://github.com/modelcontextprotocol/registry

© 2026 by Innovation In Software Corporation

RETROSPECTIVE

In this module, you learned how to configure GitHub Copilot to align with your development standards and workflow. You explored Prompt Files for reusable tasks, Instruction Files for workspace policies, and Tool Sets for organizing capabilities.

You also saw how MCP (Model Context Protocol) servers extend Copilot beyond the repository by connecting it to external tools and data.

© 2026 by Innovation In Software Corporation

LAB 8 – PROMPT FILE

DESCRIPTION

Prompt files in GitHub Copilot Chat are like reusable macros written in Markdown (.prompt.md). They hold custom instructions you can run on demand, with optional metadata like title, description, model, and tools. Instead of retyping the same guidance, you save it once and trigger it whenever needed, ensuring consistent and repeatable responses.

You can store prompt files in a workspace folder (e.g., .github/prompts/).

© 2026 by Innovation In Software Corporation

SETUP

- Open VS Code
- Find and open the Lab8 folder. Within that folder, create a new folder: prompt.
- Alternately, create a new folder called prompt somewhere convenient.
- Make that directory the root directory for the workspace
- Within the prompt folder, create a new file: lab.py.

© 2026 by Innovation In Software Corporation

CREATE PROMPT.MD FILE

Using the Chat Configure menu create a new prompt file, with the Prompt Files menu item. Name the prompt *yourname*.

- Add front-matter to set agent mode
- Add the back-matter - enter this prompt or something similar.

    Add three function to sort strings as a bubble, insertion sort, and merge sort.

- Save the file

© 2026 by Innovation In Software Corporation

SAMPLE FILE

- Return to the root directory.
- Open lab.py file in this directory.
- Run your prompt as a slash command (/)
  - Select the sample file: lab.py. Place the cursor in the file.
  - In a GitHub Copilot prompt, type /yourname to run the prompt.
  - Confirm that the three functions have been added to the source file.

© 2026 by Innovation In Software Corporation

EXECUTE

Use GitHub Copilot to create a main function to test the three functions.

Execute the program.

Great work!

© 2026 by Innovation In Software Corporation

Lab completed

Excellent

You have completed the course successfully!

AGILE METHODOLOGIES

SCRUM, USE CASES, SPRINT BACKLOG

"Develop a passion for learning."

A1

INTRODUCTION

This module shows how to turn a simple user story into precise, testable behavior. You'll start with a backlog item, write acceptance criteria, and translate them into Gherkin (Given-When-Then) scenarios so developers, testers, and stakeholders share a clear, executable understanding.

Then you'll use Copilot Chat to accelerate a tight TDD loop: generate test scaffolds from those scenarios, run failing tests, and add just enough code to pass—repeating until the feature is done.

© 2026 by Innovation In Software Corporation

AGILE METHODOLOGY

Agile methodology is a flexible approach to project management and software development that delivers value in small, usable increments. Instead of long cycles, Agile teams release features frequently, gather feedback, and adapt quickly to change. It emphasizes collaboration, customer involvement, and continuous improvement, making it ideal for fast-moving environments.

Agile roles support teamwork and accountability. The Product Owner prioritizes the backlog and defines value, while the Scrum Master guides Agile practices and removes obstacles. The Development Team is cross-functional and self-organizing, delivering working increments each cycle. Stakeholders provide feedback to keep the product aligned with user and business needs.

© 2026 by Innovation In Software Corporation

AGILE METHODOLOGY

Agile methodology is a flexible approach to project management and software development that delivers value in small, usable increments. Instead of long cycles, Agile teams release features frequently, gather feedback, and adapt quickly to change. It emphasizes collaboration, customer involvement, and continuous improvement, making it ideal for fast-moving environments.

Agile roles support teamwork and accountability. The Product Owner prioritizes the backlog and defines value, while the Scrum Master guides Agile practices and removes obstacles. The Development Team is cross-functional and self-organizing, delivering working increments each cycle. Stakeholders provide feedback to keep the product aligned with user and business needs.

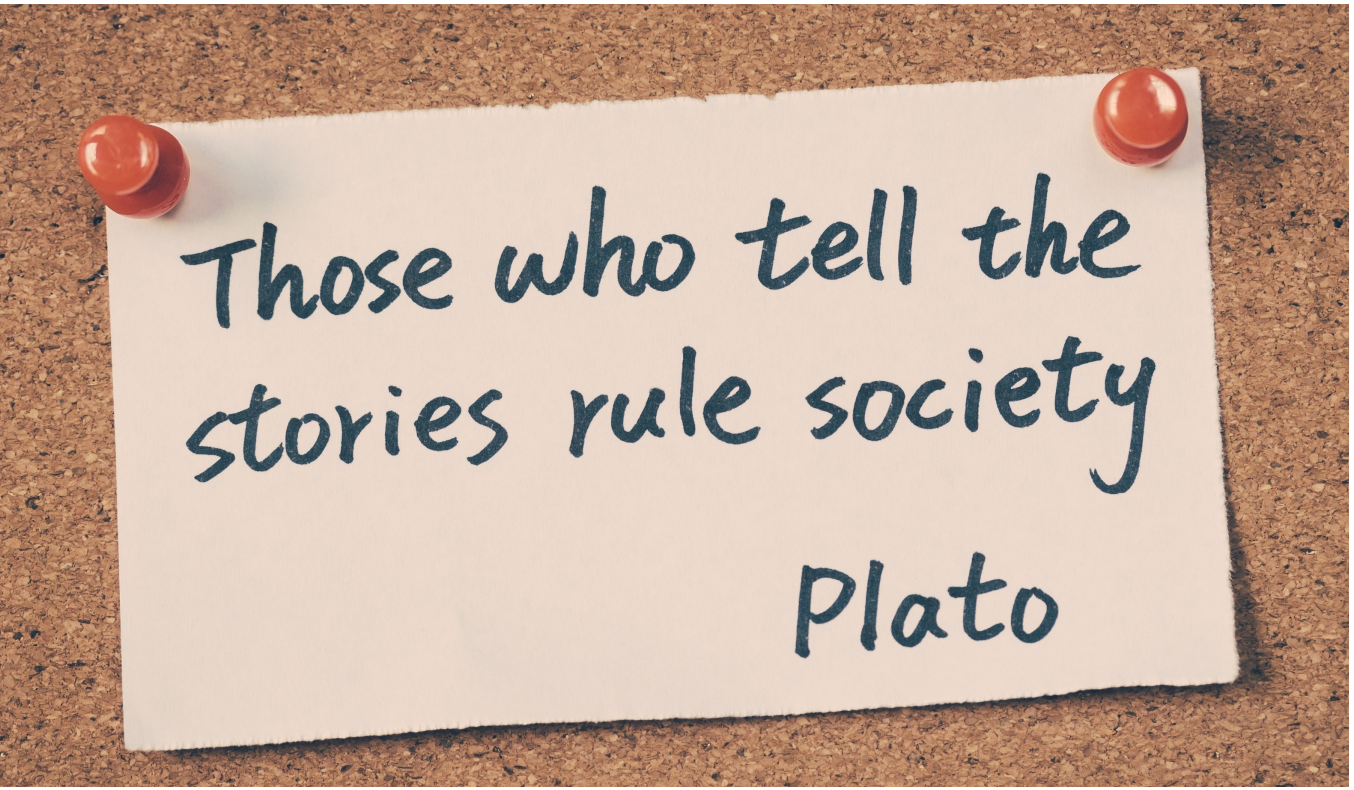
© 2026 by Innovation In Software Corporation

AGILE PRACTICES

- Scrum – Organizes work into short sprints (2–4 weeks) with defined roles and ceremonies like stand-ups, sprint planning, and retrospectives.
- Kanban – Uses a visual board to track tasks, limit work in progress, and improve flow for continuous delivery.
- Extreme Programming (XP) – Focuses on technical practices like TDD, pair programming, and continuous integration to improve code quality.
- SAFe (Scaled Agile Framework) – Extends Agile principles across large organizations, coordinating multiple teams through alignment, collaboration, and program-level planning.

© 2026 by Innovation In Software Corporation

USER STORIES



A use case in describes how a user interacts with a system to achieve a goal, written from the user's perspective. For example: "As a shopper, I want loyalty discounts applied at checkout so that I pay the correct price."

GWT (Given–When–Then) turns a use case into testable acceptance criteria. For the discount example: Given a shopper has a loyalty card, When they check out, Then the discount is applied. GWT makes expected behavior clear and testable for developers, testers, and stakeholders.

© 2026 by Innovation In Software Corporation

GHERKIN (GWT)

Gherkin is a plain-text language used in Behavior-Driven Development (BDD) to describe how software should behave in a way that both business stakeholders and developers can understand. It uses a simple structure of Given, When, Then to turn requirements into clear, testable scenarios.

A user story defines who, what, and why; from that story you write acceptance criteria that define what “done” means, and then translate them into Gherkin scenarios. In this way, user stories provide the narrative, acceptance criteria define success, and Gherkin provides a structured syntax that can be directly automated as tests.

© 2026 by Innovation In Software Corporation

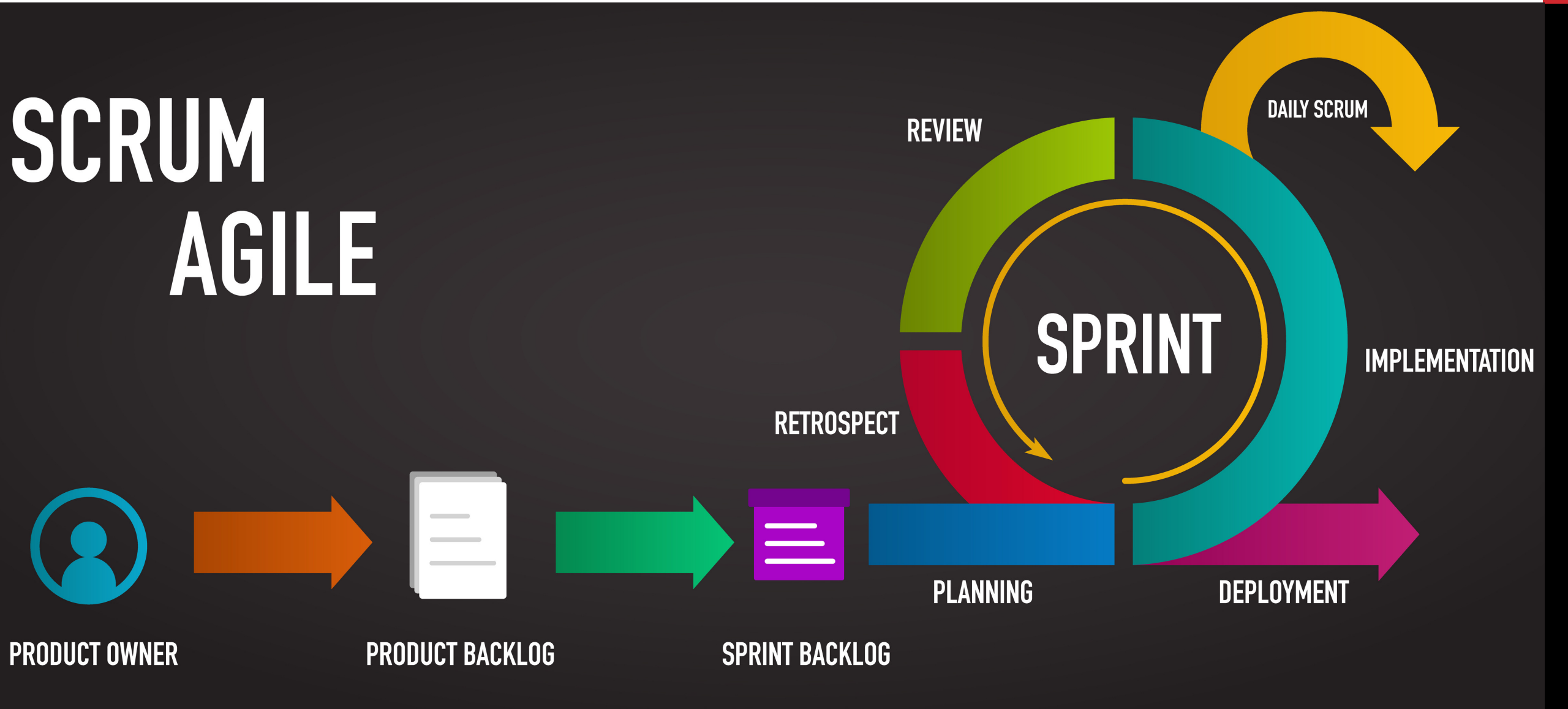
USE CASES

A use case in describes how a user interacts with a system to achieve a goal, written from the user's perspective. For example: "As a shopper, I want loyalty discounts applied at checkout so that I pay the correct price."

GWT (Given–When–Then) turns a use case into testable acceptance criteria. For the discount example: Given a shopper has a loyalty card, When they check out, Then the discount is applied. GWT makes expected behavior clear and testable for developers, testers, and stakeholders.

© 2026 by Innovation In Software Corporation

SCRUM WORKFLOW



STORY

The story used within this module is about a shopper checking out with a system that applies loyalty discounts. It illustrates how Agile teams take a customer need—ensuring the correct price is charged—and refine it step by step: starting from a backlog item, writing acceptance criteria, translating them into Gherkin scenarios, generating test scaffolds, and finally creating minimal passing code.

© 2026 by Innovation In Software Corporation

COPILOT AND AGILE

GitHub Copilot helps this workflow by turning backlog items and user stories into acceptance criteria and Gherkin scenarios. During Sprint Planning, once a team commits to a story (like applying loyalty discounts), they refine it into testable conditions. Copilot Chat can draft clear acceptance criteria and map them into Given–When–Then scenarios stored in .feature files. This makes backlog items executable and shows students how Agile artifacts evolve into specifications Copilot can use.

Copilot also generates test scaffolds and minimal code from those criteria. With slash commands like /tests or /explain, students can turn criteria into unit test skeletons, check edge cases, and then add just enough code to pass. Edit mode supports guided edits, while Agent mode handles multi-file updates. This demonstrates test-driven development—moving from stories to tests to working code—with Copilot assisting at each step.

© 2026 by Innovation In Software Corporation

(1) WORKFLOW

You want to (1) convert ACs and Gherkin (GWT syntax), (2) then build scaffolding as unit tests, and finally create (3) minimal code to make unit test pass. (3) The team can then begin to fully implementing each method iteratively.

Getting started: Paste the use cases into a use_stories.md You will add the file to the context, using #file.

Have Copilot create the acceptance criteria with this prompt:

> From #file, generate 1-3 acceptance criteria for each user story. Format your output as markdown (user_stories_a.md), with each story followed by its acceptance criteria list.

© 2026 by Innovation In Software Corporation

ACCEPTANCE CRITERIA

Acceptance criteria define when a backlog item is “done.” This is important for completing a “done” increment at the end of a Sprint.

You will use GWT to encapsulate your user stories, with “T” being derived from the acceptance criteria.

© 2026 by Innovation In Software Corporation

(2) GWT

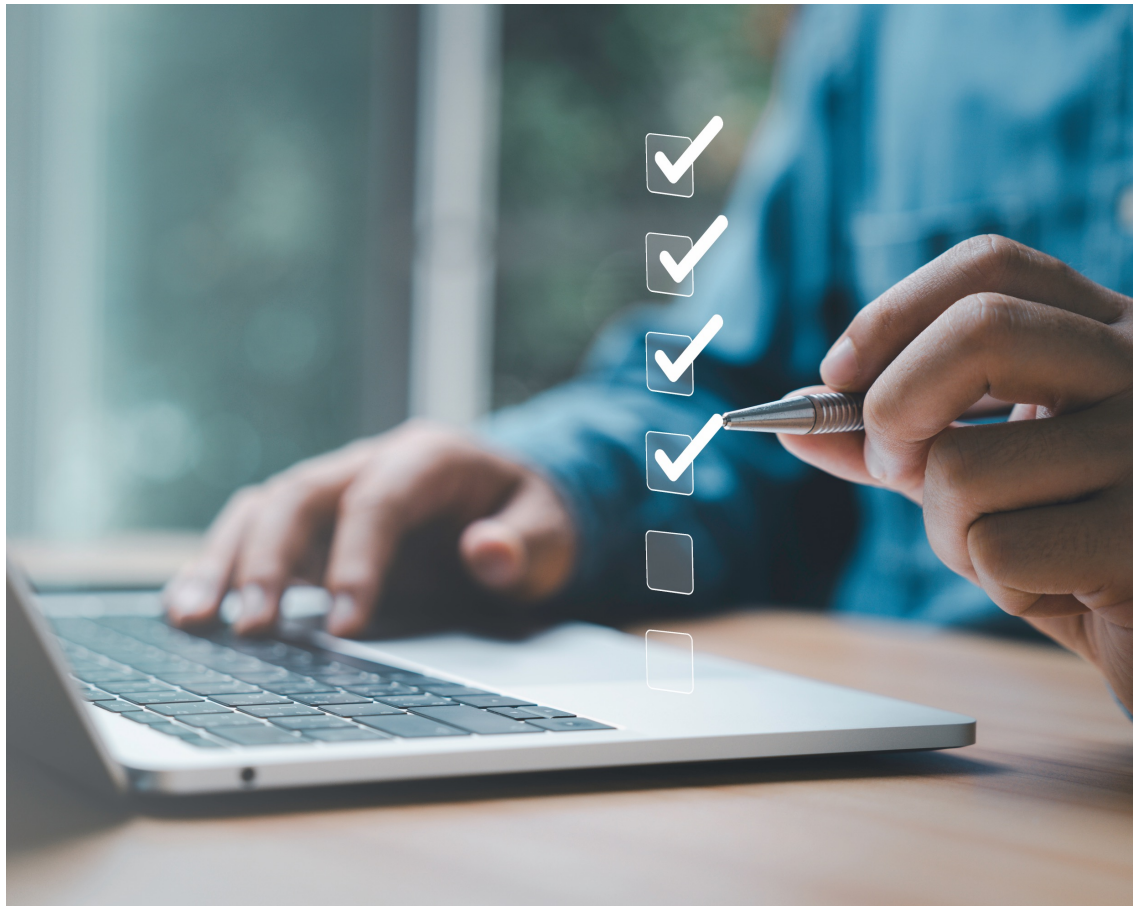
Convert user stories to GWT. Refer to the input file using #file.
You can generate unit tests from GWT using Cucumber.
However, Copilot can be used also.

Here is the prompt.

> Convert each user story in #file:user_stories_a.md into gwt syntax and save to gwt.md.

© 2026 by Innovation In Software Corporation

(3) FAILING UNIT TESTS



This step introduces Test-Driven Development (TDD): first you create failing tests (red), then you add just enough code to pass them (green). For this step, you will create the unit tests from JWT,

© 2026 by Innovation In Software Corporation

(3B) FAILING UNIT TESTS

Generate the unit test from the GWT file. Naturally, you will use Copilot instead of Cucumber. Cucumber would require agent mode. However, if using Copilot, Ask mode is sufficient.

/tests Read #file:gwt.md and create test_shop.py. One test per GWT; include one boundary and one negative test per acceptance criteria.

© 2026 by Innovation In Software Corporation

(4) MINIMAL IMPLEMENTATION

This step provides a minimal implementation of each unit, making each test pass. The status of each test will change to green. Making the test pass may require multiple passes (iteratively). For that reason, this should be done in Agent mode.

Importantly, this makes the code executable, which is required for a Sprint Increment.

© 2026 by Innovation In Software Corporation

(4B) MINIMAL IMPLEMENTATION

Provide a minimal implementation of the test suite for all tests to pass. This must be done iteratively.

Here is the prompt:

> In shop.py, minimally implement the code necessary to make the unit tests in #file:test_shop.py pass. Do this iteratively if required.

© 2026 by Innovation In Software Corporation

RETROSPECTIVE

This module ties Agile ideas to hands-on execution: you took a real user story—applying loyalty discounts at checkout—then turned it into precise acceptance criteria and Gherkin scenarios before driving a tight TDD loop from failing tests to minimal passing code. Along the way, you saw how Copilot amplifies each step—drafting criteria, mapping them to Given-When-Then, generating test scaffolds, and helping with focused edits—plus when to reach for Edit vs. Agent mode.

© 2026 by Innovation In Software Corporation

LAB A1 – ACCOUNTS RECEIVABLE

You will work with Accounts Receivable (AR) user stories such as generating invoices, tracking payments, and sending overdue notices. These stories provide the starting point for turning business needs into testable requirements.

DESCRIPTION

You will use GitHub Copilot to convert user stories into acceptance criteria, then into Given–When–Then (GWT) scenarios. From these, Copilot can generate test scaffold. Finally, you will ask Copilot for a minimal implementation to make tests pass. This end-to-end flow—story → criteria → GWT → tests → code—shows how Agile practices quickly turn AR requirements into working, validated features.

© 2026 by Innovation In Software Corporation

USER STORIES TO GWT

1. Set up

   Open invoice_use_case.md (user stories).

2. Use Copilot to generate Acceptance Criteria

   Prompt Copilot to draft clear, testable ACs for each story. Save as invoice_use_case_ac.md.

3. Translate ACs into GWT (Given–When–Then)

   Prompt Copilot to convert the ACs into concise scenarios. Save as GWT.md.

© 2026 by Innovation In Software Corporation

CREATE UNIT TESTS

4. Generate unit tests from GWT

Prompt Copilot to create Python unittest tests based on GWT.md. Save as test_accounts_receivable.py.

© 2026 by Innovation In Software Corporation

IMPLEMENT AND TEST

5. Implement minimal production code

   Prompt Copilot to generate the smallest changes in your app module (e.g., accounts_receivable.py) to make the

   tests pass, iterating as needed.

6. Run the test suite

   python -m unittest discover -v

© 2026 by Innovation In Software Corporation

Lab completed

MANAGING GITHUB REPOSITORIES

COMMIT AND PULL REQUESTS

"Develop a passion for learning."

INTRODUCTION

The presentation explains how GitHub Copilot Chat goes beyond ghost text suggestions to help with repository management. It emphasizes Copilot's role in automating and improving routine developer tasks such as writing commit messages, generating pull request descriptions, suggesting branch names, documenting repositories, and supporting code reviews

© 2026 by Innovation In Software Corporation

REPOSITORY MANAGEMENT

1. Copilot is more than just inline code completion—it supports repo management.

2. Works in VS Code, JetBrains IDEs, Neovim, and directly on GitHub.com.

3. Uses context from code and project history to suggest:

   - Commit messages and branch names
   - Pull request descriptions and review comments
   - Repository documentation and GitHub Actions

4. Benefits: faster collaboration, fewer mistakes, consistent team practices.

© 2026 by Innovation In Software Corporation

EXAMPLE

GitHub Copilot prompt:

> "Summarize this commit for me."

Copilot Suggestion:

> Refactor user authentication logic; extracted JWT validation into separate module for reusability.

Saves time writing context-heavy commit notes while keeping history meaningful.

© 2026 by Innovation In Software Corporation

COMMITS

Writing clear, consistent commit messages is often neglected.

- Copilot automatically analyzes the diff and proposes meaningful summaries.
- Helps enforce team-wide commit conventions (e.g., Conventional Commits).
- Reduces “bad commits” like update file or fix stuff.
- Especially useful in fast-moving teams where speed but quality is important.

© 2026 by Innovation In Software Corporation

EXAMPLE

- Changed Code: Added input validation to form handler.

- Copilot Suggestion:

  Add input validation for form handler to prevent invalid submissions.

Concise, descriptive, and instantly reusable!

© 2026 by Innovation In Software Corporation

BEST PRACTICES

- Always review Copilot’s suggestion for accuracy.
- Adapt to your team’s standards (Conventional Commits, Jira references, etc.).
- Add extra context if change affects multiple systems.
- Copilot = drafting assistant, not a replacement for human judgment.
- Encourages better commit hygiene across the project.

© 2026 by Innovation In Software Corporation

BEST PRACTICES - EXAMPLE

- Prompt: “Write a commit message using conventional commits style.”

- Copilot Output:

  fix(auth): correct JWT expiration handling causing login failures

© 2026 by Innovation In Software Corporation

PR

1. Copilot generates PR descriptions summarizing key changes.

2. Can include:
   - High-level purpose of the PR
   - List of modified files or features
   - Risk/impact assessment

3. Improves review efficiency by giving reviewers the “big picture.”

4. Avoids poorly written PRs like “Added stuff”

© 2026 by Innovation In Software Corporation

EXAMPLE

Prompt: “Generate a PR description for these changes.”

Copilot Suggestion:

Summary
- Added input validation to user form handler
- Refactored validation into utils module
- Updated unit tests to cover edge cases

Impact
Prevents invalid user data from reaching backend

Well-structured, follows GitHub PR template style.

© 2026 by Innovation In Software Corporation

IMPROVING CODE REVIEWS

- Copilot explains complex diffs in plain English.
- Can suggest inline comments for readability or maintainability.
- Proposes refactor opportunities before review is even requested.
- Helps newer team members understand senior-level code faster.
- Keeps reviews focused on correctness, not wording.

© 2026 by Innovation In Software Corporation

EXAMPLE

Code Snippet:

```
if(user.role == 'admin' ||
    user.permissions.includes('edit')){
   allowEdit();
}
```

Copilot Suggestion for PR comment:

"Consider extracting permission logic into a utility function to improve readability and testability."

© 2026 by Innovation In Software Corporation

BRANCH NAME SUGGESTIONS

- Encourages consistent naming (e.g., feature/, bugfix/, chore/).
- Saves time thinking of names during development rush.
- Avoids ad-hoc branches like don-test1 or temp-stuff.
- Consistency improves searchability and automation hooks.

© 2026 by Innovation In Software Corporation

EXAMPLE

Prompt: “Suggest a branch name for adding payment retries.”

Copilot Suggestion:

feature/payment-retry-mechanism

Matches convention: easy for CI/CD pipelines to pick up.

© 2026 by Innovation In Software Corporation

REPOSITORY DOCUMENTATION

1. Copilot can draft and update:
    - README.md (usage, setup, contribution)
    - Contributing.md and issue/PR templates
    - API usage guides and inline docs
2. Keeps documentation aligned with evolving codebase.
3. Reduces outdated docs, one of the top developer frustrations.

© 2026 by Innovation In Software Corporation

EXAMPLE

Prompt: “Generate README instructions for running the app.”

Copilot Output:

Running the App

1. Clone the repository
2. Install dependencies with `npm install`
3. Start the server using `npm run start`
4. Access the app at http://localhost:3000

© 2026 by Innovation In Software Corporation

RETROSPECTIVE

This module demonstrated how GitHub Copilot Chat enhances repository management by supporting commits, pull requests, reviews, and documentation. We saw how Copilot can generate meaningful commit messages, structured PR descriptions, and even suggest improvements during code review.

© 2026 by Innovation In Software Corporation